

IMPERIAL

IMPERIAL COLLEGE LONDON

IMPERIAL-X

Scalable Bayesian Uncertainty Quantification using Thermodynamic Co-Processors

Author:
Tim-Wei Schöler

Supervisor:
Dr. Andrew Duncan

Submitted in partial fulfillment of the requirements for the MSc degree in MSc AI
Applications and Innovation of Imperial College London

September 2026

Abstract

Bayesian uncertainty quantification is one of the established methods for trustworthy predictions from deep neural networks, but often requires costly sampling from a high-dimensional posterior. Thermodynamic hardware promises to remove this cost by leveraging thermal fluctuations in a physical relaxation process to sample from its own Boltzmann distribution, and therefore promises to be a natural accelerator of Bayesian uncertainty quantification. However, a scalable approach for this on discrete thermodynamic hardware has not yet been demonstrated.

This project developed the EBM-gated adapter mixture model, a hybrid architecture between conventional and thermodynamic computing that confines all randomness of a classifier to a binary gate vector over low-rank corrections on a frozen backbone model. The binary gates are drawn from an input-conditioned Boltzmann machine prior, whose Gibbs update is the operation a discrete thermodynamic sampling unit performs natively. The architecture was evaluated on artificial tasks as well as on Fashion-MNIST and CIFAR-10, and was compared against the baseline methods mean-field variational inference and MC dropout.

In distribution and under distribution shift, the architecture provides accuracy, calibration and quantified uncertainty at the same level as the established baseline methods, which shows its competitiveness on real data. Under the configurations studied, the results further indicate that around 80 % of the total GPU floating-point operations per training epoch belong to the stochastic sampling process, which can be offloaded to the thermodynamic hardware and which it presumably performs more energy-efficiently than a GPU does. While an efficiency gain of the distributed workload on conventional and thermodynamic hardware cannot be measured, as all experiments were conducted in simulation, the analysis gives an indication that the architecture is a scalable design for Bayesian uncertainty quantification using discrete thermodynamic co-processors.

This project contributes the architectural framework, a configurable parameter optimisation pipeline and an in-depth analysis of the observed results, and thereby lays the groundwork for future work in this area.

Acknowledgements

I would like to thank my supervisor, Dr. Andrew Duncan, for his excellent guidance and the freedom he gave me in equal measure while I pursued this research topic. His ideas and feedback on the architecture and on the shape of the experiments improved this work considerably.

I gratefully acknowledge the Imperial College Research Computing Service (<http://doi.org/10.14469/hpc/2232>), whose resources made the scale of the experimental study in this project possible.

Parts of this work were supported by AI-based tools, which were used for code implementation and review, literature research and language editing under my own review; all research decisions and analyses are my own.

Contents

1	Introduction	2
2	Research Objectives	3
3	Background and Literature Review	4
3.1	Uncertainty Quantification Methods in Deep Learning	4
3.1.1	Bayesian Methods	5
3.1.2	Score Function Estimator	10
3.1.3	Evaluation Metrics	10
3.2	Energy-Based Models	12
3.2.1	Energy Functions and the Boltzmann Form	12
3.2.2	Bayesian Posterior as an Energy-Based Model	12
3.2.3	Learning Energy-Based Models	13
3.2.4	Boltzmann Machines and Gibbs Sampling	13
3.3	Thermodynamic Computing	14
3.3.1	Concept	15
3.3.2	Thermodynamic Computing Approaches for Uncertainty Quantification	15
3.3.3	Extropic's Thermodynamic Sampling Unit	17
3.4	Research Gap	18
4	Architectural Framework	19
4.1	Architecture and Structure	19
4.1.1	Likelihood, Prior and Recognition Model	20
4.1.2	Predictive Distribution and Uncertainty	23
4.2	Training Objective	24
4.3	Gradients of the Trainable Parameters	26
4.3.1	Adapter Gradients	26
4.3.2	Prior Gradients	27
4.3.3	Recognition Model Gradients	28
4.4	Variational Families of the Recognition Model	29
4.4.1	Mean-Field and Tree-Structured Families	29
4.4.2	Energy-Based Recognition Model	31
4.4.3	Variational Families Summary	32
4.5	Training Algorithm	32
4.5.1	Stabilisation Techniques	33
4.6	Handoff between Conventional and Thermodynamic Hardware	33
5	Experimental Design	35
5.1	Parameter Optimisation Pipeline	35
5.2	Datasets	36
5.3	Baseline Models	36
5.4	Experiments	37

6 Experiments	38
6.1 E1: Groundwork and Feasibility Study on Artificial Data	38
6.1.1 E1.1: Blob Task	38
6.1.2 E1.2: Routing Task	42
6.1.3 E1.3: Posterior Fidelity of the Recognition Families	44
6.1.4 Interim Key Findings	48
6.2 E2: Competitiveness on Fashion-MNIST and CIFAR-10	49
6.2.1 Setup	49
6.2.2 Results	50
6.2.3 Interim Key Findings	52
6.3 E3: Operating Regime and Sensitivity	53
6.3.1 E3.1: Base Map W_0 Sensitivity	53
6.3.2 E3.2: Uncertainty Behaviour under Distribution Shift	55
6.3.3 E3.3: Capacity Scaling Sensitivity	59
6.3.4 Interim Key Findings	62
6.4 E4: Energy-Based Recognition Model	63
6.4.1 E4.1: Direct Comparison of Tree and EBM Recognition Model	63
6.4.2 E4.2: Gibbs Sweep Sensitivity	64
6.4.3 Interim Key Findings	66
6.5 E5: FLOPs Model and Scalability	66
6.5.1 Setup	67
6.5.2 Results	67
7 Limitations	69
8 Conclusion and Outlook	70
Bibliography	72
A Appendix	77

Chapter 1

Introduction

Deep neural networks are increasingly deployed in high-stakes settings such as medical diagnosis and cybersecurity. In these settings, it is often not enough for a model to be accurate on average, but it must also indicate how certain it is about its prediction. Therefore, it has been argued that model uncertainty must be quantified before the model is trusted with decisions [1, 2].

A standard neural network returns deterministic outputs and therefore does not provide such measures. Moreover, it has been shown that, particularly with growing network depth and on inputs unlike the model’s training data, modern architectures are often systematically overconfident or poorly calibrated [3, 4, 5]. Uncertainty Quantification (UQ) addresses these issues by equipping a model with an estimate of its confidence in its prediction, which is essential for out-of-distribution detection, robustness under distribution shift and safety-critical decision-making [6, 7].

The Bayesian framework offers a principled route to UQ by treating the model weights as random variables and inferring their posterior given the data, yielding a probabilistic distribution over its predictions [8, 9]. However, for a modern network with millions of parameters this posterior is intractable and therefore not possible to compute. While asymptotically exact but poorly scaling methods exist [10, 11], cheaper approximation methods reduce computational cost at the expense of fidelity [12, 13, 14]. Either way, the computational burden of drawing samples from a high-dimensional distribution is the central obstacle that limits rigorous Bayesian UQ precisely where trustworthy uncertainty matters most.

Thermodynamic computing has recently emerged as a physics-based paradigm. Instead of suppressing the thermal fluctuations of a physical substrate and then simulating stochastic dynamics algorithmically, thermodynamic hardware controls physical stochastic relaxation in its system and lets the fluctuations do the sampling by themselves [15, 16]. Therefore, thermodynamic computing promises to be a natural accelerator of Bayesian UQ. Thermodynamic approaches based on continuous variables have been developed for Bayesian inference [17] and physically demonstrated [18], whereas on the discrete side Bayesian inference has so far been shown only on very specialised magnetic substrates that are difficult to scale [19, 20].

Consequently, the model architecture presented in this work¹ follows the characteristics of a discrete thermodynamic sampling unit (TSU) developed by the company Extropic, which samples efficiently from coupled distributions over binary variables and is based on a scalable hardware platform [21].

¹The full implementation is publicly available at https://github.com/Tim-Wei-28/Thermodynamic_UQ.

Chapter 2

Research Objectives

The architecture developed in this project is a novel hybrid approach using conventional and discrete thermodynamic hardware for uncertainty quantification. Therefore, the first objective is to demonstrate the general feasibility of this approach. Once feasibility has been shown, the second objective is to evaluate its competitiveness with established UQ methods in terms of accuracy, calibration and predictive uncertainty on real datasets. These results build the foundation for the third objective, which is to assess the architecture’s scalability.

Specifically, the key questions of this research project are:

1. **Feasibility:** Does the learning mechanism work? Can a discrete energy-based model act as the stochastic component of a neural classifier, and how much structure does its gate posterior need to carry?
2. **Competitiveness:** How does the resulting accuracy, calibration and quantified uncertainty compare with established UQ methods on real data, in distribution and under distribution shift?
3. **Viability and Scalability:** How does the cost of the architecture grow with its capacity and which share of that cost could a thermodynamic sampling unit absorb?

It is important to note that all experiments were conducted in simulation on conventional GPU hardware, as the project has no access to a physical thermodynamic chip.

The report is structured as follows: After Chapter 3 reviews the classical UQ methods and explains the fundamentals and current research state of thermodynamic computing, Chapter 4 explains the developed architectural framework and its training mechanism. Then, Chapter 5 describes the experimental pipeline and design. Chapter 6 is the main chapter that presents the conducted experiments and analyses their results. Lastly, Chapter 7 discusses the limitations of the architecture before Chapter 8 summarises the project and gives an outlook for future work.

Chapter 3

Background and Literature Review

This chapter lays the foundations on which the rest of the report rests. Chapter 3.1 gives an introduction to uncertainty quantification by explaining Bayesian UQ methods, the gradient estimator required for this project and the used evaluation metrics. Chapter 3.2 then follows by building the bridge between UQ and thermodynamic computing through an energy-based representation. Next, Chapter 3.3 introduces the thermodynamic computing paradigm, the current research state of its use for UQ, and explicitly explains Extropic’s thermodynamic sampling unit. Chapter 3.4 states the gap that this project tries to tackle.

3.1 Uncertainty Quantification Methods in Deep Learning

Uncertainty quantification asks not only what a model predicts, but how far that prediction can be trusted. It is standard to separate two qualitatively different sources of predictive uncertainty [6, 7]. *Aleatoric* uncertainty arises from noise inherent in the data-generating process, such as label noise or genuine overlap between classes, and is irreducible. That means that more data of the same kind does not remove it. *Epistemic* uncertainty arises from the model and its parameters, and reflects that a finite training set leaves many weight configurations compatible with the data. It is reducible in principle because further observations narrow the set of configurations that remain plausible. Therefore, only the epistemic part can indicate that an input lies where the model has no evidence. The total predictive uncertainty is defined as the sum of the aleatoric and epistemic uncertainty.

This formal definition of uncertainty can be transferred precisely to a neural network, where $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ denotes its training set. A neural network with weights $\mathbf{w}$ defines a conditional likelihood $p(y | \mathbf{x}, \mathbf{w})$, which is categorical through a softmax for classification and Gaussian for regression. Maximum-likelihood training returns a single point estimate $\hat{\mathbf{w}}$ and predicts with $p(y^* | \mathbf{x}^*, \hat{\mathbf{w}})$. The spread of that distribution expresses aleatoric noise, but nothing in it accounts for the other weight configurations that explain the data equally well. The epistemic component is therefore not underestimated but structurally absent, which Gawlikowski et al. identify as the root cause of overconfidence in deep networks [22].

The consequences have been studied by many researchers. Guo et al. showed that modern architectures are systematically miscalibrated and that the effect grows with depth [3]. Networks may assign high confidence to inputs carrying no recognisable class content at all [4] and under distribution shift their calibration degrades faster than their accuracy [5]. Hein et al. give the sharpest form of the problem by observing that, for a ReLU network, almost any input scaled far enough from the training data is classified with arbitrarily high confidence, which is a property of the architecture that no quantity of training data removes [23].

Uncertainty quantification is the research area dealing with these issues and Gawlikowski et al. developed a typical categorisation of the resulting UQ method landscape by segmenting established methods in four categories based on where the randomness that produces

a spread of predictions is placed [22], which is shown in Appendix A.1. Since this project belongs to the first of these, the Bayesian methods, the following section develops the framework that defines them.

3.1.1 Bayesian Methods

The Bayesian framework produces a spread of predictions at its source by treating the weights themselves as random variables [8, 9]. It rests on four ingredients [24], each of which recurs throughout this report:

1. **The likelihood** $p(y | \mathbf{x}, \mathbf{w})$, which extends to the full dataset under the usual independence assumption as

$$p(\mathcal{D} | \mathbf{w}) = \prod_{i=1}^N p(y_i | \mathbf{x}_i, \mathbf{w}). \quad (3.1)$$

2. **The prior** $p(\mathbf{w})$, which encodes beliefs about plausible weights before any data is observed and is a modelling choice.

3. **The posterior**, obtained by combining the two through Bayes' theorem,

$$p(\mathbf{w} | \mathcal{D}) = \frac{p(\mathcal{D} | \mathbf{w}) p(\mathbf{w})}{p(\mathcal{D})}, \quad p(\mathcal{D}) = \int p(\mathcal{D} | \mathbf{w}) p(\mathbf{w}) d\mathbf{w}, \quad (3.2)$$

which assigns density to every weight configuration in proportion to how well it explains the data. Its spread is the epistemic uncertainty [9].

4. **The posterior predictive distribution**, obtained by marginalising the likelihood over that posterior,

$$p(y^* | \mathbf{x}^*, \mathcal{D}) = \int p(y^* | \mathbf{x}^*, \mathbf{w}) p(\mathbf{w} | \mathcal{D}) d\mathbf{w}. \quad (3.3)$$

Equation (3.3) is the ingredient this report returns to most often. A Bayesian prediction is not the output of one network but an average over all of them, each weighted by its posterior plausibility. Consequently, predictive uncertainty is high wherever plausible weights disagree. Every method reviewed in this section is, at bottom, a different way of producing the weights that this average runs over.

For deep networks the posterior is a high-dimensional distribution over millions of parameters, and both the evidence $p(\mathcal{D})$ and the predictive integral are analytically intractable [13, 8]. Practical Bayesian deep learning therefore replaces the exact posterior by a distribution $q(\mathbf{w})$ that can be sampled, and estimates Equation (3.3) by Monte Carlo averaging over S draws:

$$p(y^* | \mathbf{x}^*, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S p(y^* | \mathbf{x}^*, \mathbf{w}^{(s)}), \quad \mathbf{w}^{(s)} \sim q(\mathbf{w}). \quad (3.4)$$

Equation (3.4) is the common interface of everything that follows. The two methods reviewed below differ in what q is and in how its samples are produced, but they both enter Equation (3.4) to create the predictive distribution.

The same S draws can be used to assess the epistemic and aleatoric uncertainty by measuring the entropy of the predictive distribution (total uncertainty) and the entropy of the individual networks. The difference is the mutual information between label and weights,

$$\underbrace{I(y^*; \mathbf{w} | \mathbf{x}^*, \mathcal{D})}_{\text{epistemic}} = \underbrace{H[p(y^* | \mathbf{x}^*, \mathcal{D})]}_{\text{total}} - \underbrace{\mathbb{E}_{p(\mathbf{w} | \mathcal{D})}[H[p(y^* | \mathbf{x}^*, \mathbf{w})]]}_{\text{aleatoric}}. \quad (3.5)$$

It is also known as the BALD score [25] and is used in this project as the estimator for the epistemic uncertainty. It is large where the plausible networks each predict confidently but disagree with one another and small both where they agree and where every one of them is equally unsure. This is exactly the behaviour required from the epistemic uncertainty.

What separates the method families is therefore the trade-off each strikes between fidelity to the true posterior and the cost of drawing from it. Asymptotically exact schemes, like Markov Chain Monte Carlo exist, but scale poorly to deep networks [10, 11]. On the other hand, cheaper families buy tractability by restricting the form of q at the price of an approximation whose error is difficult to bound [12]. The two specific methods reviewed next are established methods in the UQ field and will be used as the baseline methods which the developed architecture will be compared against.

3.1.1.1 Variational Inference

Variational inference (VI) recasts Bayesian inference as an optimisation problem [26, 12]. A family $\mathcal{Q}$ of tractable distributions $q_\theta(\mathbf{w})$ is fixed in advance and inference consists of searching that family for the member closest to the true posterior $p(\mathbf{w} | \mathcal{D})$.

Evidence Lower Bound

Closeness is measured by the Kullback–Leibler divergence $\text{KL}(q_\theta(\mathbf{w}) \| p(\mathbf{w} | \mathcal{D}))$, which cannot be minimised directly because it contains the intractable evidence $p(\mathcal{D})$. The way around this is an identity of the log evidence derived in Appendix A.2. For any q_θ , the log evidence decomposes as

$$\log p(\mathcal{D}) = \mathcal{L}(\theta) + \text{KL}(q_\theta(\mathbf{w}) \| p(\mathbf{w} | \mathcal{D})), \quad \text{where} \quad (3.6)$$

$$\mathcal{L}(\theta) = \underbrace{\mathbb{E}_{q_\theta}[\log p(\mathcal{D} | \mathbf{w})]}_{\text{expected data fit}} - \underbrace{\text{KL}(q_\theta(\mathbf{w}) \| p(\mathbf{w}))}_{\text{complexity penalty}}. \quad (3.7)$$

$\text{KL}(q_\theta(\mathbf{w}) \| p(\mathbf{w} | \mathcal{D}))$ is called the *reverse KL* and since it is non-negative, $\mathcal{L}(\theta)$ can never exceed $\log p(\mathcal{D})$, which makes it a lower bound on the log evidence. Equation (3.7) is therefore called the *evidence lower bound* or *ELBO*. Rearranging Equation (3.6) gives the more useful reading:

$$\text{KL}(q_\theta(\mathbf{w}) \| p(\mathbf{w} | \mathcal{D})) = \log p(\mathcal{D}) - \mathcal{L}(\theta). \quad (3.8)$$

The evidence depends on the data, the likelihood and the prior, none of which θ touches, so the first term on the right is a constant. The two remaining quantities must therefore trade off exactly. Every nat gained in the ELBO is a nat removed from the divergence. In general,

maximising Equation (3.7) is consequently equivalent to minimising the KL-divergence of Equation (3.8) on a computable objective, since the ELBO requires only the likelihood, the prior and q_θ . The intractable term has been isolated into a constant the optimisation never has to evaluate. The fitted q_{θ^*} then replaces the posterior in Equation (3.4), from which it is cheap to draw.

Equation (3.8) also states the approximation error of this method exactly as the gap between the ELBO and the log evidence. That means that the error is well defined, but it is not computable since evaluating it would require the true posterior. Consequently, a converged ELBO certifies that the best member of $\mathcal{Q}$ has been found, but says nothing about how good that member is. In practice, the two terms of Equation (3.7) are also frequently reweighted, replacing the complexity penalty by $\beta \text{KL}(\cdot \parallel \cdot)$ with $\beta \neq 1$. This departs from the bound because at $\beta \neq 1$ the objective is no longer a lower bound on $\log p(\mathcal{D})$, and the equivalence of Equation (3.8) no longer holds. What it supplies instead is a single knob that trades fit against regularisation. It is the knob through which most implementations control the accuracy–calibration balance and it is used in this project as well.

Gradient of the ELBO

Training means maximising the ELBO by gradient ascent, so the quantity required is $\nabla_\theta \mathcal{L}(\theta)$. Equation (3.7) presents the ELBO as two terms, but the complexity penalty, the Kullback–Leibler divergence between q_θ and the prior, is itself an expectation under q_θ , so the two collapse into one. Restating the ELBO with latent variables z in place of the weights w , which is the form this work uses, gives

$$\begin{aligned} \mathcal{L}(\theta) &= \underbrace{\mathbb{E}_{q_\theta}[\log p(\mathcal{D} \mid z)]}_{\text{expected data fit}} - \underbrace{\text{KL}(q_\theta(z) \parallel p(z))}_{\text{complexity penalty}} \\ &= \mathbb{E}_{q_\theta}[\log p(\mathcal{D} \mid z)] - \underbrace{\mathbb{E}_{q_\theta}[\log q_\theta(z) - \log p(z)]}_{\text{the same penalty, as an expectation under } q_\theta} \\ &= \mathbb{E}_{q_\theta(z)}[f_\theta(z)], \quad f_\theta(z) = \log p(\mathcal{D} \mid z) + \log p(z) - \log q_\theta(z). \end{aligned} \quad (3.9)$$

What has to be differentiated is therefore an expectation taken under the very distribution being optimised. The gradient splits into two terms, as Appendix A.3 derives,

$$\nabla_\theta \mathbb{E}_{q_\theta(z)}[f_\theta(z)] = \underbrace{\sum_z f_\theta(z) \nabla_\theta q_\theta(z)}_{\text{distributional term}} + \mathbb{E}_{q_\theta(z)}[\nabla_\theta f_\theta(z)], \quad (3.10)$$

where the sum is written for a discrete z and becomes an integral otherwise. The second term is an ordinary expectation of a gradient, which samples estimate without bias. However, in the first term, changing θ changes which z are likely to be drawn and not only what f_θ returns for a given draw. This dependence is the main difficulty for calculating the gradient.

For continuous variables it can be removed by applying the *reparameterisation trick* [27], but the gates of this work are binary, which rules this route out and motivates the score function estimator reviewed in Chapter 3.1.2.

Variational Families

The choice of $\mathcal{Q}$ sets the floor on the achievable divergence, since no amount of optimisation can reach a posterior structure the family cannot express, and it simultaneously sets the cost because the ELBO is only useful if expectations and entropies under q_θ can be computed or sampled cheaply. The family is the one genuinely free design choice in VI.

Mean-field VI is the simplest answer assuming the weights are mutually independent, so that the joint factorises into a product of per-weight distributions,

$$q_\theta(\mathbf{w}) = \prod_i q_{\theta_i}(w_i), \quad q_{\theta_i}(w_i) = \mathcal{N}(w_i; \mu_i, \sigma_i^2), \quad (3.11)$$

each factor typically Gaussian [12]. The variational parameters are learned by maximising the ELBO (see Equation (3.7)) through *Bayes by Backprop* [13], which is the reparameterisation trick applied to a Gaussian, $\mathbf{w} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The cost is only a constant factor, since each weight carries two parameters (μ_i, σ_i) where a deterministic network carries one. Mean-field's limitation lies in its diagonal covariance by construction, which is why no correlation between weights can be represented when using this family. While weights within a network layer usually compensate for one another, a factorised family cannot represent that dependence, which is the reason why the uncertainty it reports is smaller than the posterior's [12].

Structured variational families relax the factorisation as far as tractability allows to a tree structure, which yields a strictly tighter bound than full factorisation at modest additional cost [28]. If every variable is conditioned on at most one parent,

$$q_\theta(\mathbf{z}) = \prod_k q_\theta(z_k | z_{\text{pa}(k)}), \quad (3.12)$$

sampling, normalisation and entropy remain linear in the number of variables, while pairwise dependence along the edges of the tree is represented exactly. However, the question remains which tree topology is the best for the task. Chow and Liu showed that minimising the divergence to a target distribution over all tree-structured approximations is equivalent to maximising the sum of pairwise mutual informations along the tree's edges. That means that the optimal tree is the maximum-weight spanning tree of the graph, using the mutual informations [29] as the edge weights. The topology can thus be learned from the same statistics used to fit the parameters rather than fixed by hand.

Amortised Inference

The families above are single distributions over the global weights. A latent variable that describes an individual observation instead carries an index, giving a separate posterior $p(z_n | \mathbf{x}_n)$ for every data point. These variables are called *local* and are used in this project.

Fitting free variational parameters per observation is impractical, since their number grows with the dataset. Therefore, amortised inference replaces the free variational parameters with a function that computes them. The function is called a *recognition model* with parameters ϕ , shared across all observations, which maps the input $\mathbf{x}$ to the variational parameters of the corresponding latent posterior, written as $q_\phi(\mathbf{z} | \mathbf{x})$. What it returns is fixed by the family, K probabilities for a mean-field family over K gates and the edge parameters in addition

for a tree. It is trained jointly with the model on the ELBO summed over the dataset, so inference for an unseen input is a single forward pass [27]. The price is the *amortisation gap*, since one shared ϕ must serve every input and the parameters it produces for any particular one fall short of what optimising that observation alone would achieve.

3.1.1.2 MC Dropout

Dropout was originally introduced as a regulariser that randomly deactivates a fraction of units at each training step to prevent overfitting [30]. Gal and Ghahramani observed that the same randomness, left switched on after training, turns a single deterministic network into a source of varied predictions [14]. The main motivation is the computational cost as it does not ask for any changes to the architecture, the training procedure and does not add additional parameters. Consequently, it has become an established uncertainty quantification method.

The mechanism is to keep dropout active at test time and average over repeated forward passes, where each pass samples a fresh binary mask and is therefore a different thinned network. Each unit is retained with probability $1 - p$, where the dropout rate p is a hyperparameter that is fixed by hand. Gal and Ghahramani state that MC dropout is not a method family of its own but variational inference with a Bernoulli variational family, drawing each weight matrix as $\mathbf{W} = \mathbf{M} \text{diag}(\mathbf{z})$ with $z_j \sim \text{Bern}(1 - p)$ drawn independently for every unit [14]. This is why Figure A.1 places it beside mean-field VI and not in a category of its own. However, it is important to note that the derivation rests on particular assumptions about the prior and the network, and its status as genuine Bayesian inference is not universally accepted [31, 32].

3.1.1.3 Bayesian Last Layer

The idea behind Bayesian last layers is to keep the weights of every layer in a neural network deterministic and only introduce a distribution over the weights of the last layer. To do so, weights are split into $\mathbf{w}_{\text{feat}}$ and $\mathbf{w}_L$, where $\mathbf{w}_{\text{feat}}$ are point estimates of the feature extractor and $\mathbf{w}_L$ are the weights with an underlying probability distribution. Consequently, the predictive average of Equation (3.4) runs only over the last layer.

The main advantage of this approach is the reduction in computational cost, since the number of random variables falls to a small fraction of the parameter count and the sampling becomes correspondingly cheaper. On the other hand, this construction gives up uncertainty about the representation. The features are learned from the same data and are then treated as certain, so nothing in the predictive spread reflects the possibility that the network is attending to the wrong things. An input unlike anything seen during training that nonetheless maps into a confident region of feature space will still be predicted confidently, which bears directly on behaviour under distribution shift. The approach is thus partially and not fully Bayesian.

However, recent findings show that Bayesian last-layer approaches are competitive with fully Bayesian networks. Daxberger et al. recommend the last layer as the default across an extensive benchmark, reporting that it typically outperforms the same approximation applied to every weight, which underfits [33]. In related work, Daxberger et al. state that inference over any small and well-chosen subset of weights is sufficient for accurate predictive posteriors [34]. Sharma et al. came to a similar conclusion, proving that partially stochastic networks are universal probabilistic predictors that can match or even outperform fully

stochastic networks at lower memory cost [35]. Additionally, Kristiadi et al. show that a Gaussian posterior over the final layer alone already bounds the confidence of a ReLU network far from the training data, which is the defect Hein et al. identified in Chapter 3.1 [36, 23].

In summary, the construction therefore buys most of the benefit of a Bayesian treatment at a fraction of its cost and at the price of an uncertainty estimate that can only be as good as the features it is conditioned on.

3.1.2 Score Function Estimator

Chapter 3.1.1.1 established that the gradient of the ELBO carries the distributional term $\sum_z f_\theta(z) \nabla_\theta q_\theta(z)$ (see Equation (3.10)), which accounts for how the sampling distribution itself moves with θ . The reparameterisation trick removes it when the latent variable is continuous, but the variational family in this project is discrete. For K binary variables, the sum in the gradient of the ELBO runs over all 2^K configurations, so it grows exponentially in K and cannot be evaluated at every gradient step for every observation.

The score function estimator is one solution to this problem, which leaves the variable discrete and rewrites the distributional term instead [37]. Applying the log-derivative identity to $\nabla_\theta q_\theta(z)$ turns that sum into an expectation under q_θ itself, so that the full gradient reads

$$\nabla_\theta \mathbb{E}_{q_\theta(z)}[f_\theta(z)] = \underbrace{\mathbb{E}_{q_\theta(z)}[f_\theta(z) \nabla_\theta \log q_\theta(z)]}_{\text{distributional term, rewritten}} + \mathbb{E}_{q_\theta(z)}[\nabla_\theta f_\theta(z)]. \quad (3.13)$$

Both terms are now expectations under the same distribution, so one set of draws estimates the whole gradient. The rewriting is exact, which is why the resulting Monte-Carlo average is unbiased for every sample count. The learning signal f_θ is only evaluated and not differentiated with respect to z , and the only gradient the variational family must supply in closed form is that of $\log q_\theta$.

What the estimator gives up is low variance, since the score is scaled by the raw value of f_θ . Because the score has zero mean, subtracting any quantity that does not depend on z leaves the expectation unchanged while altering the spread of the estimate [38]. This work uses the leave-one-out baseline, which assigns to each draw the mean of the remaining ones,

$$\frac{1}{S} \sum_{s=1}^S \left(f_\theta(z^{(s)}) - b_s \right) \nabla_\theta \log q_\theta(z^{(s)}), \quad b_s = \frac{1}{S-1} \sum_{s' \neq s} f_\theta(z^{(s')}), \quad (3.14)$$

which is independent of $z^{(s)}$ by construction [39, 40]. Both steps are derived in Appendix A.4.

In summary, the score function estimator is unbiased and asks nothing of f_θ beyond being computable, but its gradient estimates carry high variance, which has to be controlled.

3.1.3 Evaluation Metrics

Since the methods output a predictive distribution, the evaluation must score the distribution and not only its mode. In the following, the evaluation set is denoted as $\{(\mathbf{x}_m, y_m)\}_{m=1}^M$ over C classes, $\hat{p}_m(c)$ is the predictive probability of class c obtained from Equation (3.4),

$\hat{y}_m = \arg \max_c \hat{p}_m(c)$ is the predicted class and $\text{conf}_m = \max_c \hat{p}_m(c)$ the confidence attached to it.

Accuracy

Accuracy provides the share of how often the predicted class is the correct one and is reported to establish that the compared models are equally capable before their uncertainty is examined. It is given as

$$\text{acc} = \frac{1}{M} \sum_{m=1}^M \mathbb{1}[\hat{y}_m = y_m]. \quad (3.15)$$

Expected calibration error

Expected calibration error (ECE) is a widely used metric [41, 3], which partitions the predictions into B equally wide confidence bins $\mathcal{B}_1, \dots, \mathcal{B}_B$ and measures the average difference between accuracy and confidence within each of the bins,

$$\text{ECE} = \sum_{b=1}^B \frac{|\mathcal{B}_b|}{M} \left| \text{acc}(\mathcal{B}_b) - \text{conf}(\mathcal{B}_b) \right|, \quad (3.16)$$

where $\text{acc}(\mathcal{B}_b)$ and $\text{conf}(\mathcal{B}_b)$ are the means of $\mathbb{1}[\hat{y}_m = y_m]$ and of conf_m over the bin. A perfectly calibrated model should be correct 80% of the time when it is 80% confident. The values ECE therefore returns range from 0 (perfect calibration) to 1 (worst possible calibration). However, ECE has to be treated carefully and must be read alongside other metrics like accuracy rather than on its own. A classifier that returns the uniform distribution on every input is confident to degree $1/C$ and correct on a fraction $1/C$ of a balanced test set, so its confidence matches its accuracy exactly and its calibration error vanishes while the model carries no information at all. A low ECE is therefore necessary for a trustworthy model but not sufficient alone.

Negative log-likelihood

The negative log-likelihood (NLL) is considered a *proper scoring rule* as it rewards accuracy and calibration together by measuring how well a model's predicted probability distributions align with observed outcomes [42]. Its penalty is unbounded, since $\log \hat{p}_m(y_m)$ diverges as the probability of the true class approaches zero. Therefore, a single confident mistake can dominate the average as it heavily penalises overconfidence when its prediction is wrong. It is computed as

$$\text{NLL} = -\frac{1}{M} \sum_{m=1}^M \log \hat{p}_m(y_m). \quad (3.17)$$

Brier score

The Brier score is another proper scoring rule and is sometimes preferred over NLL as it is strictly bounded within a range of 0 and 2. It measures the squared distance between the full predicted distribution and the one-hot label [43],

$$\text{BS} = \frac{1}{M} \sum_{m=1}^M \sum_{c=1}^C (\hat{p}_m(c) - \mathbb{1}[y_m = c])^2. \quad (3.18)$$

Since it grows only quadratically it punishes confident errors far less harshly than the NLL.

The four are reported together because none is sufficient alone. Accuracy establishes what the model gets right, the ECE isolates whether its stated confidence matches that rate, and the two proper scoring rules combine both into a single number, differing in how severely they penalise confident mistakes.

3.2 Energy-Based Models

3.2.1 Energy Functions and the Boltzmann Form

An energy-based model specifies a probability distribution indirectly, through a scalar energy function $E(\mathbf{x})$ that ranks configurations rather than through an explicitly normalised density [44]. Low-energy configurations receive high probability through the Boltzmann (or Gibbs) form

$$p(\mathbf{x}) = \frac{1}{Z} \exp(-E(\mathbf{x})), \quad Z = \sum_{\mathbf{x}} \exp(-E(\mathbf{x})), \quad (3.19)$$

so the model is an energy landscape over configuration space. Leaving E unconstrained makes the paradigm flexible as any strictly positive distribution can be written in this form by setting $E(\mathbf{x}) = -\log p(\mathbf{x})$ [45]. The partition function Z sums over every configuration and is intractable beyond the smallest models. Therefore, the normalised distribution is generally not available in closed form. Ratios of probabilities, however, do not require it,

$$\frac{p(\mathbf{x})}{p(\mathbf{x}')} = \exp(E(\mathbf{x}') - E(\mathbf{x})), \quad (3.20)$$

since Z cancels. As a consequence, the model is accessed through sampling, which needs only these energy differences.

3.2.2 Bayesian Posterior as an Energy-Based Model

Bayes' theorem, as introduced in Equation (3.2), reads

$$p(\mathbf{w} | \mathcal{D}) = \frac{p(\mathcal{D} | \mathbf{w}) p(\mathbf{w})}{p(\mathcal{D})},$$

whose numerator is the unnormalised posterior and whose denominator normalises it. This has the same shape as the Boltzmann form (see Equation (3.19)), so identifying $\exp(-E(\mathbf{w}))$ with that numerator fixes the energy as

$$U(\mathbf{w}) = -\log(p(\mathcal{D} | \mathbf{w}) p(\mathbf{w})) = -\log p(\mathcal{D} | \mathbf{w}) - \log p(\mathbf{w}). \quad (3.21)$$

Substituting this energy into the partition function of Equation (3.19), with the sum becoming an integral over the continuous weights, recovers the denominator of Bayes' theorem,

$$Z = \int \exp(-U(\mathbf{w})) d\mathbf{w} = \int p(\mathcal{D} | \mathbf{w}) p(\mathbf{w}) d\mathbf{w} = p(\mathcal{D}). \quad (3.22)$$

The Bayesian posterior is therefore an energy-based model with the posterior energy of Equation (3.21) as its energy function and the model evidence as its partition function. The intractability established in the variational inference chapter (see Chapter 3.1.1.1) and the one described above in Chapter 3.2.1 are one obstacle under two names as the partition function Z of that EBM is the evidence $p(\mathcal{D})$ in the Bayesian posterior.

3.2.3 Learning Energy-Based Models

This section develops how the parameters of an energy function are fitted and introduces the resulting sampling problem. Fitting an EBM means shaping its energy landscape so that the distribution it induces matches the distribution of the data. Because probability depends on the energy only through the Boltzmann form of Equation (3.19), what has to match is the relative depth of the landscape against the relative frequency of configurations in the data. This results in a gradient with a characteristic two-part structure,

$$\nabla_{\theta} \mathbb{E}_{\text{data}}[-\log p_{\theta}(\mathbf{x})] = \underbrace{\mathbb{E}_{\text{data}}[\nabla_{\theta} E_{\theta}(\mathbf{x})]}_{\text{positive phase}} - \underbrace{\mathbb{E}_{p_{\theta}}[\nabla_{\theta} E_{\theta}(\mathbf{x})]}_{\text{negative phase}}, \quad (3.23)$$

derived in full in Appendix A.5.

Both terms average the same function, $\nabla_{\theta} E_{\theta}(\mathbf{x})$, but the positive phase averages it over the data, while the negative phase averages under the model p_{θ} . Under gradient descent, the positive phase lowers the energy of configurations present in the data while the negative phase raises the energy of those the model favours. Once the two expectations agree, the gradient vanishes, which is when the model reproduces the data distribution [46].

The positive phase is cheap to calculate, since the data is given. However, the negative phase is an expectation under the Boltzmann distribution of Equation (3.19) and can be obtained only by sampling from the model itself at every gradient step [45]. The partition function itself never has to be evaluated, since its gradient is again an expectation under p_{θ} (see Appendix A.5) and sampling needs only energy differences. What blocks the fitting of an EBM is therefore not the value of the normaliser but the cost of drawing those samples, which is the same operation inference requires.

3.2.4 Boltzmann Machines and Gibbs Sampling

The Boltzmann machine [46] is the canonical energy-based model over discrete variables. To each configuration of K binary units $\mathbf{s} \in \{0, 1\}^K$ it assigns the quadratic energy

$$E(\mathbf{s}) = -\frac{1}{2} \mathbf{s}^{\top} J \mathbf{s} - \mathbf{h}^{\top} \mathbf{s}, \quad J = J^{\top}, \quad \text{diag}(J) = 0, \quad (3.24)$$

with symmetric couplings J_{ij} between units and local biases h_i . Through the Boltzmann distribution (Equation (3.19)) this defines a distribution over the 2^K binary states, which is

intractable (see Chapter 3.2.1). Being quadratic in binary variables makes the model structurally identical to an Ising spin system and therefore a natural target for physical sampling hardware [47, 48].

The samples are drawn by constructing a Markov chain that has the normalised distribution as its stationary distribution. Single-site Gibbs sampling [49] approaches this by visiting one unit at a time and redrawing it from its conditional distribution given the current values of all the other units it is linked to. A sweep updates every unit once, and repeated sweeps converge to the Boltzmann distribution.

Writing σ for the logistic function, the probability that unit i is on, given the current state of all the other units it is linked to, is

$$p(s_i = 1 \mid \mathbf{s}_{-i}) = \sigma\left(\sum_{j \neq i} J_{ij} s_j + h_i\right), \quad \sigma(a) = \frac{1}{1 + e^{-a}}, \quad (3.25)$$

as derived in Appendix A.6. Each update of unit i therefore requires only the states of the units directly coupled to i . Consequently, the landscape can be explored by using energy differences alone.

Equation (3.25) depends on the rest of the configuration only through the units coupled to i , so any set of mutually unconnected units can be updated simultaneously without altering the chain. Therefore, the coupling graph can be separated into two layers that have no connections within each of the layers. This is referred to as bipartite colouring which makes the Boltzmann Machine a restricted Boltzmann machine (RBM) [50]. By doing so, the units of one layer can be updated simultaneously and a full sweep of single-site updates reduces to only two block Gibbs updates per sweep.

In order for the Gibbs algorithm to converge, a sample is only representative once the chain has mixed, meaning that it has explored the parameter space sufficiently and reached an equilibrium state. The time until it has done so is called *mixing time* which grows significantly on high-dimensional landscapes with deep and many energy barriers separating the modes of the landscape [51]. In highly multimodal energy landscapes, the number of sweeps required to overcome the energy barriers can become prohibitive.

That cost compounds with the learning rule as Gibbs samples are required at every gradient step in the negative phase of Equation (3.23). Consequently, in practice, the chain is usually not run to convergence at all. Contrastive divergence, as a workaround, takes a single Gibbs sweep from a chain initialised at a training sample [52], and although the number of sweeps can be increased as training proceeds, small values remain the standard recommendation [53, 54]. Consequently, the gradient becomes biased and there is a general trade-off between computational cost and gradient bias. However, this trade-off only exists because the relaxation towards equilibrium is being simulated, whereas thermodynamic hardware performs the relaxation towards equilibrium by itself, which is discussed next.

3.3 Thermodynamic Computing

While the first stochastic computing circuits were already proposed in the 1960s [55, 56, 57], thermodynamic computing has especially gained traction over the last years as an alternative paradigm to conventional computing [15, 16, 18], and will be introduced in the following sections.

3.3.1 Concept

Conventional physical substrates on which computation is carried out usually fluctuate. These fluctuations are suppressed because a logic level has to be held far enough above the thermal noise floor, thus calculated values are reliable. The thermodynamic computing paradigm inverts this viewpoint by treating fluctuations as the central computational resource [15, 16]. Instead of emulating stochastic dynamics in software on a substrate engineered to be deterministic, it recasts computation as controlled stochastic relaxation on a physical energy landscape. The hardware is built so that the energy function of a device can be programmed to match the energy landscape of a target distribution and that the device relaxes towards the equilibrium of the Boltzmann distribution (see Equation (3.19)). Reading its state once it has equilibrated returns a sample. A thermodynamic device is therefore not a processor that computes a sample but a physical system whose equilibrium is the target distribution.

The main advantage of the thermodynamic hardware over conventional hardware is the required energy per sample. The synthesised randomness of a digital sample dissipates energy far above the thermal scale $k_B T$. The thermal fluctuation in the thermodynamic hardware serves as the source of randomness and therefore does not have to be computed, which is why the cost per sample is set by the cost of using the noise and not the cost of producing the noise [16].

Coles et al. formulate the paradigm around two elementary building blocks, and the distinction between them organises the hardware that follows [16]. The *stochastic mode* (s-mode) is a continuous, real-valued degree of freedom whose equilibrium distribution is Gaussian, realised physically by noisy oscillators or analog circuits. The *stochastic bit* (s-bit) is a binary unit whose state fluctuates between its two values with a probability set by an input. As discussed in Chapter 3.2.4, a Boltzmann machine sets each of its units to one with a probability given by the logistic function of its local field, which is exactly what the s-bit in discrete thermodynamic hardware does physically.

3.3.2 Thermodynamic Computing Approaches for Uncertainty Quantification

This section builds on Coles et al.’s segmentation of the thermodynamic paradigm into continuous and discrete lineages [16], and discusses existing thermodynamic-based approaches for UQ. Although this project is based on the discrete hardware type, literature of the continuous lineage is reviewed for context as well.

3.3.2.1 Continuous Thermodynamic Hardware

The continuous lineage builds on a single foundational capability. A network of coupled noisy harmonic oscillators, left to relax, samples from a Gaussian equilibrium distribution. Aifer et al. showed that this primitive performs key linear algebra operations, among them matrix inversion, linear solves and determinants, at a hardware time cost scaling as $\mathcal{O}(d^2)$ rather than the classical $\mathcal{O}(d^3)$ [58]. This is the basis for the efficiency case of the whole paradigm, since a Gaussian posterior is exactly such a linear algebra problem.

In a companion paper, Aifer et al. turned the same hardware directly onto Bayesian inference, which is the pivotal result for the present work [17]. They designed analog circuits whose stochastic dynamics are engineered so that their stationary distribution coincides with a target posterior, reducing posterior sampling to letting the device equilibrate. Their Gaussian–

Gaussian model is that primitive applied to inference, whereas Bayesian logistic regression extends the same mechanism to a non-Gaussian posterior. Validated in simulation, the two designs establish a thermodynamic algorithm for sampling Bayesian posteriors and, in the logistic-regression case, a proposal for non-Gaussian posterior sampling on such hardware. Conceptually, the device acts as a weight-sample generator for an otherwise deterministic network. The stochastic, Bayesian content is offloaded entirely to the hardware while the deterministic map from each weight sample to a prediction is evaluated conventionally.

These proposals were given experimental substance by Melanson et al., who built a small-scale continuous-variable thermodynamic computer, a stochastic processing unit (SPU) of eight all-to-all coupled RLC cells [18]. On it, they demonstrated Gaussian sampling and the inversion of an 8×8 matrix, which is the first physical realisation of thermodynamic linear algebra. The work thus carries the continuous approach from algorithmic proposal to experimental validation of the sampling. However, the device's reliance on inductors and transformers limits its scalability, motivating Melanson et al. to propose a resistively coupled CMOS design for scaling [18]. Normal Computing has since reported the tape-out of CN101, announced as the first thermodynamic computing chip in CMOS silicon and targeting the same primitives [59].

The reach of the continuous paradigm extends beyond inference in two further directions. Donatella et al. applied the same hardware to neural network training, realising natural gradient descent at the cost of plain first-order stochastic gradient descent by offloading its linear solve to the analog device [60]. Whitelam has shown that an analog thermodynamic system can act as a generative model in its own right, synthesising data through its natural stochastic evolution [61]. Assessed as a lineage, the continuous paradigm has therefore progressed from algorithmic proposal through simulation-validated Bayesian algorithms to physical demonstration and a first commercial CMOS product.

3.3.2.2 Discrete Thermodynamic Hardware

The discrete lineage is built on binary stochastic units. Coles et al. term the discrete building block the s-bit, a unit whose state evolves as a continuous-time Markov chain. The hardware reviewed below is instead built from the closely related probabilistic bit (p-bit) of Camsari et al., a tunable stochastic binary unit whose state is drawn as 0 or 1 with a probability given by the sigmoid of its input [47]. This is exactly the conditional update of the Gibbs sampling algorithm in Equation (3.25), so a network of coupled p-bits updated in this way relaxes to the Boltzmann distribution of an Ising energy. The devices within this lineage differ along two axes: whether their randomness comes from genuine physical fluctuations or is generated in simulation, and whether they are built in standard CMOS or in exotic components.

Borders et al. built a room-temperature p-computer whose randomness comes physically from the thermal switching of stochastic magnetic tunnel junctions (MTJs), and used its invertible logic to factor integers [62]. This demonstrated that nanodevice p-bits can solve a real inference problem, but one tied to exotic magnetic components. Aadit et al. instead generated the randomness digitally on conventional silicon. Their FPGA sparse Ising machine sparsifies the coupling graph so that non-neighbouring p-bits update in parallel, which is the block Gibbs scheme of Chapter 3.2.4 applied to a colouring of the hardware graph, reaching sampling rates as much as six orders of magnitude beyond CPU Gibbs sampling [63]. The sparsify-and-parallelise recipe introduced there is precisely the architectural template

adopted by the hardware of Extropic which is discussed in Chapter 3.3.3. Singh et al. combined both sources of randomness, clocking digital p-bits with sMTJ-based ones, and report that a single sMTJ p-bit can replace up to ten thousand CMOS transistors at around two orders of magnitude less energy per random bit [64].

Niazi et al. carried this hardware from optimisation into machine learning, training deep Boltzmann networks on a sparse Ising machine and applying them to image classification and generation [48]. Their measured device matches the classification accuracy of a software restricted Boltzmann machine while using about two orders of magnitude fewer parameters, which establishes on real hardware that this discrete substrate can carry a deep learning workload. What it does not do is quantify the uncertainty of those predictions.

Some work has nonetheless pointed discrete stochastic hardware at Bayesian inference directly. Faria et al. showed that p-bit circuits built from stochastic MRAM can emulate Bayesian *graphical* networks, with the correlations between variables read directly off the circuit nodes [19]. Liu et al. went further and implemented a Bayesian *neural* network in magnetic-tunnel-junction-based probabilistic in-memory hardware, using device stochasticity as the source of weight randomness [20]. These are the closest existing analogues to thermodynamic UQ on discrete hardware at the time of writing. Assessed as a lineage, however, the discrete paradigm has directed its scalable, CMOS-compatible instances at optimisation and generative modelling rather than at uncertainty, while the instances that do address Bayesian inference rely on specialised magnetic substrates. Neither performs rigorous posterior sampling on a scalable, all-transistor Boltzmann-machine substrate.

3.3.3 Extropic's Thermodynamic Sampling Unit

In 2025, Jelinčič et al. published an architecture for an all-transistor probabilistic computer built from sparse, locally connected Boltzmann machines sampled by massively parallel block Gibbs [21]. The application they developed it for is diffusion-based generative modelling, which is not relevant to this project. But what this project takes as its blueprint is their sampling hardware itself, the thermodynamic sampling unit (TSU), which is developed by the company Extropic Corp. [65]. It is the first proposal in the discrete lineage that is at once physically native in its randomness and fully compatible with scalable commercial CMOS, and it therefore avoids both drawbacks identified in Chapter 3.3.2.2.

The distribution the device samples is the Boltzmann machine of Equation (3.24) and the underlying update mechanism is

$$p(x_i[k+1] = +1 \mid \mathbf{x}[k]) = \sigma \left(2\beta \left(\sum_{j \neq i} J_{ij} x_j[k] + h_i \right) \right), \quad (3.26)$$

which is equivalent to Gibbs sampling (see Equation (3.25)) with two minor modifications. In the convention of Jelinčič et al. the units take values in $\{-1, +1\}$ rather than $\{0, 1\}$, which is the reason for the factor 2 in the equation. Additionally, an inverse temperature β is explicitly carried, which scales the energy landscape against the thermal noise that explores it. The couplings J_{ij} and local fields h_i are the same two objects throughout.

What makes the architecture relevant here is that J_{ij} and h_i are physically implemented in the hardware. Each node is a mixed-signal cell built around a transistor-based sampling circuit whose randomness is drawn from the shot-noise dynamics of the transistors, and

whose probability of emitting a +1 is a sigmoidal function of a control voltage. The current states of the neighbouring nodes arrive as voltages over wires and are summed by a tunable resistor network into precisely that control voltage. The coupling matrix J is therefore a network of programmable conductances and the field h a set of local bias offsets. That means that no random number is generated and no sigmoid is evaluated. The circuit forms the local field in the analog domain and reads a thermal fluctuation biased by it, which is the discrete thermodynamic mechanism realised in silicon.

The hardware used in the experiments of Jelinčič et al. is a Boltzmann machine with a sparse grid of binary variables, each coupled to only up to twelve of its neighbours in most of the reported experiments. The connectivity pattern is chosen to be bipartite, so that block Gibbs updates apply.

The energy required to draw one sample is modelled as the product of the number of sampling cells N , the number of sweeps the chain is run for K and the energy of a single conditional update $E_{\text{cell}} \approx 2$ fJ,

$$E = K N E_{\text{cell}}. \quad (3.27)$$

On this model, the authors estimate that a device implementing their diffusion-based architecture could match the sample quality of GPU-based generative models on binarised Fashion-MNIST while using roughly 10^4 times less energy per generated sample. In a very recent publication, Jelinčič and Walker report a 10^6 energy advantage of the TSU over a conventional GPU for mRNA codon optimisation [66].

It is important to note that the results are obtained from a physical model of the architecture combined with simulation, for which the authors have open-sourced the JAX library `thrml` [67]. The authors themselves position the work as a first step towards a new type of AI system and not as a finished and polished product.

3.4 Research Gap

The literature reviewed in the previous chapters shows that Bayesian UQ is constrained by the computational cost of drawing samples from a variational posterior and that thermodynamic hardware can make exactly that operation cheap.

On the continuous side of thermodynamic hardware, Bayesian inference has already moved from algorithmic proposal to physical demonstration and a first CMOS tape-out. On the discrete side, the devices that scale in commercial CMOS have been used for optimisation, generative modelling and basic classification tasks. The papers that address Bayesian inference with discrete hardware rely on specialised magnetic substrates whose integration into commercial CMOS at scale remains open. No work has yet shown UQ on a discrete all-transistor based thermodynamic architecture, which promises to scale.

The discrete TSU from Extropic has been prototyped physically and promises a scalable and energy-efficient Gibbs sampler, which makes it interesting to investigate for Bayesian uncertainty quantification. Therefore, this project develops a model architecture, whose stochastic component comes from a discrete Boltzmann machine, and which tackles exactly this research gap.

Chapter 4

Architectural Framework

4.1 Architecture and Structure

Before explaining each component and its training in detail, this section gives a comprehensive high-level overview of the architecture, for which Figure 4.1 has been created to show the construction in full. The architecture is called an **EBM-gated Adapter Mixture Model**, later also referred to as **mixture model**, which replaces the last layer of a deterministic neural network with a family of 2^K weight matrices, generated by low-rank adapters and indexed by a binary latent vector z . This will be explained step-by-step in the following.

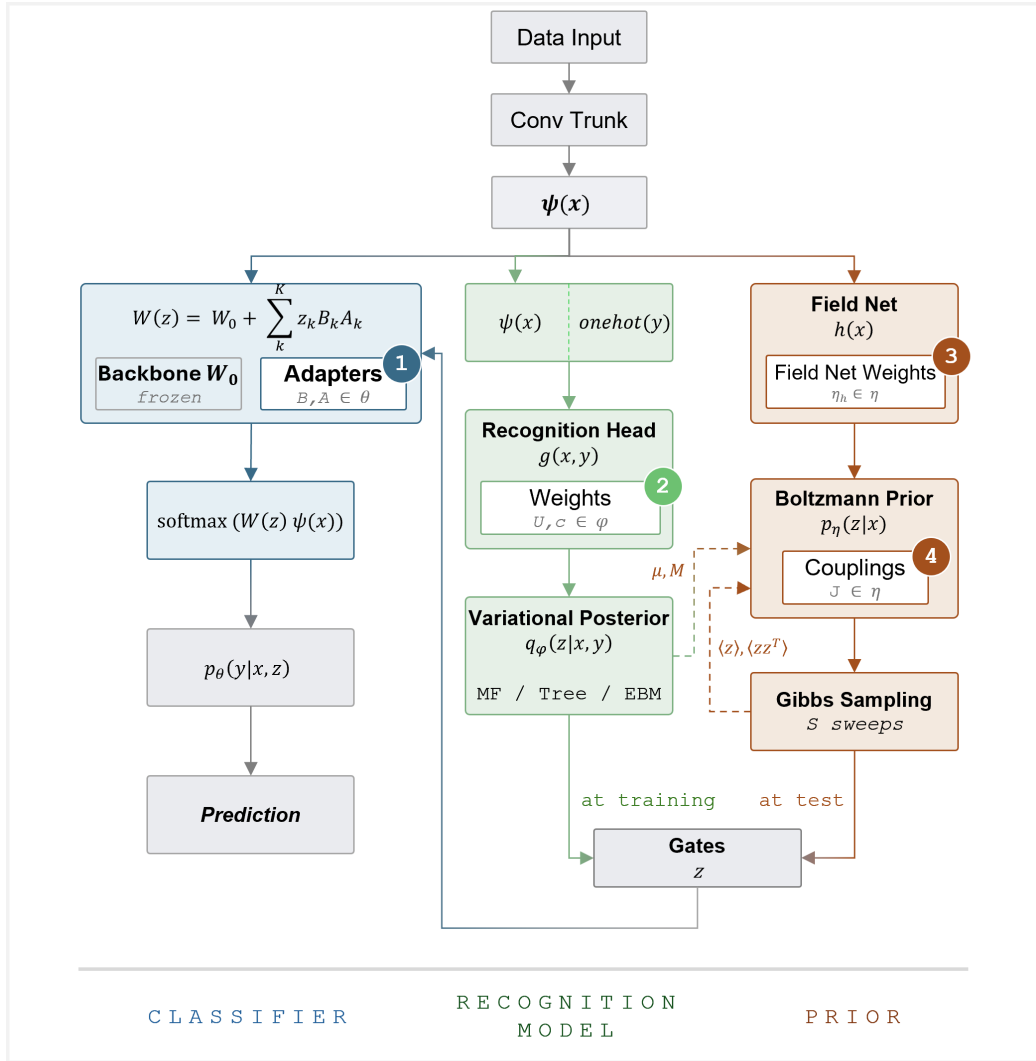


Figure 4.1: EBM-gated adapter mixture architecture

The starting point is a frozen *trunk* mapping the input to a *feature map* $\psi(x) \in \mathbb{R}^D$. The trunk may be a pre-trained network, a deliberately weakened version of one, or nothing at all, in which case $\psi(x)$ is the raw input. Next, the frozen *base map* $W_0 \in \mathbb{R}^{C \times D}$ maps feature map $\psi(x)$ to C logits. This base map can be as well completely random, the trained last layer of the trunk or an interpolation in between. Which combination of trunk and base map is used is a variable of the experiments. All that matters here is that $\psi(x)$ and W_0 are fixed before the mixture is trained and never receive a gradient.

Instead of one last-layer, a family of K *adapters* is added to the frozen W_0 , each controlled by a binary variable. An adapter is a low-rank correction of rank r consisting of the two factors A_k and B_k . A_k reads the D -dimensional feature map down to r numbers, while B_k writes those r numbers back out as adjustments to the C class logits. The *binary gate vector* $z \in \{0, 1\}^K$ controls which corrections are applied, so that the full weight matrix applied to an input can be stated as:

$$W(z) = W_0 + \sum_{k=1}^K z_k B_k A_k, \quad B_k \in \mathbb{R}^{C \times r}, \quad A_k \in \mathbb{R}^{r \times D}, \quad r \ll \min(C, D), \quad (4.1)$$

Training the model calls for gate patterns that suit the input, which requires the posterior over gates given both the feature map $\psi(x)$ and its label. Normalising that posterior sums over all 2^K gate patterns and is intractable. Therefore, a *recognition model* $q_\phi(z | x, y)$ is introduced, which reads the feature map with the true label and emits a tractable distribution over z . Since it is conditioned on the label, it serves as an approximate posterior which can only be used during training. At prediction time, the gates are drawn from the *gate prior* alone, where each draw defines a different classifier. The gate prior $p_\eta(z | x)$ is a conditional Boltzmann machine with *couplings* J and an input-conditioned *field network* $h_\eta(x)$ supplying the gate biases h . Both recognition model and prior are trained jointly in a teacher-student arrangement, in which the prior is pulled towards the recognition model, so that the prior learns to reproduce the gate patterns the recognition model selects, without having the true labels. The difference of the architecture used during training and prediction is visualised in Appendix A.7.

Two properties of this architecture are worth stating explicitly before the components are defined formally. First, no weight in this model is ever random. W_0 is frozen and the adapters A_k and B_k are ordinary point estimates trained by gradient descent. That means that for a fixed gate vector z , $W(z)$ is a single deterministic matrix and the classifier built from it carries no stochastic content at all. The entire probabilistic component of the model is the binary vector z , which is K bits wide. This is also what makes a physical sampler a plausible accelerator, since the substrate is asked to sample a few binary variables rather than several thousand real-valued weights. Second, the variational distribution is therefore over the gates z and not the weights w as in the classical Bayesian VI method (see Chapter 3.1).

4.1.1 Likelihood, Prior and Recognition Model

The architecture consists of three main components, which are the likelihood conditioned on the gates, the prior over the gates and the recognition model approximating the posterior during training. Each of them is fitted by a different mechanism (see Chapter 4.3), so their parameters are also kept in three separate groups, labelled as follows:

- θ holds the **likelihood** parameters, carried by the adapters,
- η holds the **prior** parameters, carried by the gate couplings and the field network,
- ϕ holds the **recognition** parameters, carried by the recognition head.

4.1.1.1 Likelihood

Conditioned on a gate vector, the model is an ordinary classifier. The gated weight matrix $W(z)$ of Equation (4.1) is applied to the feature map, resulting in C logits, which the softmax maps to a categorical distribution over the label,

$$\ell(\mathbf{x}, \mathbf{z}) = W(\mathbf{z}) \psi(\mathbf{x}) = W_0 \psi(\mathbf{x}) + \sum_{k=1}^K z_k B_k a_k(\mathbf{x}), \quad a_k(\mathbf{x}) = A_k \psi(\mathbf{x}) \in \mathbb{R}^r, \quad (4.2)$$

$$p_{\theta}(y | \mathbf{x}, \mathbf{z}) = \text{Cat}(y; \text{softmax}(\ell(\mathbf{x}, \mathbf{z}))). \quad (4.3)$$

The trainable parameters are the adapter factors,

$$\theta = \{A_k, B_k\}_{k=1}^K, \quad A_k \in \mathbb{R}^{r \times D}, \quad B_k \in \mathbb{R}^{C \times r}, \quad r \ll \min(C, D). \quad (4.4)$$

The factorisation is what keeps that number small since a full correction would be $C \times D$ per adapter, whereas the two factors carry only $r(C + D)$. A_k reads the D -dimensional feature map down to those r numbers and B_k writes them back out as adjustments to the C class logits, as Figure 4.2 visualises.

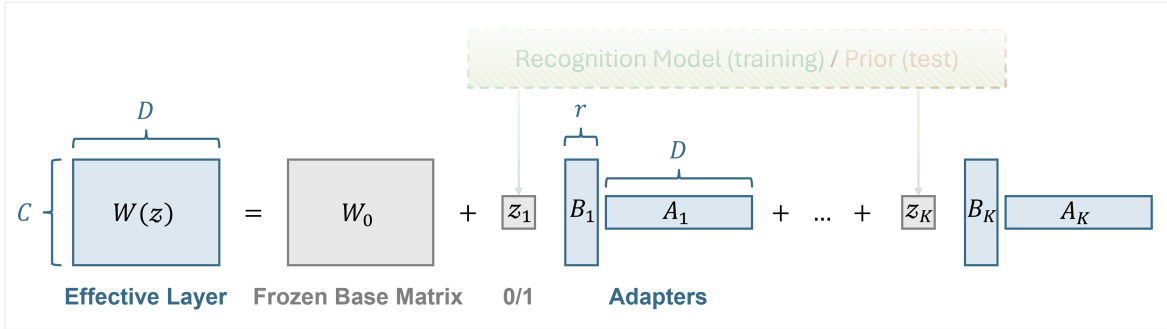


Figure 4.2: Classifier weight matrix $W(z)$ consisting of the frozen base map W_0 and the low-rank corrections $A_k B_k$ indexed by the binary gates z_k

4.1.1.2 Gate Prior

The prior is a fully-coupled Boltzmann machine conditioned on the data input through the field network $h_{\eta}(x)$,

$$p_{\eta}(z | x) = \frac{\exp(-E_{\eta}(z, x))}{Z_{\eta}(x)}, \quad E_{\eta}(z, x) = -h_{\eta}(x)^{\top} z - \frac{1}{2} z^{\top} J z, \quad (4.5)$$

with $J = J^{\top}$, $\text{diag}(J) = 0$, and $Z_{\eta}(x) = \sum_{z \in \{0,1\}^K} \exp(-E_{\eta}(z, x))$. The two halves of the energy $E_{\eta}(z, x)$ are conditioned asymmetrically, which means that the biases h provided by

the field vary from input to input, whereas the couplings $\mathbf{J}$ are global and shared by every input to express which combinations of adapters are plausible together independent of each input. The trainable parameters are the couplings $\mathbf{J}$ and the parameters of the field network $h_\eta(x)$, which are defined as follows:

$$\eta = \{\mathbf{J}, \eta_h\}, \quad \mathbf{J} \in \mathbb{R}^{K \times K} \text{ symmetric with } \text{diag}(\mathbf{J}) = 0 \quad (4.6)$$

$$h_\eta(x) = W_2 \tanh(W_1 \psi(x) + \mathbf{b}_1) + \mathbf{b}_2 \in \mathbb{R}^K, \quad \eta_h = \{W_1, \mathbf{b}_1, W_2, \mathbf{b}_2\} \quad (4.7)$$

$\mathbf{J}$ contributes $K(K-1)/2$ free numbers and the field network is a small multilayer perceptron with a single hidden layer of 24 units and a tanh nonlinearity, mapping the D -dimensional feature map to one field bias value per gate. The structure is visualised in Figure 4.3.

As every gate is coupled to every other, the prior can express that two adapters are useful together or mutually exclusive. However, this is also what makes the prior expensive, since $Z_\eta(x)$ sums over 2^K configurations and is intractable in exactly the sense established in the background chapter (see Chapter 3.2.1). Consequently, a Gibbs sampler is used to produce draws without ever forming the sum over the 2^K patterns. This local update is precisely the operation Extropic's TSUs can perform physically, which will also be discussed in Chapter 4.6.

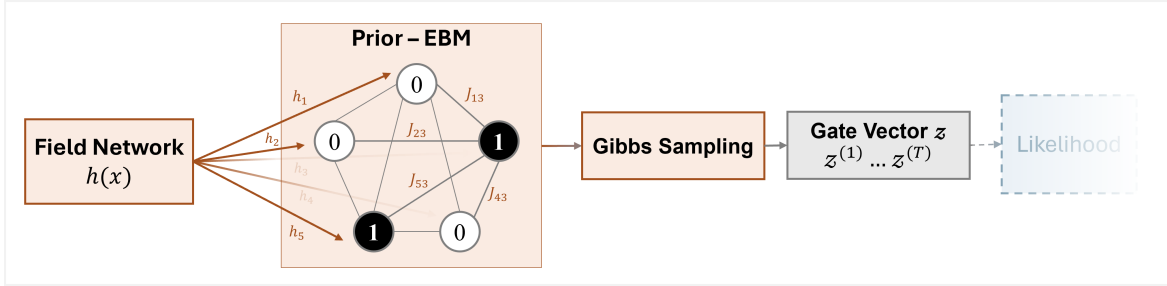


Figure 4.3: Gate prior with global couplings $\mathbf{J}$ and input-dependent biases provided by the field network $h_\eta(x)$

4.1.1.3 Recognition Model

Training requires the posterior over gates given a labelled pair $p(z | x, y)$, whose normaliser is the same intractable sum. It is therefore approximated by a recognition model, a network with parameters ϕ that reads the feature map together with the true label and returns the parameters of a tractable distribution over z .

The feature map $\psi(x)$ and the one-hot encoded label are concatenated into a single vector of length $D + C$, and the linear recognition head with weight matrix U and bias c turns it into the numbers the variational family needs,

$$\alpha_\phi(x, y) = U[\psi(x), (y)] + c \in \mathbb{R}^{d_q}, \quad \phi = \{U, c\}, \quad (4.8)$$

with $U \in \mathbb{R}^{d_q \times (D+C)}$ and $c \in \mathbb{R}^{d_q}$. Its output width d_q depends on the variational family. The mean-field family of Equation (4.29) needs one number per gate, so $d_q = K$ and each output is the logit of a gate. Another option for a variational family is a tree-structured family, as introduced in Chapter 3.1.1.1, which additionally requires a number for each

defined connection between a parent and child gate, resulting in $d_q = 2K - 1$ numbers. Swapping the variational family therefore changes the width of U and c but nothing else in the model. The structure of the recognition model is visualised in Figure 4.4.

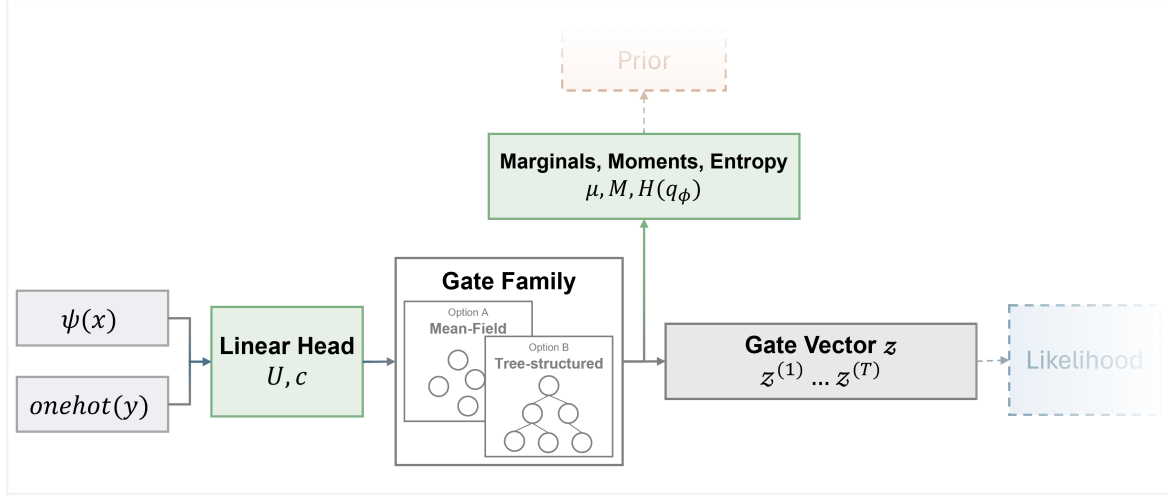


Figure 4.4: Recognition model structure consisting of the linear recognition head with weight matrix U and bias c providing the parameters for the variational family

4.1.1.4 Interim Summary of the Architecture

In classical variational inference, as defined in the background chapter Chapter 3.1.1.1, the prior is a fixed distribution and the variational posterior carries the flexibility the method can afford. The architecture described so far swaps that arrangement, giving the prior the correlations and the intractability and leaving the recognition model tractable.

In this setting, the recognition can be described as the teacher building the input-conditioned posterior distribution and knowing the true labels, while the prior can be described as the student trying to match the posterior distribution as closely as possible. When using a mean-field or tree-structured variational family in the recognition model, the only intractable normaliser is in the prior which requires sampling. Table 4.1 summarises the three components.

	Distribution	Trainable parameters	At test time
Likelihood	$p_{\theta}(y \mathbf{x}, \mathbf{z})$ Equation (4.3)	$\theta = \{A_k, B_k\}_{k=1}^K$	yes
Gate prior	$p_{\eta}(\mathbf{z} \mathbf{x})$ Equation (4.5)	$\eta = \{\mathbf{J}, \eta_h\}$, with $\eta_h = \{W_1, \mathbf{b}_1, W_2, \mathbf{b}_2\}$	yes
Recognition model	$q_{\phi}(\mathbf{z} \mathbf{x}, y)$ Equation (4.29)	$\phi = \{U, c\}$	no

Table 4.1: The three key components of the EBM-gated adapter mixture architecture

4.1.2 Predictive Distribution and Uncertainty

The joint and the predictive marginal are defined by the likelihood and the prior as follows:

$$p(y, z | \mathbf{x}) = p_{\theta}(y | \mathbf{x}, z) p_{\eta}(z | \mathbf{x}), \quad p(y | \mathbf{x}) = \sum_{z \in \{0,1\}^K} p_{\theta}(y | \mathbf{x}, z) p_{\eta}(z | \mathbf{x}). \quad (4.9)$$

As the recognition model is only active during training, it does not appear in these equations. The sum runs over all 2^K gate patterns and is therefore evaluated by sampling from the prior (see Appendix A.7) through Gibbs sampling,

$$p(y | \mathbf{x}) \approx \frac{1}{S} \sum_{s=1}^S p_{\theta}(y | \mathbf{x}, z^{(s)}), \quad z^{(s)} \sim p_{\eta}(z | \mathbf{x}), \quad (4.10)$$

The same S draws also supply the terms for the predictive uncertainty, which takes its usual decomposition as introduced in the background chapter (see Equation (3.5)) with the only difference that here it is taken between the label and the gates and not between the label and the weights:

$$\underbrace{I(y; z | \mathbf{x})}_{\text{epistemic}} = \underbrace{H[p(y | \mathbf{x})]}_{\text{total}} - \underbrace{\mathbb{E}_{p_{\eta}(z | \mathbf{x})}[H[p_{\theta}(y | \mathbf{x}, z)]]}_{\text{aleatoric}}. \quad (4.11)$$

4.2 Training Objective

In this section, the foundations for the gradient calculations and training algorithm in Chapter 4.3 will be defined.

Gated Log-Likelihood

Conditioned on a gate vector, the model is the ordinary softmax classifier of Equation (4.3), whose value at the observed label y is the single entry

$$p_{\theta}(y | \mathbf{x}, z) = \frac{\exp(\ell_y(\mathbf{x}, z))}{\sum_{c=1}^C \exp(\ell_c(\mathbf{x}, z))} \in (0, 1), \quad (4.12)$$

with the logits $\ell(\mathbf{x}, z)$ of Equation (4.2). In training, the logarithm of this equation is used throughout,

$$\log p_{\theta}(y | \mathbf{x}, z) = \ell_y(\mathbf{x}, z) - \log \sum_{c=1}^C \exp(\ell_c(\mathbf{x}, z)). \quad (4.13)$$

The logit $\ell(\mathbf{x}, z)$ is linear in z , which means that a gate that is switched off in a given draw removes its corresponding adapter from Equation (4.13). Consequently, the adapters A_k and B_k never receive a gradient update whenever their gate z_k is turned off in that draw.

ELBO

The objective is to maximise the marginal log-likelihood of the observed label, whose sum over the 2^K gate patterns in Equation (4.9) is intractable. Introducing the recognition model and applying Jensen's inequality bounds it from below by

$$\log p(y | \mathbf{x}) \geq \underbrace{\mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x}, y)} [\log p_\theta(y | \mathbf{x}, \mathbf{z})]}_{\text{reconstruction}} - \underbrace{\text{KL}(q_\phi(\mathbf{z} | \mathbf{x}, y) \| p_\eta(\mathbf{z} | \mathbf{x}))}_{\text{regulariser}} =: \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\eta}, \phi; \mathbf{x}, y), \quad (4.14)$$

which is the ELBO of Chapter 3.1.1.1 written for this model. It is a bound on the marginal log-likelihood and the amount by which it falls short is exactly the divergence between the recognition model and the true gate posterior,

$$\log p(y | \mathbf{x}) - \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\eta}, \phi; \mathbf{x}, y) = \text{KL}(q_\phi(\mathbf{z} | \mathbf{x}, y) \| p(\mathbf{z} | \mathbf{x}, y)) \geq 0, \quad (4.15)$$

with equality only when the recognition model reproduces that posterior. The derivations for Equations (4.14) and (4.15) are given in Appendix A.8.

In classical variational inference, the prior is fixed, acting as a steady force against a posterior drifting towards a state where all gates are switched off. In this architecture, the prior is trained as well and follows the posterior of the recognition model, so nothing anchors the gates, thus the model can settle in a configuration where $W(\mathbf{z}) \approx W_0$. Consequently, a KL-regularisation weight β is introduced which is a tunable hyperparameter that arbitrates between fitting the label and matching the prior. Over a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, this gives the **training objective** of this project, the β -weighted ELBO,

$$\mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta}, \boldsymbol{\eta}, \phi) = \sum_{i=1}^N \left[\mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x}_i, y_i)} [\log p_\theta(y_i | \mathbf{x}_i, \mathbf{z})] - \beta \text{KL}(q_\phi(\mathbf{z} | \mathbf{x}_i, y_i) \| p_\eta(\mathbf{z} | \mathbf{x}_i)) \right]. \quad (4.16)$$

At $\beta = 1$ this is exactly $\sum_i \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\eta}, \phi; \mathbf{x}_i, y_i)$, the bound of Equation (4.14) summed over the data. When β takes any other value it is a regularised objective and therefore bounds nothing.

KL-term in Closed Form

The KL-term of Equation (4.14) can be expanded and written in closed form. It depends on the recognition model through only its entropy and its first two moments and, as derived in Appendix A.9, is given as follows:

$$\boldsymbol{\mu} = \mathbb{E}_{q_\phi}[\mathbf{z}] \in (0, 1)^K, \quad \mathbf{M} = \mathbb{E}_{q_\phi}[\mathbf{z}\mathbf{z}^\top] \in \mathbb{R}^{K \times K}, \quad (4.17)$$

$$\text{KL}(q_\phi \| p_\eta(\cdot | \mathbf{x})) = -H(q_\phi) - \mathbf{h}_\eta(\mathbf{x})^\top \boldsymbol{\mu} - \frac{1}{2} \text{tr}(\mathbf{J}\mathbf{M}) + \log Z_\eta(\mathbf{x}). \quad (4.18)$$

No step in Equation (4.18) assumes a particular variational family. The mean-field family of Equation (4.29) is the special case in which the off-diagonal entries of $\mathbf{M}$ factorise as $M_{kj} = \mu_k \mu_j$, so that $\text{tr}(\mathbf{J}\mathbf{M}) = \boldsymbol{\mu}^\top \mathbf{J}\boldsymbol{\mu}$. A tree-structured family, on the other hand, retains

off-diagonal entries. Which family is used therefore changes how μ , M and $H(q_\phi)$ are computed, but not Equation (4.18) itself.

4.3 Gradients of the Trainable Parameters

Differentiating the training objective of Equation (4.16) with respect to each parameter group yields different gradient constructions, which will be dealt with in the following one by one. While the adapters only appear in the reconstruction term and the prior only in the KL-term, the recognition model parameters appear in both. This makes the recognition head the only group whose gradient must be calculated from two separate estimators. Figure 4.5 visualises the routing of the gradient calculation to each parameter group. Throughout this chapter $z^{(1)}, \dots, z^{(T)}$ denote the T draws from $q_\phi(z | x, y)$.

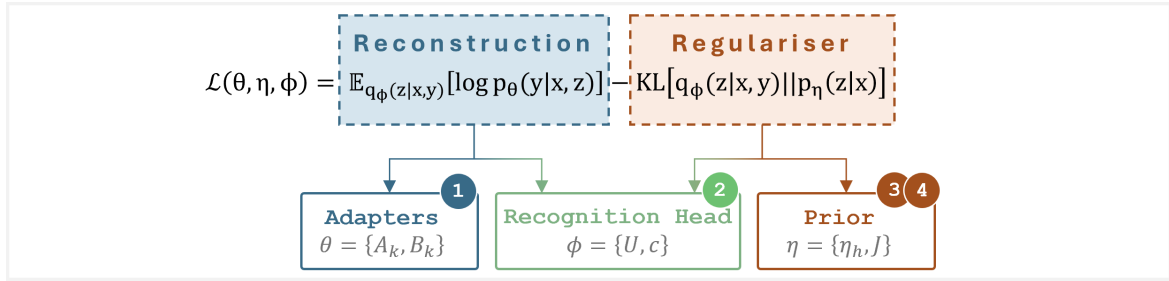


Figure 4.5: Which term of the training objective (see Equation (4.16)) moves which parameter group gradient. Equation is stated without β -regulariser for simplicity in this figure. Wherever it applies for the gradient computation, it will be stated separately.

4.3.1 Adapter Gradients

Since only the reconstruction term contains θ , the gradient can be moved inside the expectation and can be estimated by drawing T samples directly,

$$\nabla_{\theta} \mathbb{E}_{q_\phi}[\log p_\theta(y | x, z)] = \mathbb{E}_{q_\phi}[\nabla_{\theta} \log p_\theta(y | x, z)] \approx \frac{1}{T} \sum_{t=1}^T \nabla_{\theta} \log p_\theta(y | x, z^{(t)}). \quad (4.19)$$

For any $T \geq 1$, the estimator is unbiased and the gates enter as a constant, so θ is differentiated as in any softmax classifier. By differentiating the log-likelihood of Equation (4.13) with respect to the logits the residual can be obtained,

$$\delta = (y) - \text{softmax}(\ell(x, z)) \in \mathbb{R}^C, \quad \delta_c = \mathbb{1}[c = y] - p_\theta(c | x, z). \quad (4.20)$$

These are carried by the chain rule through Equation (4.2) into the two factors of each adapter, resulting in the gradients for B_k and A_k :

$$\frac{\partial \log p_\theta(y | x, z)}{\partial B_k} = z_k \delta a_k(x)^\top, \quad \frac{\partial \log p_\theta(y | x, z)}{\partial A_k} = z_k B_k^\top \delta \psi(x)^\top, \quad (4.21)$$

4.3.2 Prior Gradients

The prior parameters sit only in the KL-term. Since the entropy in the expanded KL-term, as given in Equation (4.18), only depends on the recognition model, the entire prior gradient depends on the remaining terms, which grouped together are

$$\mathbb{E}_{q_\phi}[E_\eta(z, x)] + \log Z_\eta(x).$$

The expected energy term $\mathbb{E}_{q_\phi}[E_\eta(z, x)]$ can be differentiated by moving the gradient inside the expectation, and the gradient of $\log Z_\eta(x)$ can be stated as an average of the energy gradient under the prior distribution (equivalent to derivations in Appendix A.5). Together they build the contrastive gradient consisting of the positive and negative phase,

$$\nabla_\eta \mathcal{L}(\theta, \eta, \phi; x, y) = \underbrace{\mathbb{E}_{p_\eta(z|x)}[\nabla_\eta E_\eta(z, x)]}_{\text{negative phase}} - \underbrace{\mathbb{E}_{q_\phi(z|x, y)}[\nabla_\eta E_\eta(z, x)]}_{\text{positive phase}}, \quad (4.22)$$

The positive phase is the expectation under the posterior q_ϕ and lowers the energy of configurations the posterior favours, while the negative phase is the expectation under the prior p_η which raises the energy of configurations the model currently samples. The gradient vanishes when the prior marginals equal the aggregated posterior, thus a stationary point is reached. When comparing this architecture's contrastive gradient to the contrastive gradient in Equation (3.23) of the background chapter, it can be observed that in this architecture, the recognition model takes over the role of the data distribution for the prior gradient. The energy gradients can be derived from Equation (4.5) as $\partial E_\eta / \partial \mathbf{h}_\eta = -\mathbf{z}$ and $\partial E_\eta / \partial J_{kj} = -z_k z_j$, which gives the two parameter blocks,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_\eta(x)} &= \underbrace{\boldsymbol{\mu}}_{\text{recognition model}} - \underbrace{\mathbb{E}_{p_\eta(z|x)}[\mathbf{z}]}_{\text{sampled from the prior}}, \\ \frac{\partial \mathcal{L}}{\partial J_{kj}} &= \underbrace{M_{kj}}_{\text{recognition model}} - \underbrace{\mathbb{E}_{p_\eta(z|x)}[z_k z_j]}_{\text{sampled from the prior}}, \quad k < j. \end{aligned} \quad (4.23)$$

Both lines hold for any recognition model, since the family enters only through the moments $\boldsymbol{\mu}$ and $\mathbf{M}$. The prior expectations are estimated from Gibbs draws $\tilde{\mathbf{z}}^{(1)}, \dots, \tilde{\mathbf{z}}^{(n)} \sim p_\eta(\mathbf{z} | x)$ as sample averages,

$$\mathbb{E}_{p_\eta(z|x)}[\mathbf{z}] \approx \frac{1}{n} \sum_{t=1}^n \tilde{\mathbf{z}}^{(t)}, \quad \mathbb{E}_{p_\eta(z|x)}[z_k z_j] \approx \frac{1}{n} \sum_{t=1}^n \tilde{z}_k^{(t)} \tilde{z}_j^{(t)}, \quad (4.24)$$

so the first moment is the fraction of draws in which a gate is on and the second the fraction in which two gates are on together. As the KL weight β is applied to the KL-term, the field and the couplings move by $\beta(\boldsymbol{\mu} - \mathbb{E}_{p_\eta(z|x)}[\mathbf{z}])$ and $\beta(\mathbf{M} - \mathbb{E}_{p_\eta(z|x)}[\mathbf{z}\mathbf{z}^\top])$ respectively.

To reach the gradients for the weights $\mathbf{W}$ and the biases $\mathbf{b}$ of the field network, one further chain rule must be applied, yielding

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\eta}_h} = \left(\frac{\partial \mathbf{h}_\eta(\mathbf{x})}{\partial \boldsymbol{\eta}_h} \right)^\top \left(\boldsymbol{\mu} - \mathbb{E}_{p_\eta(\mathbf{z}|\mathbf{x})}[\mathbf{z}] \right), \quad (4.25)$$

which is an ordinary backward pass through the field network.

4.3.3 Recognition Model Gradients

In contrast to the two previously discussed components, both sides of the ELBO are required to obtain the gradient for the recognition model parameters ϕ . Therefore, the gradient is assembled from a contribution from each term,

$$\nabla_\phi \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\eta}, \phi; \mathbf{x}, y) = \nabla_\phi \mathbb{E}_{q_\phi}[\log p_\theta(y | \mathbf{x}, \mathbf{z})] - \nabla_\phi \text{KL}(q_\phi \| p_\eta(\cdot | \mathbf{x})). \quad (4.26)$$

KL-Term

The expanded KL-term of Equation (4.18) includes $\log Z_\eta(\mathbf{x})$ which does not depend on ϕ . Therefore, the gradient-relevant part of the KL-term reduces to

$$-H(q_\phi) - \mathbf{h}_\eta(\mathbf{x})^\top \boldsymbol{\mu} - \frac{1}{2} \text{tr}(\mathbf{J}\mathbf{M}),$$

which is a closed-form function of ϕ through the entropy and the two moments. Consequently, the gradient for the recognition model parameter ϕ is given as

$$\nabla_\phi \text{KL}(q_\phi \| p_\eta(\cdot | \mathbf{x})) = -\nabla_\phi H(q_\phi) - \sum_{k=1}^K h_{\eta,k}(\mathbf{x}) \nabla_\phi \mu_k - \sum_{k < j} J_{kj} \nabla_\phi M_{kj}, \quad (4.27)$$

The entropy gradient and the gradients of the moments can be computed in closed-form.

Reconstruction-Term

The difficulty of the reconstruction term is that the expectation is taken over q_ϕ itself, so the gradient cannot be moved inside it as it was done for the adapter gradients. Additionally, $\mathbf{z}$ is discrete which prohibits a reparameterisation of the draws. Consequently, the score function estimator, as explained in Chapter 3.1.2, is applied with the leave-one-out baseline of Equation (3.14),

$$\begin{aligned} \nabla_\phi \mathbb{E}_{q_\phi}[\log p_\theta(y | \mathbf{x}, \mathbf{z})] &= \mathbb{E}_{q_\phi}[\log p_\theta(y | \mathbf{x}, \mathbf{z}) \nabla_\phi \log q_\phi(\mathbf{z} | \mathbf{x}, y)] \\ &\approx \frac{1}{T} \sum_{t=1}^T \left(\log p_\theta(y | \mathbf{x}, \mathbf{z}^{(t)}) - b^{(t)} \right) \nabla_\phi \log q_\phi(\mathbf{z}^{(t)} | \mathbf{x}, y). \end{aligned} \quad (4.28)$$

$b^{(t)}$ denotes the mean log-likelihood of the remaining $T - 1$ samples. The likelihood contains no ϕ and is never differentiated with respect to it. Each draw is instead scored by how well it classified and that score weights the gradient of its own log-probability.

In summary, the recognition model receives its gradient from one exact (KL-term) and one approximated source (reconstruction term).

4.4 Variational Families of the Recognition Model

Every derivation of the preceding Chapters 4.2 and 4.3 was stated without committing to a particular variational family as it is a modelling choice of the experiments conducted in this project. It can be exchanged as long as it delivers certain quantities that the components of the architecture consume for their gradient calculation, which are:

1. **First two moments μ and M** of Equation (4.17), for the positive phase of the prior gradient (Equation (4.23)) and the KL-term of the recognition model gradient (Equation (4.27)),
2. **Entropy $H(q_\phi)$** for the KL-term of the recognition model gradient (Equation (4.27)),
3. **Draws $z^{(t)} \sim q_\phi$** for the estimation of the adapter gradient (Equation (4.19)) and the reconstruction term of the recognition model gradient (Equation (4.28)),
4. **Score $\nabla_\phi \log q_\phi(z|x, y)$** of the draws for the recognition model gradient (Equation (4.28)).

The choice of the family also influences the output width of the recognition head d_q as it depends on the number of pairwise couplings between the gates of the variational family. Importantly, no equations of the training objective change when the family changes, only the sub-mechanisms that supply these four quantities. But the choice of the family is important as it influences what the recognition model can express and what it costs to obtain it. The variational families used in this project will be discussed in the following.

4.4.1 Mean-Field and Tree-Structured Families

Both families were introduced in Chapter 3.1.1.1 and are restated here only in the form in which the architecture consumes them.

Mean-Field

The mean-field family treats gates as mutually independent,

$$q_\phi(z | x, y) = \prod_{k=1}^K \text{Bern}(z_k; \mu_k), \quad \mu = \mu_\phi(x, y) \in (0, 1)^K, \quad (4.29)$$

so $d_q = K$ and each pre-activation $a_{\phi,k}$ is read as a gate logit. Its moments and entropy are given as

$$\mu_k = \sigma(a_{\phi,k}), \quad M_{kj} = \mu_k \mu_j \quad (k \neq j), \quad M_{kk} = \mu_k, \quad H(q_\phi) = \sum_{k=1}^K H_b(\mu_k). \quad (4.30)$$

Since the gates are binary and therefore $z_k^2 = z_k$, the diagonal is μ_k and not μ_k^2 . The samples are obtained by K independent Bernoulli draws and the score factorises over the gates. Consequently, all four quantities are available exactly.

Since Equation (4.30) forces $M_{kj} = \mu_k \mu_j$ for every off-diagonal entry, the positive phase of the coupling gradient in Equation (4.23) is not able to carry any correlation signal. A mean-field recognition model therefore cannot teach the prior any off-diagonal coupling, whatever the data contains, which makes the correlated prior of Equation (4.5) structurally unreachable.

Tree-Structured

The tree family, as introduced in Equation (3.12) of the background chapter, conditions every gate on at most one parent gate. The edge slope s_k is based on the two conditional probabilities given the parent gate,

$$\begin{aligned}\alpha_k &= q_\phi(z_k = 1 \mid z_{\text{pa}(k)} = 0), & \beta_k &= q_\phi(z_k = 1 \mid z_{\text{pa}(k)} = 1), \\ s_k &:= \beta_k - \alpha_k \in [-1, 1],\end{aligned}\tag{4.31}$$

and carries the correlation of the edge. As the tree must be acyclic to maintain its closed form, the recognition head must therefore provide $d_q = 2K - 1$ numbers. The marginals and moments can be propagated along the topological order and are given as

$$\mu_k = \alpha_k + s_k \mu_{\text{pa}(k)}, \quad M_{kj} = \mu_k \mu_j + \mu_m (1 - \mu_m) \prod_{c \in \mathcal{P}(m \rightarrow k)} s_c \prod_{c \in \mathcal{P}(m \rightarrow j)} s_c.\tag{4.32}$$

The entropy accordingly decomposes per edge into conditional entropies,

$$H(q_\phi) = H_b(\mu_{\text{root}}) + \sum_{k \neq \text{root}} \left[(1 - \mu_{\text{pa}(k)}) H_b(\alpha_k) + \mu_{\text{pa}(k)} H_b(\beta_k) \right].\tag{4.33}$$

Samples are drawn starting from the root gate along the tree and the score is available in closed form. Therefore, all four quantities can be provided and remain exact.

Learned Topology

Since a tree is not allowed to form loops to keep its closed form, a variety of different tree topologies can be used (see examples in Figure 4.6). Which tree topology performs best can depend on different factors, like the input data structure or the number of gates K .

As explained in Chapter 3.1.1.1, the best tree-structured approximation to a target distribution is a maximum spanning tree. Applied in this setting, the target is the recognition model's posterior, so the edge weights are read off from the moments the family already computes,

$$W_{kj} = |M_{kj} - \mu_k \mu_j|, \quad \text{pa} = \text{MST}(W),\tag{4.34}$$

with the absolute pairwise covariance used in place of the mutual information as a cheaper surrogate. The run starts from a warm-up topology and rebuilds the parent array from

Equation (4.34) at a fixed epoch, after which training continues unchanged. This structure-selection step is the only difference to the fixed-topology tree, everything else still holds.

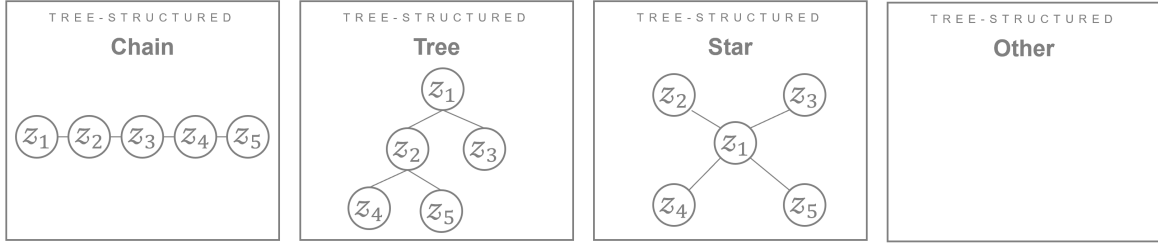


Figure 4.6: Examples for different tree structures

4.4.2 Energy-Based Recognition Model

While the preceding families offer tractability, they are not able to reflect the fully-coupled gate structure of the prior. The logical next step is therefore to also use an EBM as the variational family, with the only difference to the prior that it also receives the label as an input in the recognition head,

$$q_\phi(z | x, y) = \frac{\exp\left(\mathbf{f}_\phi(x, y)^\top \mathbf{z} + \frac{1}{2} \mathbf{z}^\top \mathbf{J}_q(x, y) \mathbf{z}\right)}{Z_\phi(x, y)}, \quad \text{diag}(\mathbf{J}_q) = \mathbf{0}, \quad (4.35)$$

Consequently, the recognition head must output $d_q = K + \frac{1}{2}K(K-1)$ numbers for the fully connected EBM graph. As in the prior EBM, the normaliser $Z_\phi(x, y)$ sums over all 2^K gate patterns and is therefore intractable. Consequently, neither the moments nor the entropy are available in closed form anymore and the exact recognition gradient is lost, which is why these quantities must be computed from samples.

Since Equation (4.35) is a pairwise Boltzmann machine, its single-site Gibbs conditional is the one of Equation (3.25) with the only difference that the input-dependent couplings $\mathbf{J}_q(x, y)$ replace the global $\mathbf{J}$,

$$q_\phi(z_k = 1 | z_{-k}, x, y) = \sigma\left(f_{\phi,k}(x, y) + \sum_{j \neq k} J_{q,kj}(x, y) z_j\right). \quad (4.36)$$

All four required quantities can be read off the same chains. The first two moments are counts. With T chains $\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(T)}$ drawn per input,

$$\mu_k = \frac{1}{T} \sum_{t=1}^T z_k^{(t)}, \quad M_{kj} = \frac{1}{T} \sum_{t=1}^T z_k^{(t)} z_j^{(t)}, \quad (4.37)$$

μ_k is the fraction of chains in which gate k is on and M_{kj} the fraction in which gates k and j are on together. The same chains give the draws $\mathbf{z}^{(t)}$ and, through the energy of Equation (4.35), the score. The entropy is recovered only up to the unknown offset $\log Z_\phi(x, y)$, which is sufficient because the objective of Equation (4.16) is optimised through gradients and never uses the entropy itself. In both the entropy gradient and the score, that offset

cancels, so $Z_\phi(\mathbf{x}, y)$ never has to be computed. The full derivations are provided in Appendix A.10. The chains are treated as fixed once drawn, so no gradient is propagated through the Gibbs updates, which would be in any case undefined for binary draws.

4.4.3 Variational Families Summary

Table 4.2 summarises each family against the four requirements set out in Chapter 4.4. Reading it downwards, the variational families gain in expressiveness. But while mean-field and both tree variants deliver all four quantities in closed form, the quantities must be estimated from samples when using an EBM for the recognition model.

Family	Head width d_q	Moments μ, M	Entropy $H(q_\phi)$	Draws $\mathbf{z}^{(t)}$	Score
Mean-field	K	exact, Equation (4.30)	exact, Equation (4.30)	independent Bernoulli	closed form
Tree, fixed	$2K - 1$	exact, Equation (4.32)	exact, Equation (4.33)	ancestral along the tree	closed form
Tree, Chow–Liu	$2K - 1$	exact, Equation (4.32)	exact, Equation (4.33)	ancestral along the tree	closed form
Energy-based	$K + \frac{1}{2}K(K - 1)$	sampled, Equation (4.37)	sampled up to $\log Z_\phi$, Equation (9.20)	Gibbs chains, Equation (4.36)	from the energy, Equation (9.22)

Table 4.2: The variational families of the recognition model against the head width they require and the four quantities the training objective consumes. All four families leave Equation (4.18) and the gradients of Chapter 4.3 unchanged and differ only in how these quantities are obtained.

4.5 Training Algorithm

The training algorithm runs on the full dataset, so that gradients are formed once per epoch and applied together. All parameters are updated by plain gradient ascent with a shared learning rate across all parameter groups. As described in the previous sections, the three parameter groups are reached via different mechanisms. The adapters and the recognition head receive their gradients from a differentiated objective and the couplings and field network from a difference of moments whose negative phase must be sampled. As a consequence, the following training algorithm has been derived, stated as pseudocode in Appendix A.11.

The algorithm follows the gradient routing of Figure 4.5. Before running the training algorithm over a set number of epochs, the required quantities must be initialised (lines 1 to 8 in Appendix A.11). This includes among others the base classifier W_0 , the adapters, the recognition head or certain anti-collapse schedules, which will be explained in Chapter 4.5.1. Moreover, this also includes the Gibbs chain that will be carried from epoch to epoch.

Next, in every epoch of the training, the recognition model and the field network are evaluated first as the algorithm further downstream consumes them (lines 10 to 12). Both of them do not require sampling since the entropy and moments are in closed form and the field is a single forward pass, when using a mean-field or tree-structured recognition model.

The next step is to perform Gibbs updates based on the field network inputs just computed (line 13), producing the samples from which the gate marginals and their second moments under $p_\eta(z | x)$ are estimated. Since the prior is conditioned on the input, every data point receives its own chain.

Subsequently, the quantities for the reconstruction gradient are computed before the adapters are updated, since a draw must be scored against the classifier that produced it (lines 14 to 17). All previously computed quantities are then assembled to form the gradients for the adapters and the recognition head, which are directly being updated (lines 18 to 21).

The only missing update required is the update of the prior’s field network weights and couplings. Therefore, the contrastive gradient is computed and both parameters updated accordingly (lines 22 to 27). Before continuing with the next epoch, the gradients are discarded and only the updated parameters as well as every data point’s Gibbs chain are carried into the next iteration. This is the persistent variant of contrastive divergence applied to this architecture, so that S sweeps are sufficient to stay close to the prior.

4.5.1 Stabilisation Techniques

On top of the training algorithm, a set of measures has been implemented to stabilise training. They are collected in Appendix A.12 and fall into four groups, each addressing a different way in which training can fail: the gates collapsing, the adapters degenerating, the gates failing to specialise, and the parameters growing without bound.

The KL weight β has already been introduced in Chapter 4.2 and is annealed over a configurable number of epochs, which is used by default. Further measures active by default are the free bits, which allow each gate to keep a fixed amount of divergence from the prior at no cost, the initialisation of the field network’s output bias to a positive value so that every gate is usable from the first epoch, the initialisation of the couplings to a uniformly negative value so that the gates inhibit one another, and gradient clipping together with weight decay to constrain parameter growth.

Options that are not active by default but are used in individual experiments are, for example, a warmup during which the recognition head and the prior are held fixed while the adapters train alone, adapter anchoring, which replaces the weight decay on the adapters by a spring towards their initial values, and the pressures that penalise gate patterns which are not sparse or not evenly used across the gates. All of these options can therefore be viewed as a toolbox of configurable measures to counteract failure modes observed in the experiments, which can be switched on and off depending on the task and setup.

4.6 Handoff between Conventional and Thermodynamic Hardware

In the discussed architecture, there is a clear separation between the operations that can be performed on thermodynamic hardware and on conventional hardware like CPUs or GPUs. Figure 4.7 visualises the handoff between the hardware types.

Independently of the choice of the recognition model and independently of the training or test mode, the prior can always offload its single-site conditional of Equation (3.25) to the TSU. The TSU receives its required parameters $h_\eta(x)$ and J from the GPU, so it can relax to equilibrium and provide the binary gate vector z . The couplings J are global across all inputs

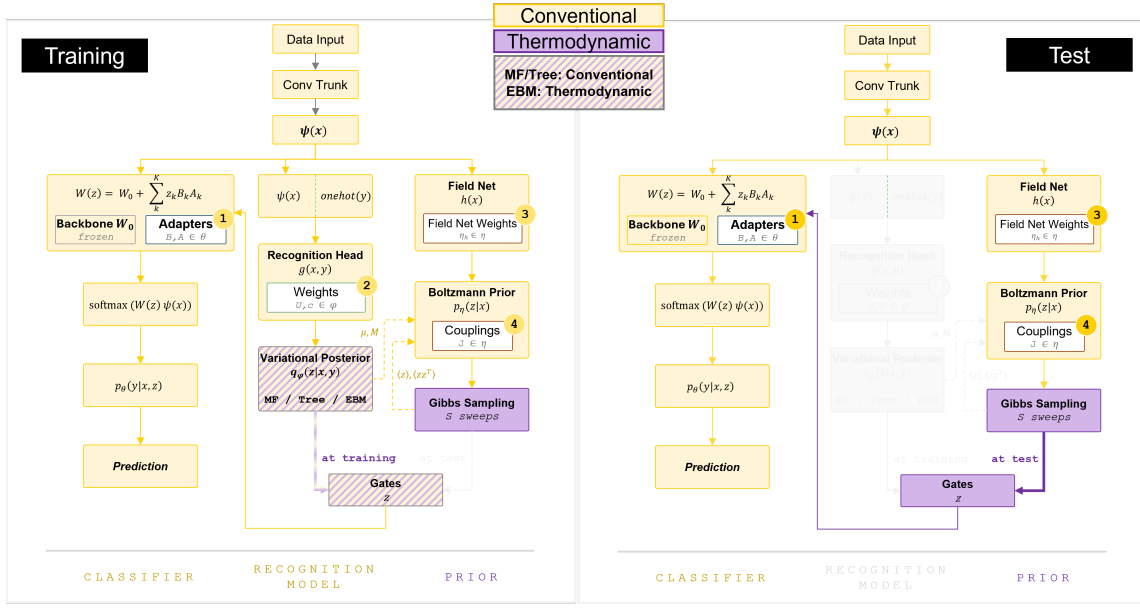


Figure 4.7: Handoff between conventional and thermodynamic hardware

and therefore stay resident on the hardware until being re-written when the contrastive update changes them once per epoch. The K -dimensional field vector, on the other hand, is streamed per input to the device, where it works as a bias at each gate. Since the training algorithm (see Appendix A.11) uses persistent chains, the state of the chain remains on the TSU across epochs.

The choice of the variational family decides whether there is another operation that can be offloaded to the TSU. Under mean-field or tree-structured recognition models, no Gibbs sampling is required (see Chapter 4.4.1). However, when using an energy-based recognition model of Chapter 4.4.2, the sampling operation can be computed by the TSU similarly as in the prior but with two differences. The first is that the couplings $J_q(x, y)$ are input-dependent as well, so the $K + \frac{1}{2}K(K - 1)$ values for the biases and couplings must be streamed for every data point. Secondly, chains are not persistent, which means that chain states are not carried across calls and each set of draws restarts from its warm-start configuration at the factorised guess $z_k^{(0)} \sim \text{Bern}(\sigma(f_{\phi, k}))$.

In summary, Extropic's TSU serves as the Gibbs sampler, providing the binary gate vector $z \in \{0, 1\}^K$. Everything that is computed from z , such as moments or entropy, belongs to the conventional hardware. Every update of the fields and couplings going into the Boltzmann machine for the thermodynamic hardware belongs to the conventional hardware as well. The quantification of computational costs and its scaling behaviour is one object of analysis discussed in Chapter 6.

Chapter 5

Experimental Design

The three high-level research objectives, as defined in Chapter 2, are focused on *feasibility*, *competitiveness*, and *scalability*. Consequently, the experiments conducted in this project are centred around these questions, each building on top of the previous one.

Chapter 5.1 first explains the developed experimental pipeline shared across the experiments. After the Chapters 5.2 and 5.3 provide an overview of the used datasets and baseline UQ methods, Chapter 5.4 gives an high-level introduction to all conducted experiments.

5.1 Parameter Optimisation Pipeline

The model architecture allows many different configurations along many axes, which are compared both with one another and against external baselines. Therefore, an extensive pipeline has been built through which every configuration reported in this work is trained and evaluated. Taking one shared pipeline as the basis gives a fair comparison and makes scaling up differently configured runs feasible, so that more than 6000 runs in total have been evaluated throughout this project. This allows studying the influence of different hyperparameters through a simple click and select interface, for which an Excel sheet (run_config.xlsx) is used.

Input

A run consumes three components as inputs. The first input is the configuration of the run defined in the Excel sheet. In total, 111 parameters can be configured in the Excel sheet, of which 64 parameters define the configuration of an individual task and 47 parameters are general adjustment levers for the model architecture. These include not only hyperparameter values such as number of gates K or the learning rate and number of epochs, but for example also the choice of the variational family or task setup. The full list of options is given in Appendix A.13. Additionally, a run receives the data as an input, which has been generated or loaded through specific task modules. If a convolutional trunk has been selected in the Excel sheet, the trunk is trained as a deterministic network first and then provided as a frozen input to the pipeline as well.

Processing

The figure in Appendix A.14 visualises the processing structure of the pipeline in Python 3.13. The orchestration layer parses the configuration input into a dictionary of run parameters which are given one-by-one to the engine, so that it can route each run to the correct modules of the training loop. For this, different sub-modules of the central training script (trainer.py) are being used, for example, for the different variational families, the field network or the gradient estimator. As a side note, while all large-scale studies are executed

exactly via this pipeline, some specific studies run via separate scripts, which, however, always call the same central training script as the pipeline to ensure fair comparison.

Output

Once the training loop finishes, one result metric record per seed and its diagnostic figures are saved in its own folder. Additionally, the results of each run are collected in a central Excel sheet (results.xlsx), which serves as the central result library of all runs. The four key metrics reported are accuracy, the ECE with 15 confidence bins, NLL and the Brier score, which have been introduced in Chapter 3.1.3.

5.2 Datasets

For the competitiveness study, the model is evaluated on two datasets that are very common in the UQ research field, Fashion-MNIST and CIFAR-10 [68, 69].

The Fashion-MNIST dataset contains 70 000 greyscale images of clothing articles with a resolution of 28×28 pixels. The CIFAR-10 dataset, on the other hand, contains 60 000 RGB-coloured natural photograph images with a resolution of 32×32 pixels. So, its flattened dimension of 3 072 is around four times that of Fashion-MNIST which has 784. The training set size is set to 55 000 for Fashion-MNIST and 45 000 for CIFAR-10. For both datasets, the validation set size is set to 5 000 and the test set size to 10 000 images.

Additionally, the models are evaluated under distribution shift. Therefore, the test images are blended with out-of-distribution images from unrelated datasets by applying

$$\mathbf{x}_f = (1 - f) \mathbf{x} + f \mathbf{x}_{\text{ood}}, \quad f \in \{0, 0.1, \dots, 0.9\}, \quad (5.1)$$

where the label is taken from the clean input image $\mathbf{x}$. The Fashion-MNIST images are blended with greyscale images of letters from the EMNIST dataset and the CIFAR-10 images are blended with RGB-coloured images of house numbers from the SVHN dataset [70, 71]. The Appendix A.15 figure shows example images under different blend fractions.

For each dataset, a convolutional trunk has been chosen for the frozen backbone based on Liu et al. [20], so that the comparison rests on a published architecture and not a specific one designed for this work. The Fashion-MNIST experiments use a LeNet-5 convolutional trunk whose last layer is replaced by the mixture model's base map W_0 , leaving a frozen trunk with 60 856 parameters. CIFAR-10 experiments, on the other hand, use a ResNet-13 whose output layer with 2 570 parameters is replaced the same way, leaving a trunk of around 1.23 million parameters. The reason for the bigger convolutional architecture used in CIFAR-10 compared to Fashion-MNIST is the higher complexity of the data. The LeNet-5 trunk is trained over 20 and the ResNet-13 trunk over 100 epochs, both over minibatches of size 32. The table in Appendix A.16 summarises the used datasets and their implementations.

5.3 Baseline Models

The mixture model's results are compared against three baselines, which are all trained on the same backbone. First, the deterministic version of the backbone sets the level of model accuracy the mixture model must match while adding calibration. The second baseline is

classical Bayesian mean-field VI (see Chapter 3.1.1.1), where each model weight is treated as a Gaussian distribution. Consequently, the parameter count is roughly twice the parameter count of the deterministic network. The third baseline is MC dropout whose predictive spread comes at no additional parameters since the same trained network is evaluated repeatedly under a different dropout mask (see Chapter 3.1.1.2). Mean-field VI and MC dropout have been chosen as they are two established methods that remain computationally tractable. Additionally, Liu et al. compare their thermodynamic UQ study based on magnetic tunnel junction hardware to exactly the same baselines. Their baselines are reimplemented and retrained in this project and not simply quoted because the published values cannot be interrogated and only a subset of the metrics, which are used in this project, are reported.

All methods are evaluated with the same Monte-Carlo budget of 100 samples per image, so that no method benefits from averaging over more samples than another.

5.4 Experiments

The experiments conducted in this project are grouped into five experimental groups, labelled as E1 to E5. Each of the experiments is based on the results of the previous one and the full setup per experiment is given with its results in Chapter 6.

The E1 experiments deal with the first research objective (feasibility) to analyse the learning mechanism and the resulting predictive uncertainty on artificial tasks. Further in-depth analyses evaluate the effectiveness of the input-dependent routing mechanism and the influence of different variational families in the recognition model, thereby laying the groundwork for the experiments on real data.

Next, the E2 to E4 experiments assess the second research objective (competitiveness) on real data. While E2 gives the fundamental comparison between the mixture model in different configurations against the established baseline methods on Fashion-MNIST and CIFAR-10, E3 adds further analyses regarding the mixture model's operating regime, behaviour under distribution shift and effects of increased expressive capacity. For all experiments in E2 and E3, a tree-structured recognition model is used, whereas E4 then exchanges it against an EBM recognition model. In E4, the EBM recognition model is compared to the tree recognition model and a sensitivity analysis is conducted to assess the required number of Gibbs updates.

Lastly, E5 gives an indication for objective three (scalability) by providing an approximation of the GPU floating-point operations that can be absorbed by the thermodynamic hardware.

Please note that all experiments were conducted in simulation on conventional GPU hardware. The experiments and their objectives are summarised in Appendix A.17.

Chapter 6

Experiments

This chapter provides the results and its analyses according to the experimental design defined in the previous Chapter 5.

6.1 E1: Groundwork and Feasibility Study on Artificial Data

The objective of the E1 experiments is to demonstrate general feasibility and understand the important mechanisms that underlie the results. This groundwork is necessary and valuable as it provides the required fundamental learnings for the following experiments on real data. Therefore, the experiments conducted here will, first, focus on assessing the learning mechanism and its produced results on artificial data (experiment E1.1), and then analyse the contribution of the input-dependent gating mechanism (experiments E1.1 and E1.2). A further sub-experiment E1.3 next analyses whether the different variational families behave as the theory previously outlined in Chapters 3 and 4.

6.1.1 E1.1: Blob Task

The blob task is the simplest setting presented in this research project. It is a low-dimensional classification problem for which the Bayes-optimal accuracy can be calculated, so that the model performance can be compared against actual ground truth. This therefore allows a quantitatively precise assessment of whether the model reaches its ceiling on this task and whether the predictive uncertainty produces the expected uncertainty geometry.

6.1.1.1 Task and Data

The data produced for this task are three isotropic Gaussian clusters on a 2D grid. Each cluster belongs to one class and consists of 200 training points. The clusters are placed with equal distances in between them and close enough so that the clusters overlap around a triple point. Consequently, the data is not perfectly separable and the Bayes ceiling, which can be computed in closed form from the known generating densities, reaches an accuracy of **0.956**. This is the accuracy benchmark the model is compared against based on 1 000 test data points per class.

6.1.1.2 Setup

Due to the task’s simplicity, no convolutional trunk or backbone is used. W_0 is therefore a frozen linear map with random weights that map the data input to three logits. On its own, W_0 achieves an accuracy of 0.33 which equals the chance of a random guesser. The mixture model consists of $K = 6$ gates and adapters of rank $r = 2$.

Five arms are trained on top of this configuration under otherwise equal conditions. The first three of them have the full EBM-gated mixture architecture and only differ in the variational

family they use in the recognition model, which are mean-field, a chain-structured tree and an EBM. The remaining two are gate ablation studies to assess the contribution of the gating mechanism. One of them is called *dropout gates*. Please note that *dropout gates* and *MC dropout* refer to different things in this report. In dropout gates, only the binary gate vector z is drawn from a Bernoulli distribution with a gate rate equal to the gate rate the EBM settles on. That means, dropout gates works over the latent space of z , while MC dropout is a baseline model, as introduced in Chapter 5.3, that works over the weight space of a deterministic network. This convention will be used throughout the report.

Consequently, in dropout gates adapters are mixed at the same rate as in the model with the EBM recognition model but without any dependence on the input. The other gate ablation model is called *all-on gates* where all gates are always on, collapsing the mixture to a single merged adapter. Both ablation models serve as reference models for control and are trained from scratch under their constraint.

At test time, every arm uses 200 gate draws per test point, each following 30 Gibbs sweeps. Every number presented in the results table is the mean over 10 seeds.

6.1.1.3 Results

The results of the five models are presented in Table 6.1. Since every arm is underconfident to a different degree ($\hat{T} < 1$ throughout), the table also reports the metrics after temperature scaling, which removes the confidence-scale difference between arms.

		without temperature scaling			with temperature scaling			$\hat{T}$
		Accuracy	ECE	NLL	Brier	ECE	NLL	
<i>Variational family of the recognition model</i>								
mean-field	0.9542 $\pm$ ₇	0.0307 $\pm$ ₂₀	0.141 $\pm$ ₂	0.071 $\pm$ ₁	0.0083 $\pm$ ₇	0.125 $\pm$ ₁	0.069 $\pm$ ₁	0.75
tree (chain)	0.9544 $\pm$ ₇	0.0339 $\pm$ ₂₁	0.143 $\pm$ ₂	0.071 $\pm$ ₁	0.0088 $\pm$ ₆	0.123 $\pm$ ₁	0.068 $\pm$ ₁	0.73
EBM	0.9541 $\pm$ ₇	0.0301 $\pm$ ₁₉	0.141 $\pm$ ₂	0.071 $\pm$ ₁	0.0089 $\pm$ ₈	0.125 $\pm$ ₁	0.069 $\pm$ ₁	0.75
<i>Baselines</i>								
dropout gates	0.9548 $\pm$ ₅	0.0672 $\pm$ ₄₉	0.173 $\pm$ ₅	0.075 $\pm$ ₁	0.0146 $\pm$ ₈	0.123 $\pm$ ₁	0.068 $\pm$ ₁	0.62
all-on gates	0.9545 $\pm$ ₅	0.0107 $\pm$ ₇	0.120 $\pm$ ₁	0.067 $\pm$ ₁	0.0089 $\pm$ ₅	0.118 $\pm$ ₁	0.067 $\pm$ ₁	0.95

Table 6.1: Blob task results. Subscripts give the standard error in units of the last digit, bold marks the column best.

Accuracy Ceiling

All three mixture models reach an accuracy between 0.9541 and 0.9544 with a spread between them of around one standard error. With a difference of around 0.2 percentage points to the Bayes ceiling of 0.956, there is a negligible gap to the optimal classifier for this data. Additionally, the accuracy is around 62 percentage points higher than the accuracy after W_0 , so the accuracy gain can be clearly attributed to the adapters. Moreover, all three variational families produce results very similar to each other across all metrics, which will be further analysed in Chapter 6.1.3.

Uncertainty Geometry

Figure 6.1 shows the decision map, the total predictive entropy $H[p(y|x)]$ and the epistemic uncertainty of Equation (4.11) for the EBM recognition model.

The decision map shows that the model has learned clear decision boundaries converging in the triple point. Along the decision boundaries, the total predictive entropy plot shows regions of high uncertainty which is reasonable since these are the areas where clusters overlap and points are not clearly separable.

Uncertainty Geometry on the Blob Task

BLOB-TASK

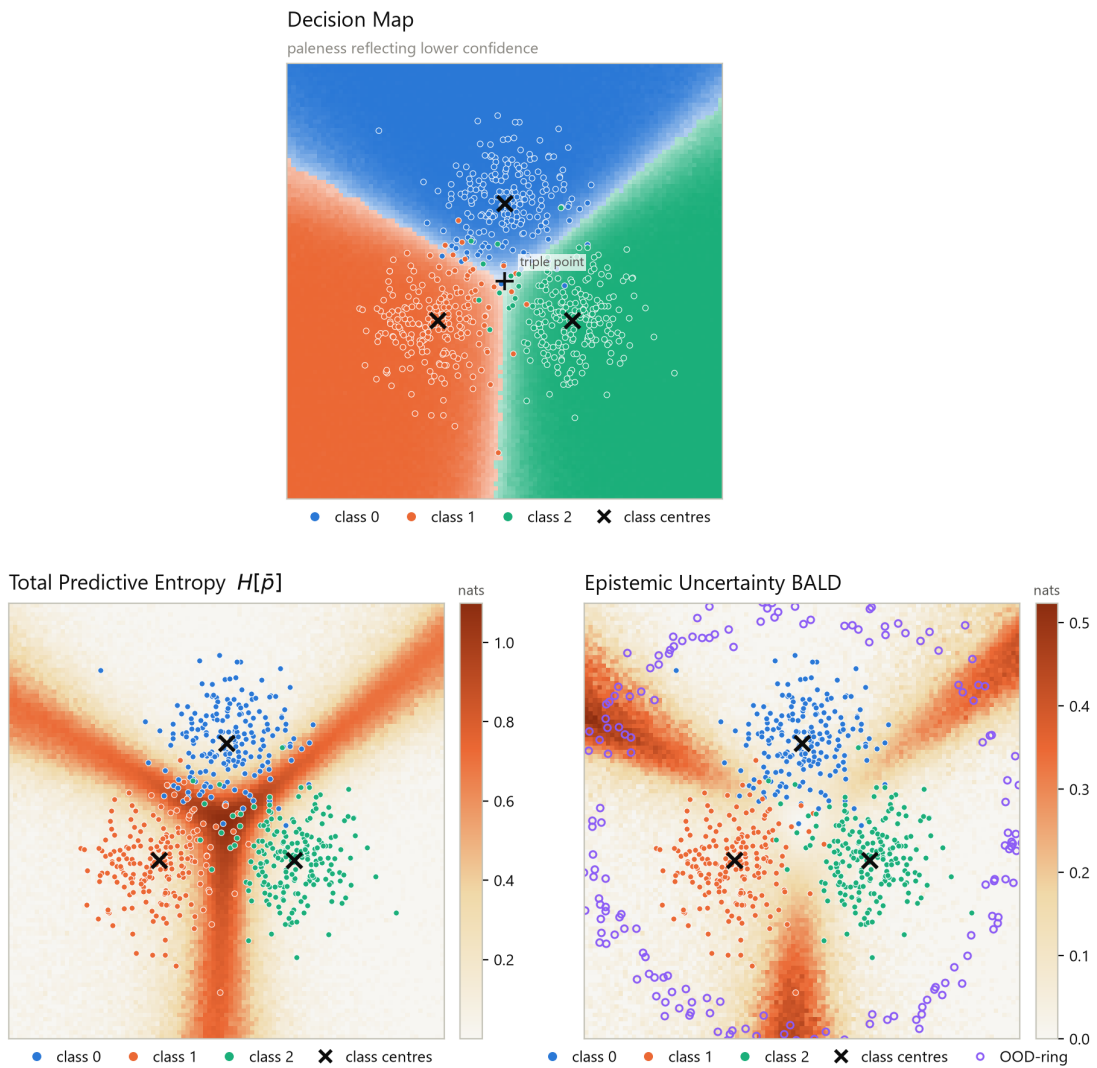


Figure 6.1: Decision map, total predictive entropy and epistemic uncertainty of the model on the blob task

However, when comparing the uncertainty surrounding the decision boundaries in the total predictive entropy and the epistemic uncertainty plot, the regions of higher and lower uncertainty are near mirror images. The reducibility argument introduced in Chapter 3.1.1 explains why: At the triple point, the data points are dense. Every gate pattern in the model that has been assigned significant prior mass, has been fitted to the same observations and

assigns the same class probabilities there. Therefore, the models agree, the mutual information is close to zero and the total entropy is almost entirely aleatoric. This is also in line with the logical explanation that the clusters overlap in this region which is why the classes are not clearly separable, so the uncertainty lies in the data and not in the model. When moving along the decision boundaries away from the triple point, the density of the data points decreases. Different adapter combinations extrapolate slightly differently and the uncertainty region widens. This also matches the characteristic that more data can reduce epistemic but not aleatoric uncertainty, which is exactly the effect that can be observed in the plots.

In the third panel, purple data points, as a visualisation of out-of-distribution data that are far away from the data the model has been trained on, have been added to hint at a limitation of the model. In prediction, out-of-distribution data points are usually detected when they are flagged as points where models are uncertain. That means that these are points where the epistemic uncertainty is high. The OOD data points can be split into two groups for demonstration purposes. One group is within a 20° angle of a decision boundary extension and the other beyond 40° . When comparing each group’s epistemic uncertainty, it can be shown that the mean BALD of the near decision boundaries group is around 21-times the BALD of the other group (0.255 to 0.012). Additionally, the median BALD on the OOD-ring (0.024) is below the training data’s BALD (0.044), which means that the model reports less epistemic uncertainty on inputs it has never seen than on the ones it was trained on (see Table 6.2). The reason for these observations is structural in the architecture as gates are sampled from the prior $p_\eta(z | x)$, which is based on a field $h_\eta(x)$ with couplings and no density term. Therefore, far from the data, the field extrapolates to a single confident gate configuration and disagreement vanishes. This shows that the model is boundary-aware and not density-aware, which limits its capability for OOD-detection purposes in this configuration.

Input group	n	Mean BALD	Median BALD
OOD ring, near a boundary ($< 20^\circ$)	50	0.255	—
OOD ring, away from boundaries ($> 40^\circ$)	60	0.012	—
OOD ring, all probes	180	0.086	0.024
Training data (reference)	600	0.053	0.044

Table 6.2: Epistemic uncertainty at the out-of-distribution ring of Figure 6.1, split by angular distance to the nearest extrapolated decision boundary. Distance from the training data does not raise the epistemic uncertainty, but proximity to a decision boundary does.

Gate Ablation

When comparing the results between the three mixture models and the two baseline models *dropout gates* and *all-on gates*, it can be clearly seen that the baselines produce equal and in most metrics even better results than the original models. The all-on gates model receives the lowest temperature-scaled NLL (0.118 nats) and Brier score (0.067). It can be inferred that the likelihood is satisfied by a single map from the data point coordinates to the class. Therefore, there is no need for a combination of adapters and input-dependence is not learned because it is not needed. These findings conclude this sub-experiment E1.1: While the general ability to learn and correctly reflect uncertainty has been proven, this task has also shown that it does not require input-dependent routing through the gates. Whether the

gates can do more when a task requires the gating mechanism is the question sub-experiment E1.2 is built around.

6.1.2 E1.2: Routing Task

The task complexity of E1.1 did not demand input-conditioned routing from the adapter mixture model as a linear map satisfied the likelihood everywhere. Therefore, this sub-experiment E1.2 is designed such that the correct class prediction depends on which rule generated the input. The hypothesis is that this is the regime in which the gating mechanism will outperform a non-mixing baseline model without input-conditional gating. In this setting, the correct routing is known in advance and it can therefore be directly validated if the model learns the correct routing, so that it does not have to be inferred from accuracy.

6.1.2.1 Task and Data

Input data is now built as the concatenation of a context and a content vector, $\mathbf{x} = [\mathbf{x}_{\text{ctx}}, \mathbf{x}_{\text{cnt}}]$ with $[d_{\text{ctx}}]$ and $[d_{\text{cnt}}]$ dimensions respectively. The context vector is a one-hot vector deciding which one of the $R = 3$ linear rules M_r is applied to the content and the label is defined as the class the rule assigns to it, $y = \arg \max_c (M_r \mathbf{x}_{\text{cnt}})_c$. Data is generated such that one content vector can be assigned different labels under different context vectors. Consequently, which linear map applies depends on the context and no single fixed map from $\mathbf{x}$ to the logits can represent the rule. All three contexts are equally represented in the 2 400 training and 3 000 test inputs. The majority class accounts for 0.378 of the labels, which is the chance level that the results are read against.

Since the rule that generated each input is known, the correct routing has a known shape. For an input under rule r exactly one gate should be on and the other two off. The learned prior gate activation can therefore be compared directly with the correct routing.

6.1.2.2 Setup

The setup of this task is exactly the same as in the preceding blob task (see Chapter 6.1.1.2) with two changes. The number of gates is set to $K = R = 3$, so that each rule can correspond to one gate, and adapters have rank $r = 4$. As before, W_0 is a frozen linear map with random weights from the $[d_{\text{ctx}} + d_{\text{cnt}}]$ -dimensional input to three logits which is classifying on its own at an accuracy of around 0.335.

6.1.2.3 Results

Table 6.3 presents the results of the five models. Temperature scaling has been applied as well since especially the adapter mixture models were highly underconfident.

Gate Ablation Results

In contrast to the results of the blob task, a clear gap between the three mixture models and the two baselines becomes evident. While the three mixture models achieve an accuracy between 0.9492 and 0.9523, dropout and all-on gates only reach an accuracy of 0.5612 and 0.5637 respectively, which is around 39 percentage points lower. Both baselines, however, sit at the same level and well above the chance level of 0.378. The gap between the mixture models and the baselines comes exactly from the context-dependent rule which a single

		without temperature scaling			with temperature scaling			$\hat{T}$
		Accuracy	ECE	NLL	Brier	ECE	NLL	
<i>Variational family of the recognition model</i>								
mean-field	0.9493 $\pm_{27}$	0.1726 $\pm_{46}$	0.293 $\pm_6$	0.144 $\pm_3$	0.0066 $\pm_5$	0.124 $\pm_6$	0.074 $\pm_4$	0.23
tree (chain)	0.9523 $\pm_{26}$	0.1678 $\pm_{20}$	0.280 $\pm_3$	0.136 $\pm_2$	0.0079 $\pm_7$	0.116 $\pm_6$	0.069 $\pm_3$	0.22
EBM	0.9492 $\pm_{87}$	0.1499 $\pm_{39}$	0.267 $\pm_{16}$	0.133 $\pm_{10}$	0.0095 $\pm_{20}$	0.127 $\pm_{24}$	0.075 $\pm_{13}$	0.25
<i>Baselines</i>								
dropout gates	0.5612 $\pm_{19}$	0.0507 $\pm_{36}$	0.939 $\pm_2$	0.562 $\pm_1$	0.0221 $\pm_{26}$	0.933 $\pm_1$	0.558 $\pm_1$	0.78
all-on gates	0.5637 $\pm_{20}$	0.0268 $\pm_{16}$	0.928 $\pm_1$	0.556 $\pm_1$	0.0256 $\pm_{18}$	0.928 $\pm_1$	0.556 $\pm_1$	0.97

Table 6.3: Routing task results. Subscripts give the standard error in units of the last digit, bold marks the column best

map cannot represent. These numbers show that the same architecture, data and training procedure as in E1.1 produces the opposite verdict if the task requires a choice.

The results of the raw calibration metrics show clearly that the calibration metrics must be read with care. Without temperature scaling, the baselines have lower ECE values (all-on gates: 0.0268, dropout gates: 0.0507) than the mixture models. But a model can be very well calibrated although having a low accuracy, which is why they must always be read together and are always reported together throughout this project. The fitted temperature shows that the mixture models were highly underconfident with $\hat{T}$ being between 0.22 and 0.25, whereas the baseline models were calibrated much better at 0.78 and 0.97. After temperature scaling, the mixture models get to significantly lower calibration metrics reflected in the ECE, NLL and Brier score values: The mixture models reach an ECE of 0.0066–0.0095, the baselines 0.0221–0.0256, and the NLL of the mixture models sits between 0.116 and 0.127 nats while the baselines have an NLL of 0.928–0.933 nats.

Routing Correctness

The accuracy scores show that the adapter mixture models solve the task successfully. Figure 6.2 further analyses how the models solve the task and whether the models learn a one-rule-one-gate structure on their own through training. Since which adapter takes which rule is arbitrary and differs between seeds, each seed’s matrix is aligned to its best permutation of rows before averaging.

Each value in cell (k, r) gives the mean activation of gate k over the test inputs generated by rule r (gate-on rate). The top row in Figure 6.2 represents the ideal one-rule-one-gate structure (left), the observed equally distributed gate-on rate of the dropout gates baseline (middle) and the gate-on rate of the all-on gates baseline (right). Neither of the baseline models can produce a pattern that depends on the context, which is the structural reason for their lower accuracy. Note that the gate-on rate of the dropout gates baseline sits at 0.62 as this is the Bernoulli probability given by the average gate-on rate from the EBM mixture model. For each of the three mixture models, a dark blue diagonal is visible, which indicates the gate belonging to the active rule is on with a mean prior activation of 0.93–0.98. The average gate-on rate for a gate not corresponding to the active rule is not close to zero across all three models, which means that the learned routing is not clearly one-hot. This is not a failure of the mechanism as the accuracy is very high and the task therefore does not require

exclusive routing.

The results of this sub-experiment demonstrate that the gating mechanism works as expected on the routing task, validating the hypothesis stated in the beginning of this sub-experiment.

Routing Correctness vs. Ground Truth

ROUTING-TASK

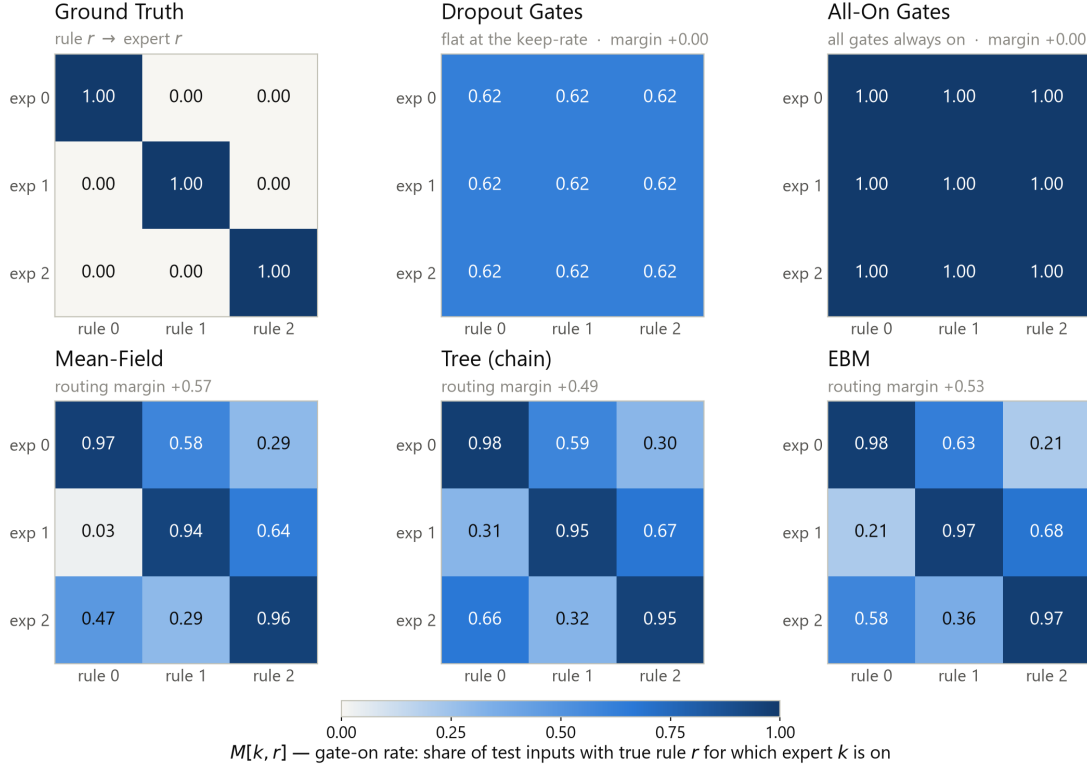


Figure 6.2: Routing correctness against the ground truth

6.1.3 E1.3: Posterior Fidelity of the Recognition Families

In Chapter 4.4, a strict hierarchy of the recognition families has been shown from a mathematically structural point of view. It was explained that mean-field cannot keep gate correlations, a tree can keep $K - 1$ of the $K(K - 1)/2$ couplings and only the EBM is able to capture all of them. E1.1 and E1.2 showed that all three families achieve very similar results across all metrics and both tasks. Therefore, this sub-experiment E1.3 assesses the hierarchy directly to answer what each family can represent in principle on a target that has been designed to have correlation structure, and how far each trained recognition model is from its own true posterior on the blob and routing task. With $K \leq 6$, the gate space contains $2^6 = 64$ configurations, so the true posterior $p(z | x, y)$ can be normalised by direct summation for every input and the $\text{KL}(q \| p)$ between each family's approximation and that posterior is available exactly.

6.1.3.1 Setup

To evaluate each family's capabilities in matching a target distribution thoroughly, they are assessed under four configurations described below. In each of the settings, the exact $\text{KL}(q \| p)$ is available by enumeration over the 2^K gate configurations.

- **Designed target:** This setup is the only one of the four which runs outside of the mixture model architecture to assess how well a family can approximate a designed target distribution, so no data, no likelihood and no trained model are used here. This target distribution is defined over $K = 6$ binary gates as $p^*(z) \propto \exp(\frac{1}{2}z^\top Jz)$ with $h = 0$, so no gate is favoured a priori, and $J = -J_0(1 - I)$, so every one of the 15 pairs carries the same negative coupling. The inhibition strength is swept over a grid of $J_0 \in \{0.25, 0.5, 1, 1.5, 2\}$, which raises the target's mean absolute pairwise correlation from 0.047 to 0.126. Each family is fitted to the target distribution p^* by exact reverse-KL descent, so the converged value is the best approximation each family can achieve.
- **In-run:** This setup is exactly the same as in E1.1 and E1.2 based on the blob and routing data with the full mixture model training. After the last training epoch, each recognition arm's distribution is compared with the arm's own exact posterior obtained by enumeration on the training set. This is the KL-gap the training run actually ended with.
- **Free refit:** After training, each arm's parameters are frozen and the exact posterior is enumerated on 500 samples from the test distribution and held fixed as the target. Each family is then fitted to the target through exact reverse-KL descent with an independent parameter vector per input. The resulting distribution of the family is the best possible fit to that posterior and the representational floor of the family.
- **Amortised refit:** In this setup, everything is the same as in *free refit*, with the only difference that the recognition head is restored. That means that the linear map from $[x, \text{one-hot}(y)]$ to the family's parameters is now the object being fitted. This is exactly how amortisation is implemented in the architecture, so the KL-gap between the free refit and the amortised refit shows the cost of amortisation.

In summary, the *designed target* setup builds a theoretical foundation and establishes what each family can represent in principle. The *in-run* setup shows which KL-gap a mixture model has actually achieved on the blob and routing data, and the *free refit* setup what it could have achieved under ideal circumstances. Consequently, the three measurements form a ladder for every family and task: The KL-gap between *in-run* and *amortised refit* is the cost of training, and the gap between *amortised refit* and *free refit* is the cost of amortisation. Every result number is the mean over 10 seeds.

6.1.3.2 Results

The results are presented chronologically along the ladder from the designed target to the three measurements on the trained arms.

Designed target

As can be seen in Figure 6.3, the three families separate exactly as Chapter 4.4 predicts and the separation grows close to linearly with increasing inhibition strength. The target carries 15 equally strong couplings, of which the tree can only represent 5 and the mean-field 0. So the omitted couplings cost more the stronger the interaction they omit.

At $J_0 = 0.25$, where the target's mean absolute pairwise correlation is 0.047, mean-field reaches 0.0225 nats (blue line) and the chain 0.0145 nats (red). At $J_0 = 2$, with the correlation at 0.126, mean-field is left at 0.335 nats and the chain at 0.218 nats. On the other hand,

Variational Family Limits on Free Target Distribution

DESIGNED TARGET

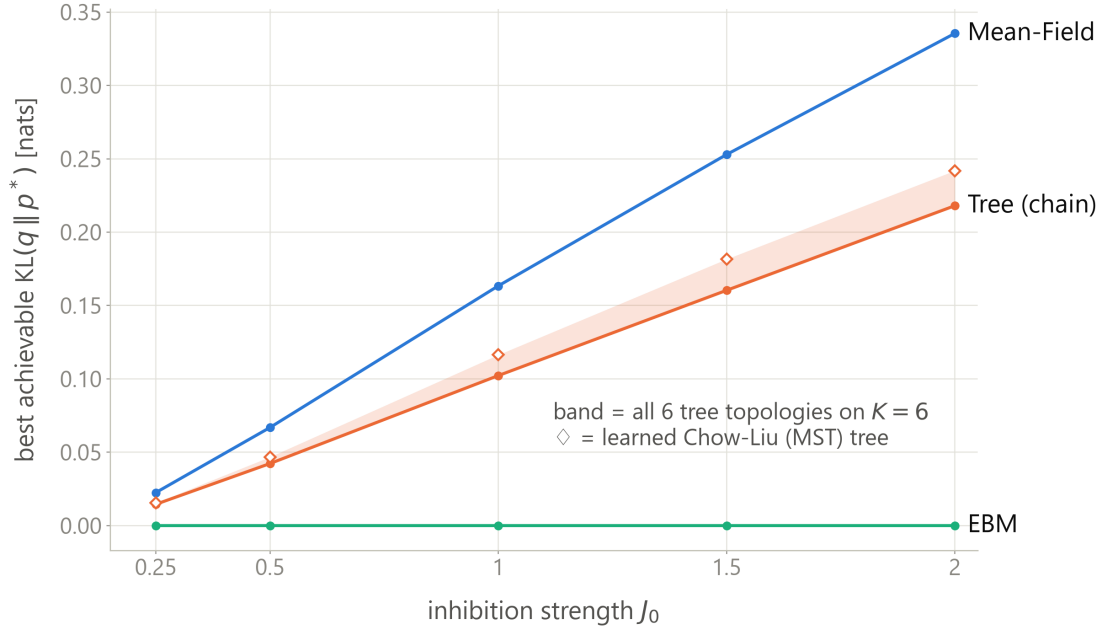


Figure 6.3: Best achievable $KL(q || p^*)$ of the three recognition families against the inhibition strength J_0 on the designed target $p^*(z) \propto \exp(\frac{1}{2}z^\top Jz)$ with $K = 6$ gates, $h = 0$ and $J = -J_0(1 - I)$. For the values of each data point see Appendix A.18.

the EBM reaches 0.0000 nats at every J_0 (green), which can be viewed as a consistency check since the target itself is a pairwise Boltzmann distribution.

Furthermore, the shaded band around the red tree curve in Figure 6.3 visualises the KL of all non-isomorphic tree topologies. It shows that the maximum spanning tree algorithm chooses a worse tree topology than the, in this setting, optimal chain structure. The reason for this is that the target distribution has the unique characteristic of exactly the same J_0 on every pairwise coupling. Consequently, every edge weight of the maximum spanning tree ties. The tie-breaker implemented always returned a tree formed as a star, which, however, in this case is the worst of the six non-isomorphic tree topologies. Moreover, the band width at $J_0 = 2$ between the best and worst topology of 0.024 nats is only about a tenth of the tree’s gap to the EBM. The tree’s residual is therefore structural and cannot be repaired by topology.

Trained arms

The results of the three trained arms are presented in Table 6.4 and visualised in Figure 6.4. It is important to keep in mind that the KL measured is always between the family’s learned posterior and its own exact posterior. That means that, for instance, the true posterior of the mean-field arm is not the true posterior of the tree or EBM arm. The last row in Table 6.4, showing the mean absolute correlation of the exact posterior, reflects this. Each family has a different value there and mean-field produces the most factorised posterior of the three with a mean of 0.041 on the blob against 0.050 for the EBM arm.

Across the three ladder rungs, the ordering of the three families established in the *designed target* reappears. On the blob task, it can be seen that mean-field $>$ tree $>$ EBM holds

	blob ($K = 6$)			routing ($K = 3$)		
	MF	tree	EBM	MF	tree	EBM
in-run	0.0383 $\pm$ 11	0.0314 $\pm$ 10	0.0262 $\pm$ 10	0.0519 $\pm$ 51	0.0548 $\pm$ 23	0.0364 $\pm$ 36
amortised refit	0.0338 $\pm$ 14	0.0258 $\pm$ 12	0.0129 $\pm$ 7	0.0316 $\pm$ 29	0.0296 $\pm$ 18	0.0213 $\pm$ 19
free refit	0.0192 $\pm$ 9	0.0113 $\pm$ 4	0.0009 $\pm$ 2	0.0038 $\pm$ 4	0.0022 $\pm$ 2	0.0030 $\pm$ 2
$ \text{corr} _p$	0.041 $\pm$ 1	0.049 $\pm$ 1	0.050 $\pm$ 1	0.029 $\pm$ 2	0.036 $\pm$ 4	0.034 $\pm$ 6

Table 6.4: Exact $\text{KL}(q \parallel p)$ in nats between each recognition family and the arm’s own enumerated posterior at the three rungs of the ladder

at every rung and is separated by several standard errors. On the routing task, the EBM is the lowest in-run with 0.0364 nats against 0.0519 and 0.0548 nats for mean-field and tree, which tie in their seed spread. Similarly on the amortised refit, MF and the tree tie, while the EBM is the lowest with 0.0213 nats. What was shown in theory and on the designed target is therefore also found in the routing task. However, there is an exception on the free refit where the EBM floor of 0.0030 nats comes out marginally above the tree floor of 0.0022 nats. Since the EBM can also be considered as a tree with extra couplings, it could theoretically set its couplings so that it matches the tree. Therefore, the observation that the tree performs better than the EBM in this case is likely to be an artefact of not being fully fitted to its minimum. Additionally all three families sit very close to each other in the free refit as $K = 3$ in the routing task. Consequently, the fully-coupled EBM consists of only three couplings, while the tree has two. So the tree almost is an EBM in that sense. The routing floors (free refit) can therefore be read as indistinguishable rather than as ordered.

The waterfall diagrams in Figure 6.4 visualise the gaps between the ladder rungs. The step from the amortised to the free refit is the cost of amortisation alone, because target, data and optimiser are identical and only the parameterisation changes. This step is 0.0146, 0.0145 and 0.0120 nats on the blob and 0.0277, 0.0274 and 0.0183 nats on the routing task, which is around 40 to 50 % of every in-run gap. The remaining gap between the amortised refit and the in-run KL amounts to 0.0045, 0.0057 and 0.0133 nats on the blob, and 0.0204, 0.0252 and 0.0151 nats on the routing task. It has its origin in multiple factors of the training procedure and measurements, for instance, the noise of the score-function estimator, the β -annealing with a free-bits floor or the change from the training to test probe.

Consequence for the predictive metrics

None of this moves the accuracy, ECE, NLL or the Brier score on the two tasks as Tables 6.1 and 6.3 have shown. The correlation of the true posterior is not set by the recognition family but by the task, through the prior couplings and the likelihood (see Bayes’ theorem Equation (3.2)) by coupling gates whenever adapters have to act jointly or exclusively to express a class. The recognition model can only bend the true posterior towards what its family can represent through the jointly maximised ELBO. The blob and the routing task do not require correlated gates, so all arms converge to near-factorised posteriors which is reflected in the mean absolute correlation of 0.03 to 0.05. Consequently, the choice of the variational family does not influence the predictive metrics of these tasks.

In summary, this sub-experiment E1.3 verifies that the variational family hierarchy is real and clearly observable. The variational families work as they are supposed to by design

of the mixture model. These results, however, also identify the condition under which the choice of the family itself matters, which is the required posterior correlation of the task.

Ablation Study KL Posterior Gap

BLOB & ROUTING



Figure 6.4: Decomposition of the exact $\text{KL}(q \parallel p)$ gap of every trained arm on the blob and routing task

6.1.4 Interim Key Findings

The four key learnings of the E1 experiments, which the subsequent experiments build on, are summarised here:

- **Feasibility is established:** The mixture model reaches the Bayes ceiling on the blob task and solves the routing task that no single map can represent. The predictive uncertainty shows the geometry the construction predicts (E1.1, E1.2).
- **Input-dependent routing is only learned where the task requires it:** On the blob task, the trained gate ablations match the mixture on every metric, while on the routing task, the mixture leads them by 39 accuracy points (E1.1, E1.2).
- **The variational family hierarchy is real:** $\text{EBM} < \text{tree} < \text{mean-field}$ in exact KL holds on the designed target and at every rung of the ladder on the blob. No tree topology closes the structural gap of the tree to the EBM (E1.3).
- **Choice of variational family only affects predictive metrics when the task requires posterior correlation:** The blob and routing posteriors are nearly factorised (mean absolute correlation 0.03 to 0.05), so all three families make the same predictions with the

same confidence (E1.3).

6.2 E2: Competitiveness on Fashion-MNIST and CIFAR-10

In the preceding experiments, small artificially generated datasets were used to establish feasibility of the EBM-gated adapter mixture model, answering the first research objective of this project (see Chapter 2). They have demonstrated that the model learns, routes and reflects uncertainty as expected.

Next, the second research objective, evaluating the competitiveness on real data, is the focus of the E2 experiment. For this purpose, the mixture model is placed on top of a frozen convolutional trunk (see Chapter 5.2) and is compared with mean-field VI and MC dropout as the baselines (see Chapter 5.3). E2 states the fundamental results to assess its competitiveness and E3 conducts further sensitivity analyses to gain deeper insights into the results.

6.2.1 Setup

The experiment is run on two datasets, Fashion-MNIST and CIFAR-10. For each of them, its chosen convolutional trunk, LeNet-5 for Fashion-MNIST and ResNet-13 for CIFAR-10, has been pre-trained and then frozen as described in Chapter 5.2. The output layer of the trunk is removed and replaced by the gated adapter mixture, whose frozen base map W_0 receives the feature representation of the trunk with the dimension $D = 85$ for LeNet-5 and $D = 257$ for ResNet-13. For reference, the full deterministic LeNet-5 architecture achieves an accuracy of 0.900 on Fashion-MNIST and ResNet-13 an accuracy of 0.911 on CIFAR-10, which are the floor values the models are read against. Both architectures are visualised in Figure 6.5.

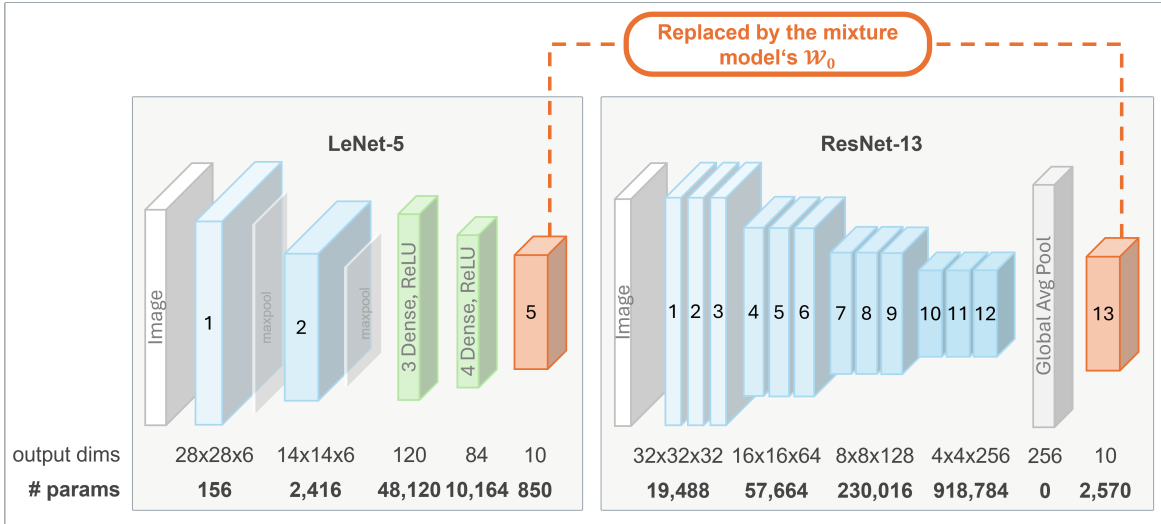


Figure 6.5: Convolutional trunks from LeNet-5 and ResNet-13. Last layers are replaced by the mixture model's W_0

Three variants of the mixture model and two baselines are trained and evaluated per dataset (summarised in Table 6.5). The three mixture models only differ in the used feature map $\psi(x)$ and base map W_0 . The first mixture model uses a random frozen base map W_0 whose entries are drawn independently from $\mathcal{N}(0, 0.1^2)$, so the gates and adapters must carry all class information. In comparison, the second mixture model uses the trained last layer of the trunks as its base map W_0 , so the adapters and gates only have to correct a base map that

already solves the task. The third mixture model does not use a convolutional trunk, so its base map W_0 is a fully-connected layer which reads the $D = 784$ and $D = 3072$ raw pixels of Fashion-MNIST and CIFAR-10, respectively. The base map weights are drawn from the same Gaussian with standard deviation 0.1. The third model is included to separate what the mixture contributes from what the trained features contribute.

Every mixture model is trained on the same configuration consisting of $K = 10$ gates and adapters of rank $r = 8$. The models use a tree-structured recognition model with the chain topology and the field network has 24 hidden units. The full list of all used hyperparameters is given in Appendix A.19.

The two baselines the mixture models are compared against, mean-field VI BNN and MC dropout (see Chapter 5.3), are trained on the same trunk. Mean-field VI uses a KL weight of 0.2 and MC dropout uses a dropout rate of 0.5.

Arm	Features $\psi(\boldsymbol{x})$	Base map W_0	Source of randomness	Trained parameters*	
				Fashion	CIFAR-10
<i>Gated adapter mixture</i>					
trunk, random W_0	frozen conv trunk	$\mathcal{N}(0, 0.1^2)$	$K = 10$ gates $\boldsymbol{z} \sim p_{\boldsymbol{\eta}}(\boldsymbol{z} \mid \boldsymbol{x})$	11 783	32 939
trunk, trained W_0	frozen conv trunk	trained output layer	$K = 10$ gates $\boldsymbol{z} \sim p_{\boldsymbol{\eta}}(\boldsymbol{z} \mid \boldsymbol{x})$	11 783	32 939
raw pixels, random W_0	pixels, $D = 784 / 3\,072$	$\mathcal{N}(0, 0.1^2)$	$K = 10$ gates $\boldsymbol{z} \sim p_{\boldsymbol{\eta}}(\boldsymbol{z} \mid \boldsymbol{x})$	97 760	379 184
<i>Baselines</i>					
mean-field VI BNN	trained, stochastic	—	Gaussian over every weight	123 176	2 454 602
MC dropout ($p=0.5$)	trained, deterministic	—	dropout masks at prediction	61 706	1 228 522

*Parameters that receive a gradient. For the mixture arms this excludes the frozen convolutional trunk (60 856 parameters for LeNet-5, 1 225 952 for ResNet-13). The two baselines train the trunk, so their count is the full network.

Table 6.5: The five arms of the base comparison. The mixture arms differ only in the feature map and in what the frozen base map is fixed at.

At prediction time, all five arms use 100 draws per test image, so no method benefits from a larger Monte Carlo budget than another. For the mixture arms, the gate pattern samples are drawn from the prior $p_\eta(z | x)$ by Gibbs sampling. Both in-distribution and OOD-mix results are calculated on the same 1 000 test images at $f = 0$ and $f = 0.5$. Each stated result number is the mean over 10 seeds.

6.2.2 Results

Tables 6.6 and 6.7 present the results of the five models on each dataset.

Accuracy

On Fashion-MNIST, the mixture models with the LeNet-5 trunk achieve an accuracy of 0.904 and 0.901, which is on the same level as the deterministic LeNet-5 floor of 0.900. The two baseline models reach the same level as well with an accuracy of 0.904 and 0.895 respectively. In comparison, the mixture model without the LeNet-5 backbone scores an accuracy of 0.817, so the gap of around 8 percentage points can be attributed to the missing convolutional trunk.

	in distribution ($f = 0$)				letter blend ($f = 0.5$)			
	Acc	ECE	NLL	Brier	Acc	ECE	NLL	Brier
<i>Gated adapter mixture</i>								
LeNet-5 random W_0	0.904 ± 2	0.033 ± 1	0.316 ± 5	0.146 ± 2	0.470 ± 8	0.114 ± 9	1.612 ± 28	0.696 ± 9
LeNet-5 trained W_0	0.901 ± 1	0.053 ± 2	0.389 ± 14	0.153 ± 3	0.465 ± 10	0.388 ± 10	3.752 ± 146	0.890 ± 18
Raw pixels random W_0	0.817 ± 2	0.060 ± 2	0.554 ± 3	0.267 ± 1	0.589 ± 5	0.074 ± 6	1.165 ± 7	0.551 ± 3
<i>Baselines</i>								
LeNet-5 MF-VI BNN	0.904 ± 2	0.025 ± 1	0.273 ± 3	0.139 ± 2	0.551 ± 13	0.136 ± 9	1.461 ± 59	0.611 ± 17
LeNet-5 MC dropout	0.895 ± 2	0.024 ± 2	0.291 ± 4	0.150 ± 2	0.580 ± 8	0.059 ± 6	1.248 ± 26	0.555 ± 8

Table 6.6: Base comparison on Fashion-MNIST. Subscripts give the standard error in units of the last digit, bold marks the column best

	in distribution ($f = 0$)				SVHN blend ($f = 0.5$)			
	Acc	ECE	NLL	Brier	Acc	ECE	NLL	Brier
<i>Gated adapter mixture</i>								
ResNet-13 random W_0	0.903 ± 1	0.009 ± 1	0.297 ± 2	0.141 ± 1	0.717 ± 4	0.075 ± 3	0.896 ± 12	0.402 ± 5
ResNet-13 trained W_0	0.911 ± 1	0.059 ± 1	0.420 ± 2	0.146 ± 1	0.733 ± 4	0.188 ± 4	1.507 ± 26	0.439 ± 6
Raw pixels random W_0	0.393 ± 1	0.015 ± 1	1.731 ± 2	0.743 ± 1	0.306 ± 4	0.052 ± 3	1.948 ± 3	0.813 ± 1
<i>Baselines</i>								
ResNet-13 MF-VI BNN	0.857 ± 1	0.050 ± 1	0.431 ± 2	0.211 ± 1	0.658 ± 3	0.040 ± 2	1.043 ± 9	0.472 ± 3
ResNet-13 MC dropout	0.912 ± 1	0.043 ± 1	0.335 ± 2	0.137 ± 1	0.737 ± 3	0.146 ± 3	1.155 ± 13	0.402 ± 4

Table 6.7: Base comparison on CIFAR-10. Subscripts give the standard error in units of the last digit, bold marks the column best

On CIFAR-10, the two ResNet-13-backed mixture models and the MC dropout model reach the deterministic floor of 0.911 as well. However, the mean-field VI network achieves an

accuracy of 0.857, which is around 5 percentage points lower. Furthermore, the raw pixels mixture model only reaches an accuracy of 0.393, which is significantly lower than the other models. Without a trunk, the model is a mixture of linear maps on pixels and is not competitive at all on CIFAR-10. It indicates that with increased data complexity and size, a pre-processed feature map significantly contributes to the accuracy of the architecture.

Calibration in Distribution

On Fashion-MNIST, the mixture model with the random W_0 is calibrated at the level of the two baselines. Its ECE of 0.033 sits slightly above the 0.024 of MC dropout and the 0.025 of the VI BNN, and its NLL of 0.316 nats slightly above the 0.273 nats of the VI BNN and the 0.291 nats of MC dropout, with the Brier score within 0.007 of the best. On CIFAR-10, the same mixture model is the best calibrated method in the table. Its ECE of 0.009 is a fifth of the 0.043 of MC dropout and the 0.050 of the VI BNN, its NLL of 0.297 nats is the lowest of all arms, and its Brier score of 0.141 is within 0.004 of MC dropout. These results show that the adapter mixture model is not only competitive in terms of accuracy, but also when comparing the uncertainty calibration metrics with the established baselines.

Under Distribution Shift

Under the 50% blend, the ECE values must be read carefully, showing again, as in Chapter 3.1.3 and E1.2, that they cannot be read alone. For example, the mixture on raw pixels has the second-best ECE on CIFAR-10 at an accuracy of 0.306, meaning that it predicts badly but is quite well calibrated. Consequently, the ECE results must be read together with the accuracy and the proper scoring rules. There it can be observed that the two datasets do not rank the mixture models against the baselines in the same way. On Fashion-MNIST, the baseline models are better calibrated than the LeNet-5 mixture model with the random W_0 in NLL, 1.612 nats against 1.461 and 1.248 nats, and the Brier score, 0.696 against 0.611 and 0.555. On CIFAR-10, on the other hand, the ResNet-13 with the random W_0 is equally or better calibrated than the MC dropout model (equal Brier score of 0.402 and better NLL of 0.896 nats against 1.155 nats). Therefore, the behaviour under distribution shift will be further investigated in experiment E3.2.

6.2.3 Interim Key Findings

The key findings of the E2 experiment are summarised here:

- **In distribution, the mixture model matches the deterministic floor accuracy:** On the trunk features, the mixture models reach the accuracy of the deterministic trunk and of MC dropout on both datasets.
- **In distribution, the mixture model is competitive in terms of calibration:** The mixture with the random W_0 is calibrated at the level of the two established baselines on Fashion-MNIST and is the best calibrated method on CIFAR-10 in ECE and NLL.
- **Under distribution shift, competitiveness is ambiguous:** The ordering of the mixture against the baselines at $f = 0.5$ differs between the two datasets and will be therefore further analysed in E3.2.

6.3 E3: Operating Regime and Sensitivity

The experiments conducted in this section are based on the results of E2, analysing them more in-depth to gain further valuable insights.

6.3.1 E3.1: Base Map W_0 Sensitivity

This sub-experiment E3.1 focuses on the first two rows of the base tables in E2, the mixture models with a convolutional trunk with a random and trained base map W_0 (see Tables 6.6 and 6.7). There it can be seen that the random W_0 model achieves an accuracy equivalent to or slightly lower than the trained W_0 model (0.904 against 0.901 on Fashion-MNIST and 0.903 against 0.911 on CIFAR-10), while being clearly better calibrated in both datasets in terms of ECE, NLL and Brier score. This is the subject of study in this sub-experiment by sweeping the base map W_0 between the two endpoints of being fully trained or random to understand when the input-dependent gating mechanism engages and what it contributes.

6.3.1.1 Setup

The setup is exactly the same as in E2 as it builds on top of it. The only new variable introduced is α , defining the frozen base map W_0 via

$$W_0(\alpha) = (1 - \alpha) W_0^{\text{trained}} + \alpha W_0^{\text{random}}, \quad \alpha \in [0, 1], \quad (6.1)$$

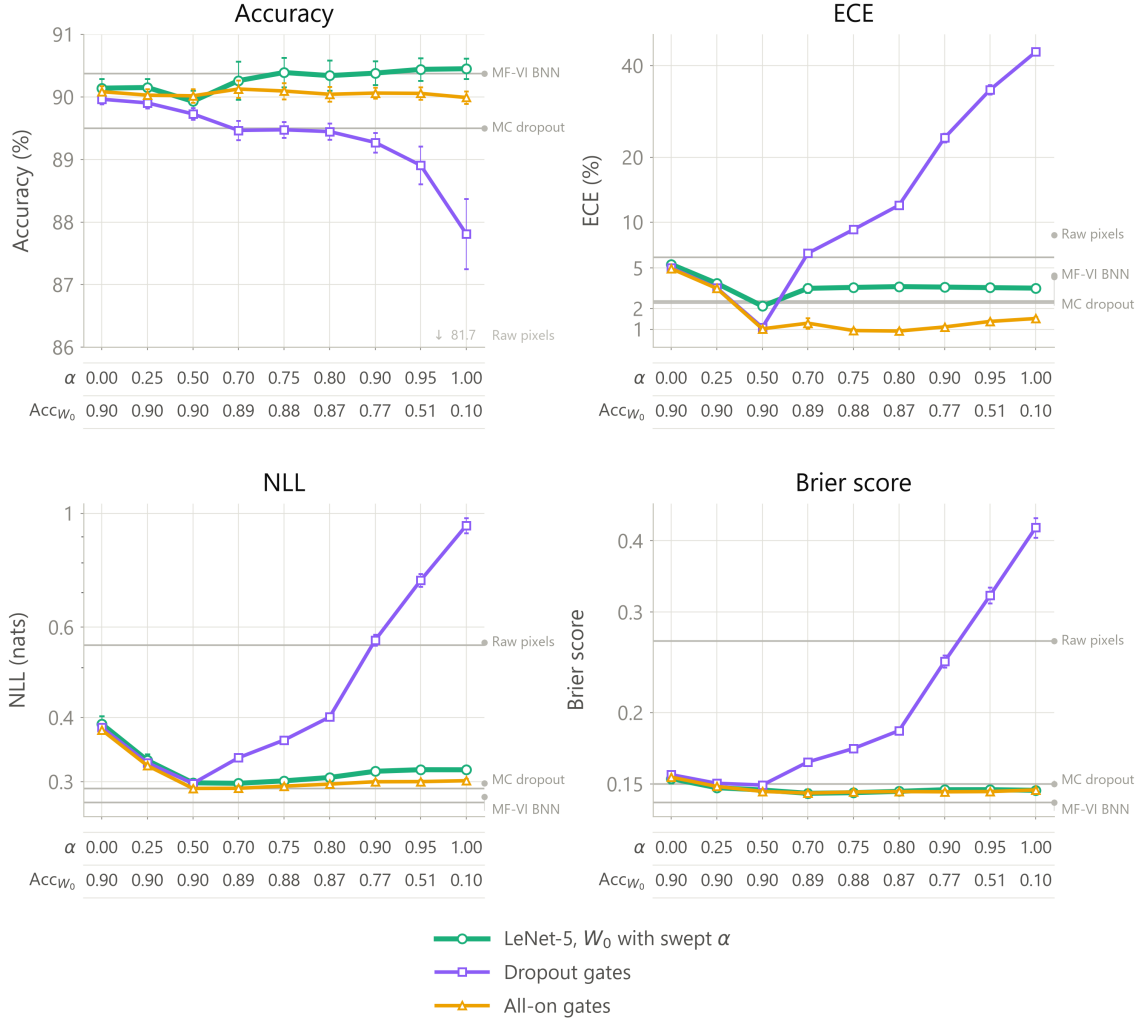
where W_0^{trained} is the output layer of the trained trunk and W_0^{random} the random base map as explained in E2 (see Chapter 6.2.1). The share α therefore changes the degree of randomness in the frozen base map. An α value of 1 is therefore creating an entirely random base map W_0 , so the model is equivalent to the mixture model with the random W_0 (first row in the base tables). Using $\alpha = 0$, on the other hand, creates the mixture model with the entirely trained W_0 (second row). The values for α are swept across a grid of nine values, $\alpha \in \{0, 0.25, 0.5, 0.7, 0.75, 0.8, 0.9, 0.95, 1\}$ and for each of the α -values the accuracy the base map reaches on its own without adapter-gated routing (Acc_{W_0}) is computed as a reference. Alongside the two mixture models the two gate ablations *dropout gates* and *all-on gates*, introduced in E1.1 (see Chapter 6.1.1.2), are trained at every alpha value as well.

6.3.1.2 Results

Figure 6.6 presents the four result metrics for the mixture model and the two ablation models under the α -sweep for Fashion-MNIST. The mixture model is depicted in green, the dropout gates in purple, all-on gates in orange and, for reference, the constant levels of the baseline methods mean-field VI and MC dropout have been added to the graphs in grey. The accuracy of the base maps under different α -values is stated below the α -values in the plots. As the models show exactly the same dynamics on both datasets, the analysis only focuses on Fashion-MNIST here. Nevertheless, the same plots are presented for CIFAR-10 in Appendices A.20 and A.21.

Variations of Share α of the Random W_0

FASHION-MNIST

Figure 6.6: Accuracy, ECE, NLL and Brier score under the α -sweep on Fashion-MNIST

Accuracy and Calibration Dynamics

The accuracy plot shows that the accuracy of the mixture model (green line) is invariant to the base map competence. Its accuracy moves between 0.899 and 0.905 across all α -values, while the base map accuracy Acc_{W_0} drops from 0.90 at $\alpha = 0.50$ to 0.10 at $\alpha = 1.00$. This shows that whatever the base map loses due to its increased random share, the adapters absorb. This is in line with the mean gate-on rate of the mixture model across the α -sweep that can be seen in Figure 6.7. It shows that the mean gate-on rate is at 0.093 at $\alpha = 0.0$ and increases with growing α -values to 0.308 at $\alpha = 1.0$. That means that on a trained base map, which on its own already has a high accuracy, the gates are mostly switched off. Once the base map accuracy drops, the mixture model starts to absorb the drop by turning on the gates more frequently. The gate-on rate can be therefore viewed as a metric of mixture model activity. In summary, this shows the operating regime of the mixture model which is where the base map does not solve the task on its own.

The calibration of the mixture model, on the other hand, improves up to $\alpha = 0.50$ and then

saturates at an ECE of around 0.033, an NLL of around 0.31 nats and a Brier score of around 0.14.

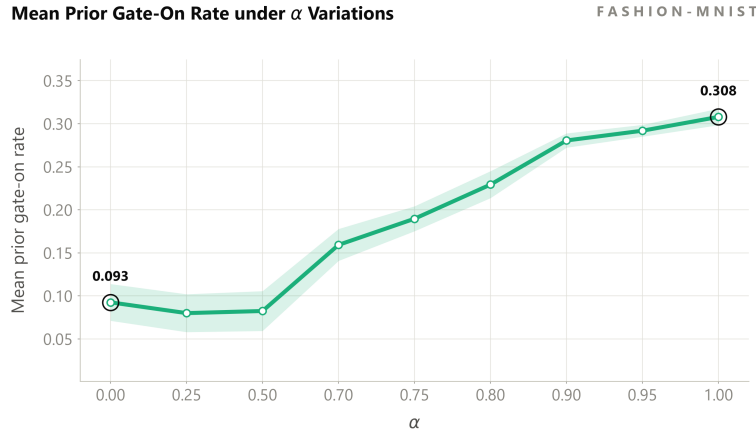


Figure 6.7: Gate-on rate under the α -sweep on Fashion-MNIST

Ablation Results

Next, the two gate ablations (orange and purple lines) are taken into comparison with the mixture model. Regarding the dropout gates model (purple), it can be observed that up to $\alpha = 0.50$, accuracy is mostly stable and the calibration metrics move in the same direction as those of the mixture model. But once the base map accuracy starts to drop at $\alpha = 0.50$ and the mixture model activity rises, the input-dependent gating mechanism comes into play. Consequently, once the adapters carry class information, the gate patterns that have been learned in training and produced by the prior at test time are input-dependent, so that a random composition of the gates and adapters is not solving the task. This finding demonstrates that the input-dependent gating mechanism of the mixture model is working as expected, as it also has been shown in E1.2.

The all-on gates ablation (orange) moves closely with the mixture model across all α -values and is slightly better calibrated, but pays for it with slightly worse accuracy. For example at $\alpha = 1.00$, its Brier score is 0.003 points and the accuracy is 0.46 percentage points below the mixture model. The closeness of the two lines, however, shows that one dense head on the trunk features containing all adapters can solve the task, so that in general routing only adds minor accuracy and calibration value for these datasets. These results are similar to the blob task in E1.1, showing that input-dependent routing is not essentially required for Fashion-MNIST and CIFAR-10.

6.3.2 E3.2: Uncertainty Behaviour under Distribution Shift

In experiment E2, the models were compared at two OOD-mixture points, $f = 0$ and $f = 0.5$. Therefore, this sub-experiment E3.2 analyses the results of the models under a full ladder $f \in \{0, 0.1, \dots, 0.9\}$. For this purpose, each model is evaluated across all four metrics at every step of the ladder to assess the degradation in the results as the OOD-share grows and whether the competitiveness observed in distribution carries over to out-of-distribution inputs. Additionally, the predictive entropy is split into its aleatoric and epistemic part to evaluate each model's behaviour under distribution shift.

The setup is otherwise equivalent to E2 (see Chapter 6.2.1) and all results are stated as a mean of 3 seeds.

6.3.2.1 Results

Figures 6.8 and 6.9 present the four result metrics along the OOD-blend. The mixture model with the random base map W_0 is given in green, while the one with the trained W_0 is in purple. Both baselines are presented as well: The mean-field VI model is shown in yellow, while the MC dropout model is shown in red.

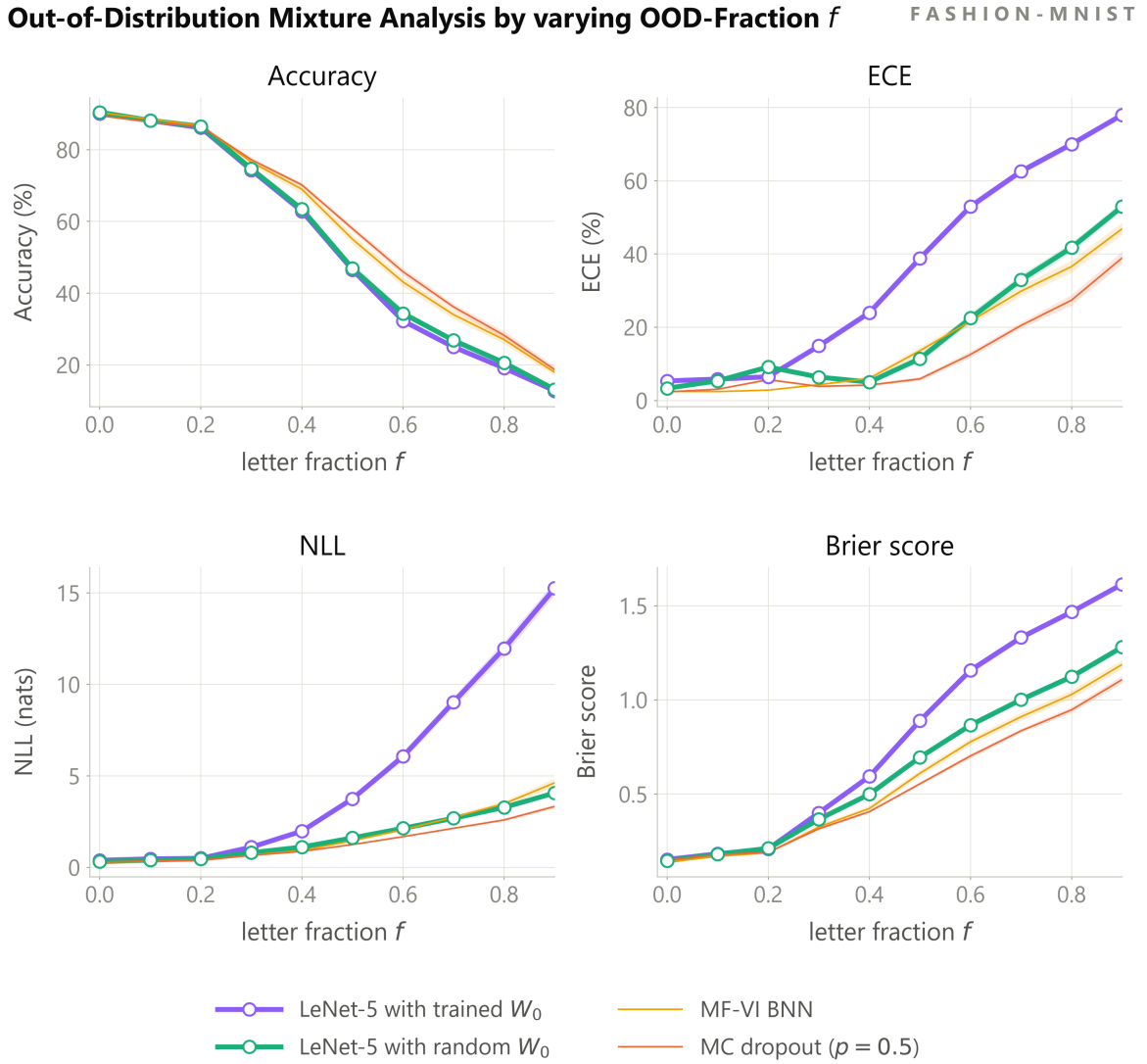


Figure 6.8: Accuracy, ECE, NLL and Brier score under the f -sweep on Fashion-MNIST

Accuracy and Calibration

In the accuracy plot for Fashion-MNIST, all four models lose accuracy with almost the same rate up to $f = 0.3$. Beyond that point, the two baseline models are able to uphold a higher accuracy of 0.551 for the VI BNN and 0.580 for MC dropout against the mixture models standing at 0.470 and 0.465 at $f = 0.5$. This gap stays between 5 and 10 percentage points

until the end of the ladder. In comparison, on the CIFAR-10 dataset, the mixture models move at very similar rates as the baselines and even score consistently higher on accuracy than the MC dropout baseline model along all f -values. Consequently, there is no clear evidence whether the mixture models or baseline models perform better in terms of accuracy under distribution shift. However, what holds over both datasets is that the two mixture curves lie on top of each other throughout, so the gating mechanism neither costs nor buys accuracy under distribution shift.

Out-of-Distribution Mixture Analysis by varying OOD-Fraction f

CIFAR-10

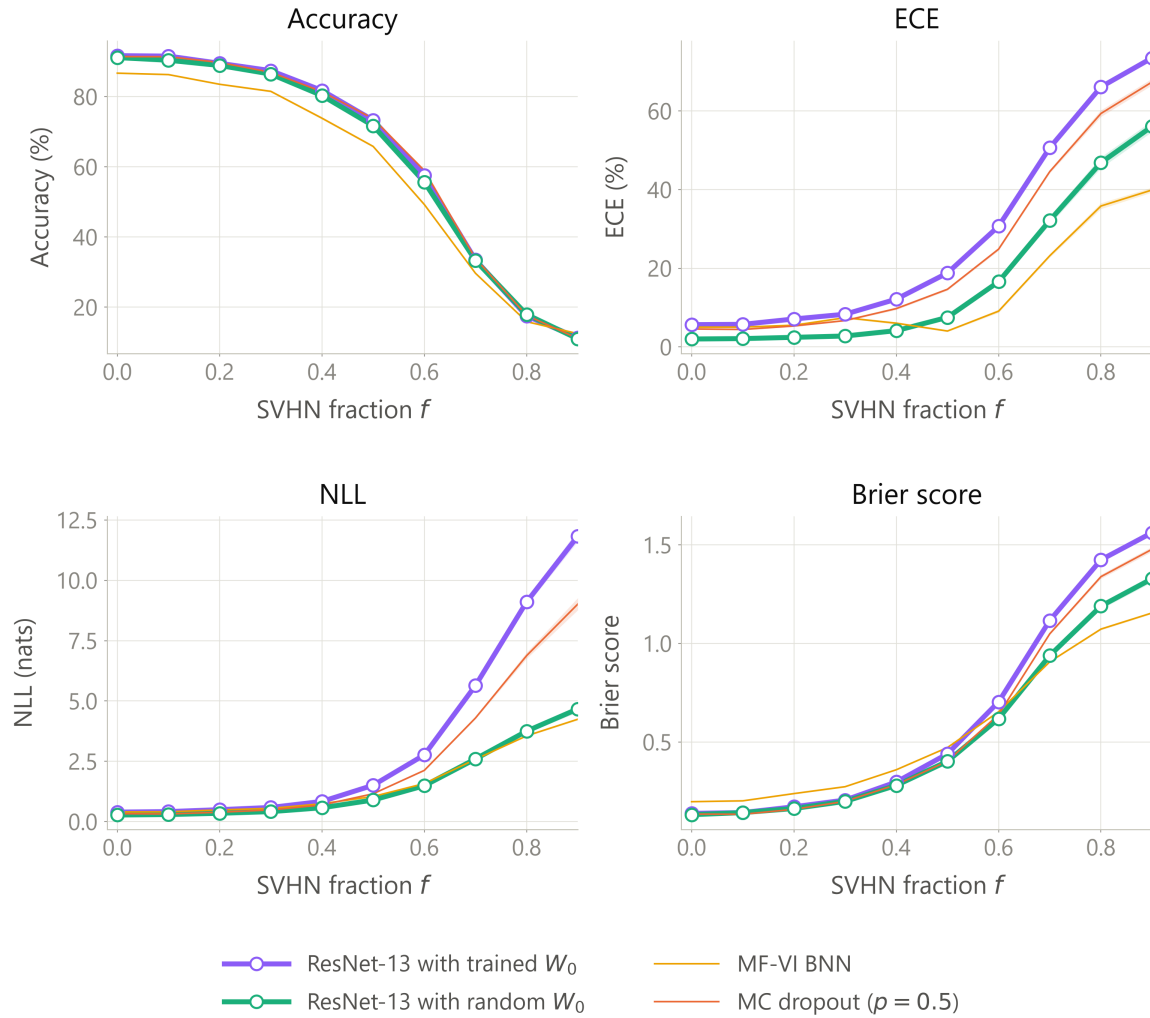


Figure 6.9: Accuracy, ECE, NLL and Brier score under the f -sweep on CIFAR-10

Over all three calibration metrics, the two mixture models separate significantly for $f \geq 0.3$ on both datasets. On the Fashion-MNIST task, the mixture model with the trained W_0 reaches an ECE of 0.780 and NLL of around 15.26 nats at $f = 0.9$. The mixture model with the random W_0 , on the other hand, receives an ECE of 0.528 and NLL 4.07 nats at $f = 0.9$. Similar results hold on CIFAR-10 where the mixture model with the random W_0 receives an ECE 17.42 percentage points lower and an NLL 7.161 nats lower than the one with the trained W_0 . As shown in E3.1 (see Chapter 6.3.1), the routing mechanism is barely active in the mixture model with a trained W_0 ($\alpha = 0$), so its results produced here are close to

a deterministic LeNet-5 or ResNet-13 architecture. Consequently, the better calibration of the mixture model with the random W_0 can be attributed to its active routing mechanism. Furthermore, it is observable that the mixture model with the random W_0 stays close to the baselines across all values for f , all metrics and both datasets, thereby indicating its competitiveness also under distribution shift and answering the open question from E2.

Uncertainty Decomposition

Figure 6.10 splits the predictive entropy of every model into its aleatoric (lower band) and epistemic part (upper band) on the Fashion-MNIST dataset. All y-axes are on the same scale. The corresponding plot for CIFAR-10 is presented in Appendix A.22.

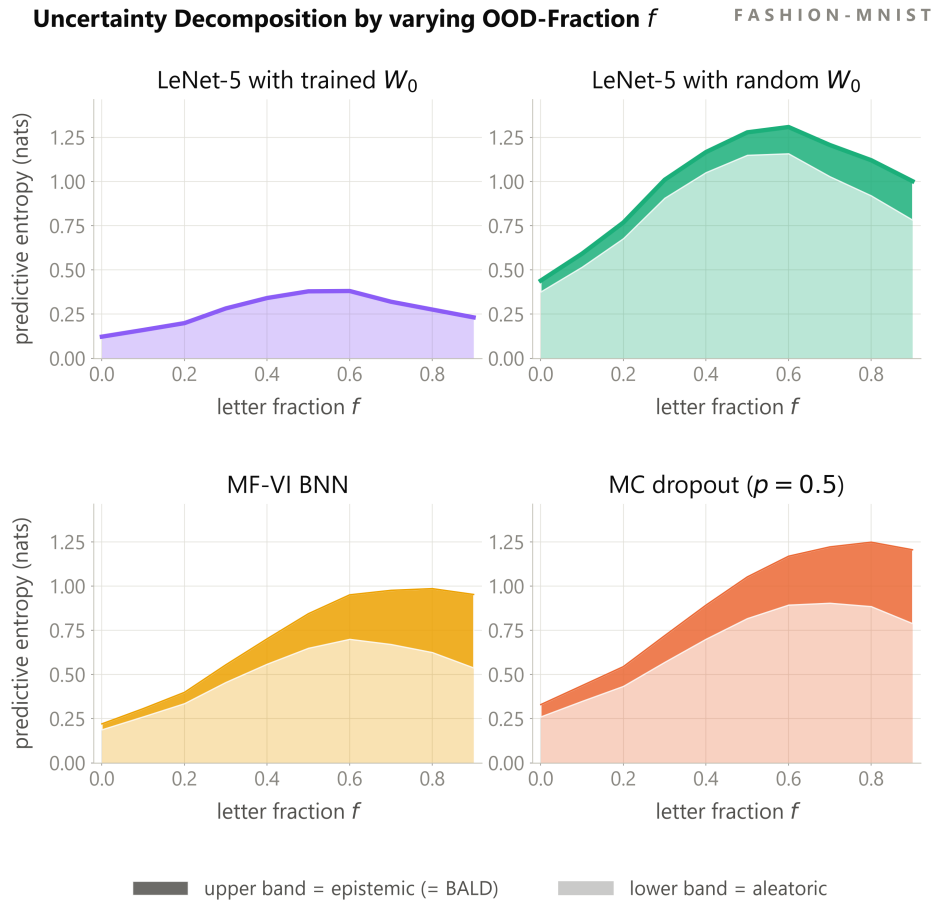


Figure 6.10: Uncertainty Decomposition under the f -sweep on Fashion-MNIST

The mixture model with the trained W_0 (purple) reports a very low epistemic uncertainty across all letter blends with values between 0.0003 and 0.0007 nats. This is reasonable because the gates are mostly off, as seen in E3.1 (see Chapter 6.3.1), so there is nothing to disagree on. Consequently, the uncertainty between the sub-models is low.

Across all four models and both datasets, the predictive entropy rises with the blend fraction over the first two thirds of the ladder. That means that every model reports more uncertainty the more images overlap. Beyond $f = 0.6$, the total uncertainty plateaus in the baseline models, while it declines in the mixture models. The fall in the mixture model with the random W_0 (green) does not come from the epistemic term, which continues to

grow from 0.1507 nats at $f = 0.6$ to 0.2193 nats at $f = 0.9$, but from the aleatoric part, which significantly declines from 1.1578 nats to 0.7828 nats. Across both datasets, the epistemic uncertainty consistently grows with increasing f , reflecting the increasing model disagreement with higher EMNIST or SVHN blend on each dataset respectively.

6.3.3 E3.3: Capacity Scaling Sensitivity

All results on Fashion-MNIST and CIFAR-10 were obtained under the same capacity of the mixture so far, $K = 10$ gates and adapters of rank $r = 8$. This sub-experiment E3.3 conducts a sensitivity analysis to assess whether increasing or decreasing the capacity influences the accuracy and calibration.

6.3.3.1 Setup

The mixture model with the random W_0 is run over a grid of $K \in \{1, 2, 3, 4, 5, 6, 8, 10, 15, 20, 30, 50\}$ and $r \in \{1, 2, 3, 4, 6, 8, 10, 16, 24, 36\}$, resulting in 120 configurations per model. Both axes raise the expressive capacity of the mixture, since by Equation (4.1) the rank r bounds how far each adapter $B_k A_k$ can move the frozen base map W_0 , while the number of gates K sets the 2^K gate patterns and thus the number of distinct classifiers $W(z)$ the mixture can select between. All other hyperparameters are set as in experiment E2 (see Chapter 6.2).

The grid is run over three models with different levels of the feature map $\psi(x)$. The first model is exactly the same as in E2 using the full LeNet-5 convolutional trunk of 61.7k parameters. The second model uses also the LeNet-5 convolutional trunk, but its layer width is scaled by $w = 0.25$, leaving the trunk with 6.7k parameters that are trained and frozen the same way. The third model is the raw pixels model of E2 with no trunk at all, where the random W_0 reads the pixels directly.

The reason why the grid is run over all three levels is to assess whether and how the capacity of the mixture model must be increased to absorb the weaker features the mixture model receives as inputs.

For wall-clock time complexity reasons, these runs are evaluated over the in-distribution images of Fashion-MNIST, and only accuracy and the Brier score are reported for all combinations.

To compare the required expressive capacity of the mixture model of each level, a *saturation boundary* is defined as the combination of K and r where the accuracy no longer increases and the Brier score no longer decreases. Since the results are computed as the mean over 3 seeds, the saturation boundary must have some tolerance. Therefore, a cell (K, r) is defined as saturated if its value lies within 5 % of the best value in the grid, measured on the error rate for accuracy and on the Brier score directly. The saturation boundary then separates the saturated from the unsaturated cells.

6.3.3.2 Results

Figure 6.11 presents the accuracy values and Brier scores on the $K \times r$ grid, while Figure 6.12 visualises the results on a 3D surface plot to show the plateau region and the saturation boundary.

Capacity Scaling Study by Varying Number of Gates K and Adapter Rank r
LeNet-5 Trunk with $w = 1$ (# of Parameters: 61.7k)

FASHION-MNIST

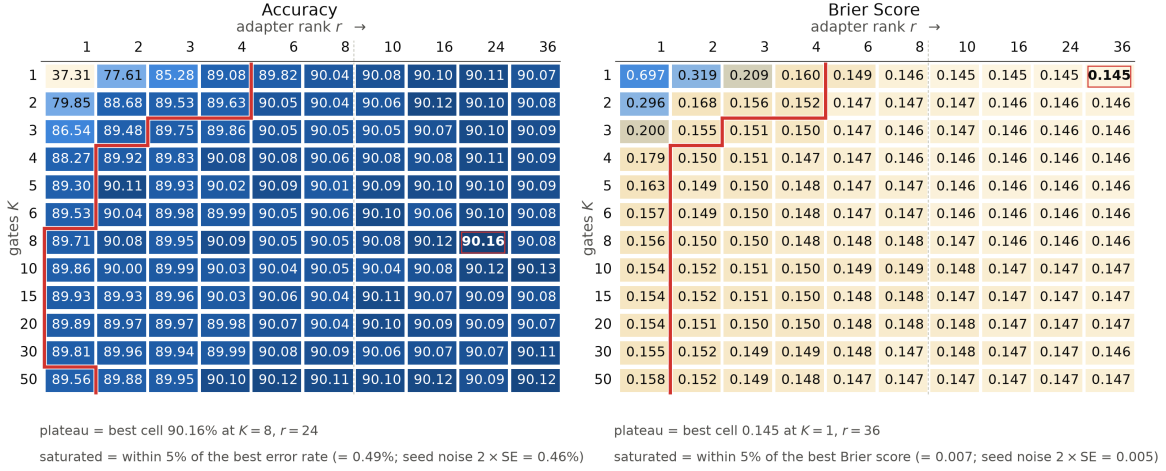


Figure 6.11: Accuracy and Brier score of the mixture model with the full LeNet-5 trunk ($w = 1$) over the $K \times r$ grid

Capacity Scaling Study by Varying Number of Gates K and Adapter Rank r
LeNet-5 Trunk with $w = 1$ (# of Parameters: 61.7k)

FASHION-MNIST

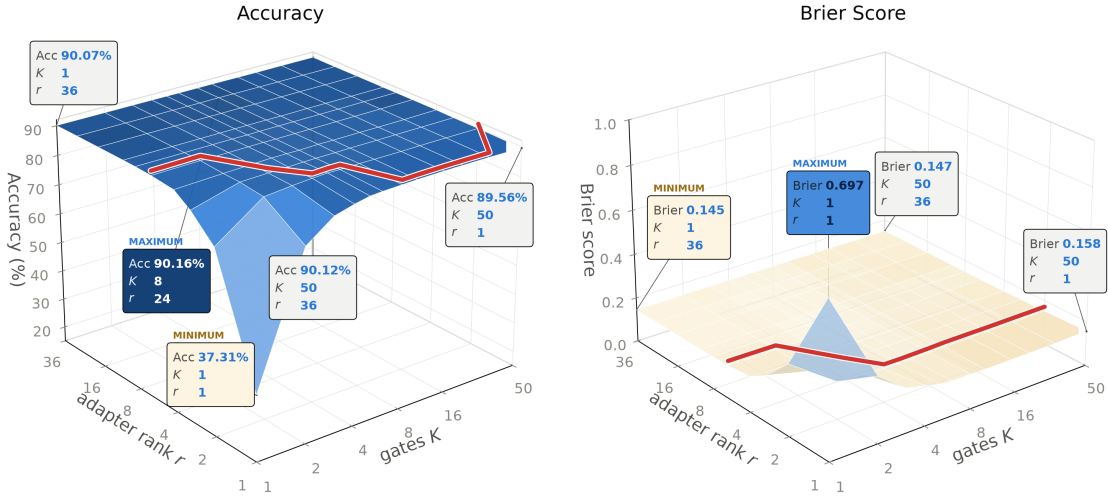


Figure 6.12: Accuracy and Brier score of the mixture model with the full LeNet-5 trunk ($w = 1$) over the $K \times r$ grid visualised on a 3D surface plot

Full Convolutional Trunk ($W = 1$)

The grid is dominated by a large plateau region, in which no direction of the grid is preferred. The E2 configuration of $K = 10$ and $r = 8$ therefore sits deep inside the plateau region and it shows that adding capacity from there does not improve the accuracy or the Brier score significantly.

The unsaturated region is confined to the upper left corner and the saturation boundary follows a line of a roughly constant product $K \times r$. For accuracy, saturation is reached at

$r = 6$ for $K \leq 2$, at $r = 3$ for $K = 3$, at $r = 2$ for $K = 4$ to 6 and at $r = 1$ from $K = 8$, so a combination of $K \times r \approx 6$ to 9 is required by the task given the full convolutional trunk.

The Brier score reproduces the plateau at 0.145 to 0.152, with a boundary of roughly the same shape. However, it shows that in the column $r = 1$, it does not saturate at any value for K , so it asks for slightly more rank than the accuracy does.

Saturation Boundary Comparison

After computing the three saturation boundaries, they can be compared in one consolidated plot, which is shown in Figure 6.13.

The consolidated plot shows that the saturation boundary of the model with the weakened LeNet-5 trunk (yellow line) is almost not distinguishable from the model with the full convolutional LeNet-5 trunk (blue). Only the model without any convolutional trunk (purple) appears to have a saturation boundary shifted to the bottom right corner compared to the models with a convolutional trunk. However, the effects are rather small.

Comparison of the Saturation Boundaries under Backbone Weakening

FASHION-MNIST

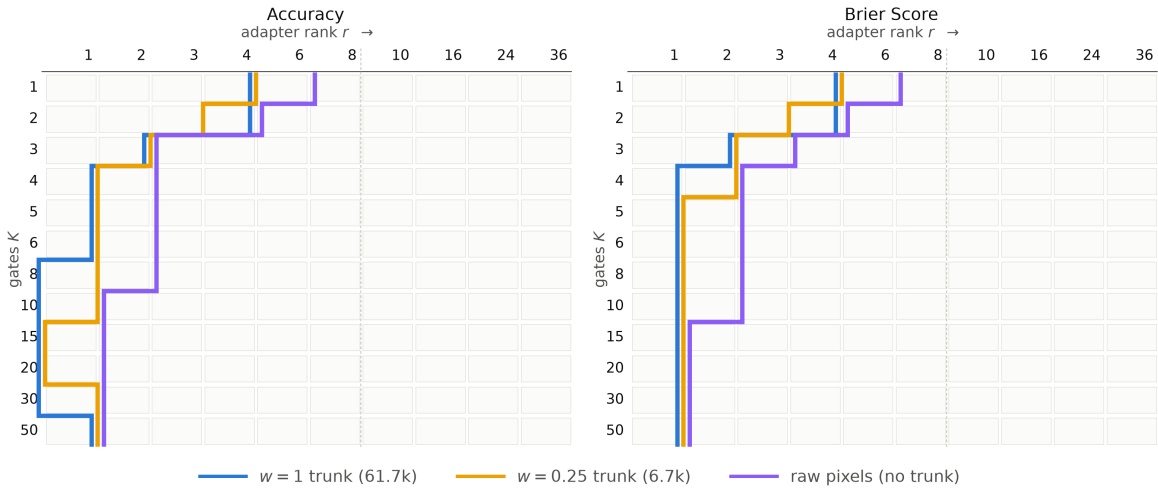


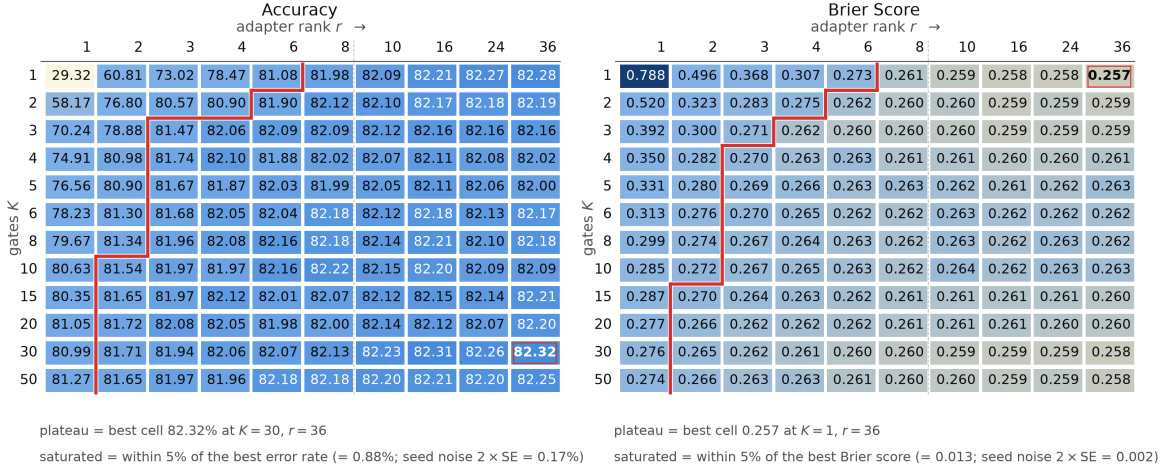
Figure 6.13: Saturation boundary comparison over all three models on Fashion-MNIST

The dominant effect that can be observed is when comparing the plateau height of the raw pixel model, shown in Figure 6.14, with, for instance, the model with the full convolutional trunk. The raw pixel model only reaches an accuracy of 82.32% and a Brier score of 0.257 in its best configurations, which is significantly worse than the accuracy of 90.16% and Brier score of 0.145 of the model with the LeNet-5 trunk. This shows that the model is reaching its plateau at low $K \times r$ configurations, from which point onward, increasing the expressive capacity does not benefit the accuracy or calibration and does not allow it to reach the same heights as with a convolutional trunk.

For completeness, the full $K \times r$ grid for the model with the $w = 0.25$ trunk is provided in Appendix A.23.

Capacity Scaling Study by Varying Number of Gates K and Adapter Rank r
Raw Pixels — No Trunk

FASHION-MNIST

**Figure 6.14:** Accuracy and Brier score of the mixture model without a convolutional trunk over the $K \times r$ grid

6.3.4 Interim Key Findings

The key findings of the E3 experiments are summarised here:

- **Gating mechanism engages only under an incompetent base map:** Under a fully-trained base-map and trunk, the gating mechanism is inactive. Only once Acc_{W_0} falls, the mixture model's mean gate-on rate increases and the model absorbs the loss through its mechanism successfully with better calibration (E3.1).
- **Input-dependent routing works as designed:** The input-dependent routing mechanism works as designed (dropout gates fails), although the tasks, in general, do not require it (all-on gates). However, routing adds predictive uncertainty, which the all-on model cannot provide (E3.1).
- **Gating mechanism buys calibration under distribution shift:** Under distribution shift, the actively routing mixture model with random W_0 moves similarly in accuracy as the near-deterministic model (trained W_0), but better in calibration and on an equal level as the baseline models (E3.2).
- **Mixture model successfully disagrees under distribution shift:** Epistemic uncertainty grows monotonically with the blend fraction on both datasets, while the fall in predictive entropy beyond $f = 0.6$ comes entirely from the aleatoric part (E3.2).
- **Feature map sets the accuracy ceiling, mixture model reaches it with little expressive capacity:** On Fashion-MNIST, the accuracy and calibration saturate at low number of gates K and adapter rank r to the level the convolutional trunk and base map indirectly define (E3.3).
- **No clear indication of a scaling law under weakened feature map:** Weakening the

feature map moves the saturation boundary only marginally, while significantly impacting accuracy and calibration (E3.3).

6.4 E4: Energy-Based Recognition Model

In the E2 and E3 experiments on Fashion-MNIST and CIFAR-10, the recognition model used was a chain-structured tree. Next, the E4 experiments exchange the variational family of the recognition model with an EBM.

6.4.1 E4.1: Direct Comparison of Tree and EBM Recognition Model

In this sub-experiment E4.1, the $K \times r$ study of E3.3 is repeated on the model with the EBM recognition model to directly compare the influence of the variational family choice. Moreover, the posterior fidelity of the different variational families to their exactly enumerated posterior is assessed to analyse the learned correlation structure of the posterior.

6.4.1.1 Setup

The setup is the same as of E3.3 using the same $K \times r$ grid and the model with the full LeNet-5 convolutional trunk with one change. A full-pairwise connected EBM family of Chapter 4.4.2 replaces the tree family in the recognition model and runs over $S = 12$ sweeps per training step. The resulting $K \times r$ grid is then compared with the grid of the model with the tree recognition family of E3.3 to evaluate the influence of the variational family choice on real data. All results are stated as the mean over 3 seeds.

Additionally, to explain the results of the grid comparison, the posterior fidelity of each variational family, including the tree and a mean-field, to its own posterior is computed for a $K = 10$ and $r = 8$ configuration by exact enumeration of the 2^{10} gate patterns on 2000 test images. This builds the counterpart on real data to the E1.3 experiment on the blob and routing task (see Chapter 6.1.3).

6.4.1.2 Results

Figure 6.15 visualises the difference between the models with the EBM and the tree recognition model in accuracy and Brier score over the full $K \times r$ grid. The blue cells highlight where the EBM has a higher accuracy and lower Brier score, while the red cells highlight where the tree is ahead. Furthermore, the purple line is the saturation boundary as shown in E3.3 for the model with the tree recognition model and the red line is the new saturation boundary of the model with the EBM recognition family.

Grid Value and Absolute Plateau Height Comparison

On average over the full grid, the EBM recognition model achieves a slightly higher accuracy of 0.07 percentage points than the tree and a marginally lower Brier score of 0.001 less. That means that the difference between the recognition models is comparably low. In absolute plateau heights, the EBM recognition model reaches an accuracy of 90.21% and a Brier score of 0.144 (as shown in Appendix A.24), while the tree achieves an accuracy of 90.16% and a Brier score of 0.145 in their best configuration. Consequently, the effects on the accuracy and calibration metric can be summarised as insignificant.

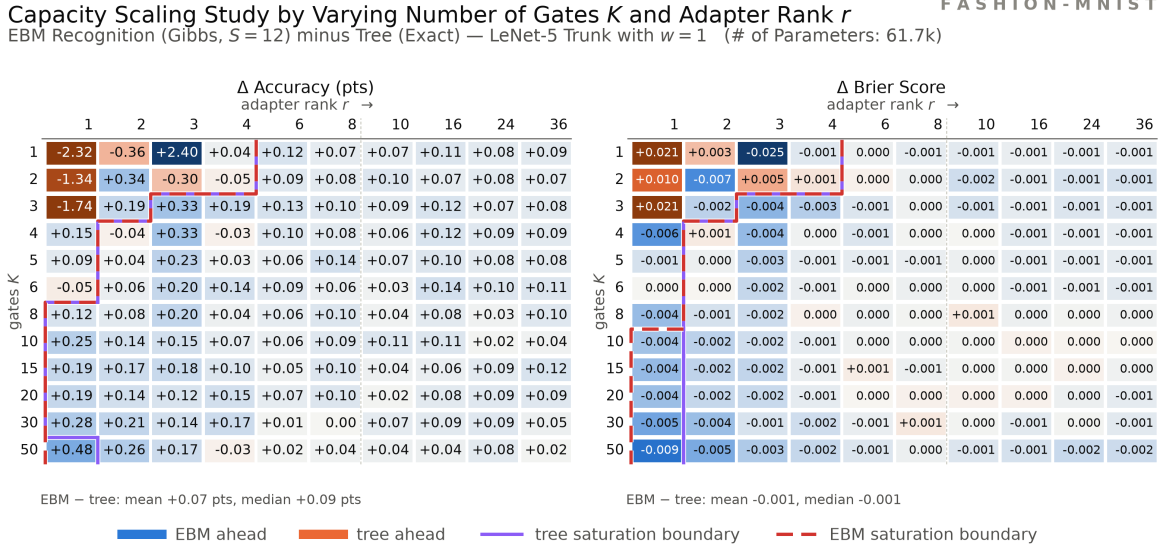


Figure 6.15: Accuracy and Brier score difference of the EBM and tree recognition model over the $K \times r$ grid

A similar conclusion can be drawn for the saturation boundary comparison, where the purple and the red line follow each other very closely. The only real difference observable is in the Brier score table where the tree's saturation plateau starts at $r \geq 2$ for $K \geq 10$, while for the EBM it also includes $r = 1$ for $K \geq 10$.

Posterior Fidelity Comparison

Table 6.8 summarises the KL-gap between each model's true posterior and its recognition posterior as well as the mean absolute correlation of the true posterior. The second column shows that the model which uses an EBM recognition model during training also creates a true posterior with the most correlation structure out of the three models, having a mean absolute correlation of 0.047 compared to 0.042 and 0.033 for the model with the tree and mean-field recognition model respectively.

The EBM recognition model has the lowest KL-gap to the true posterior of its mixture model with 0.053 nats. This is in line with the results on the artificial data of E1.3, showing the EBM is working as expected also on real data.

However, since the mean absolute correlation values of all three models are very low, it means that there is not much correlation structure to be learned for the two datasets. This explains why the model with the EBM recognition model only performs slightly better in the evaluation metrics compared to the model with the tree-structured recognition model.

6.4.2 E4.2: Gibbs Sweep Sensitivity

As explained in Chapter 4.4.2, when changing the variational family of the recognition model from a tree to an EBM, the quantities the training objective requires are not available in closed-form anymore, but must be computed from T samples. These samples are drawn using S_q Gibbs sweeps, which is a key hyperparameter of the model as setting it too low could provide samples whose chain has not fully-mixed yet, while setting it too high makes the

Recognition model	$\text{KL}(q_\phi \ p)$ [nats]	$ \text{corr} _p$	corr. lost
mean-field	$0.134_{\pm 7}$	0.033	0.033
tree (chain)	$0.104_{\pm 3}$	0.042	0.016
EBM (full pairwise)	$0.053_{\pm 3}$	0.047	0.000

Table 6.8: Posterior fidelity of the trained recognition models on Fashion-MNIST at $K = 10$, $r = 8$, measured against each arm's own exact posterior by enumeration of the 2^{10} gate patterns on 2 000 held-out images. $|\text{corr}|_p$ is the mean absolute gate correlation the exact posterior carries, corr. lost the share of it the family cannot represent. Subscripts give the standard error in units of the last digit, bold marks the column best.

whole algorithm slower and increases computational costs. The question of this small sub-experiment to answer is therefore how many Gibbs sweeps the recognition model requires during training.

6.4.2.1 Setup

The setup is that of E4.1 on Fashion-MNIST with the full LeNet-5 trunk and the random base map W_0 (see Chapter 6.2.1). The model uses the EBM recognition model of Chapter 4.4.2 and $T = 64$ chains per input. The new variable is the number of Gibbs sweeps S_q and the model is swept over $S_q \in \{1, 2, 3, 4, 6, 8, 12, 16, 24\}$ in the three configurations $(K, r) \in \{(10, 8), (10, 1), (5, 1)\}$.

For each configuration and sweep, a model is compared against a reference model which uses an idealised sampler. That means that in the reference model, the Gibbs chain is replaced by exact enumeration of the 2^K gate patterns, so that the moments, the entropy gradient and the recognition score are computed exactly instead of estimated from chains. By doing so, it can be compared from which S_q onward a sampled EBM recognition model produces stable samples for the training.

Besides the usual four metrics, the posterior fidelity $\text{KL}(q_\phi \| p)$ is measured as in E4.1 by exact enumeration on 2 000 held-out images.

6.4.2.2 Results

In Figure 6.16, the reference model is drawn as the zero line in all five plots. Each coloured line represents one of the three configurations.

All five plots show a consistent picture, indicating that from $S_q = 3$ to 4 on, the metrics stabilise around the value of the reference model. Consequently, the Gibbs sweep budget of $S_q = 12$ used in the previous setting is more than sufficient for this task and under these configurations.

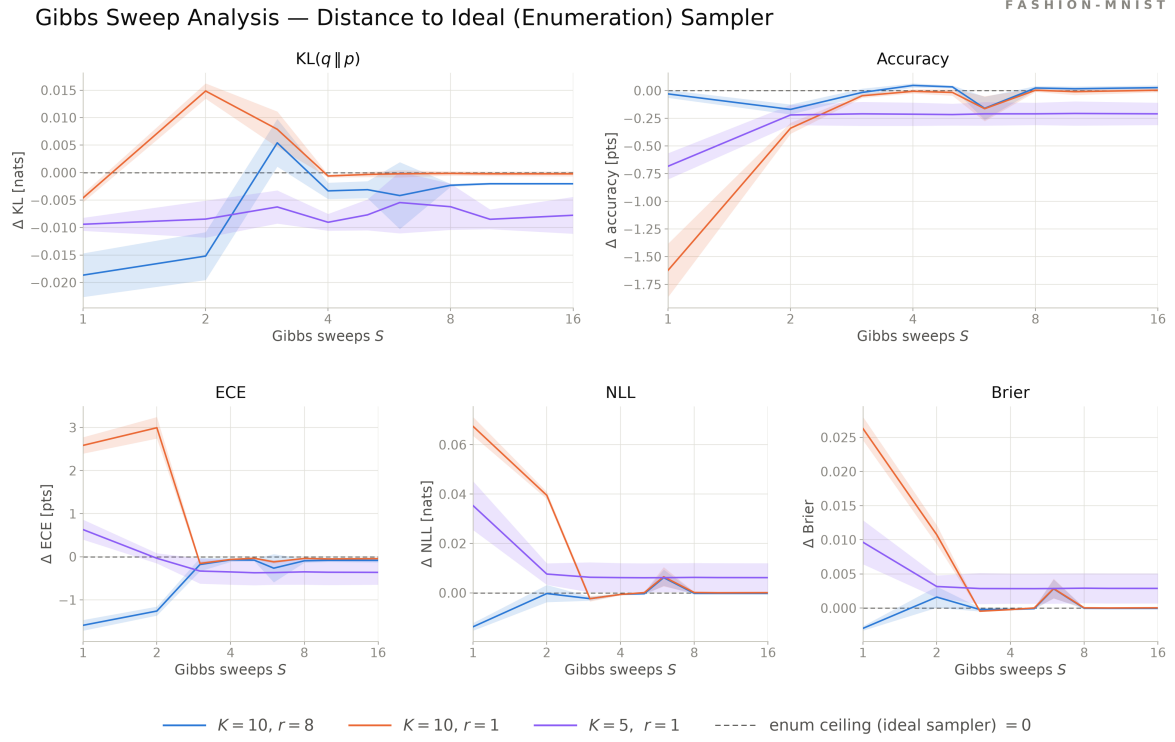


Figure 6.16: Gibbs sweeps results over three configurations on Fashion-MNIST

6.4.3 Interim Key Findings

The key findings of the E4 experiments are summarised here:

- **Choice of recognition family does not impact the results:** On Fashion-MNIST and CIFAR-10, learned posteriors carry little correlation between gates. Consequently, the choice of the variational family has no significant impact on accuracy and calibration. Nevertheless, the EBM follows its true posterior closer than a tree or mean-field recognition model (E4.1).
- **A Gibbs sweep budget of $S_q \geq 4$ is sufficient:** On these tasks, a model with an EBM recognition model requires at least 4 Gibbs sweeps per input and training step to be indistinguishable from exact enumeration in posterior fidelity, accuracy and calibration (E4.2).

6.5 E5: FLOPs Model and Scalability

The third research objective of this project is to assess the scalability of the architecture (see Chapter 2). Since this project has no access to a physical TSU, the computational cost of the architecture is analysed through a floating-point operations (FLOPs) model per training epoch, splitting it along the discussed handoff of Chapter 4.6 into the part that stays digital and the part the TSU can compute. The question of this experiment is therefore how the arithmetic of one epoch grows with K and how much of it is sampling.

6.5.1 Setup

The FLOPs model is derived from two separate approaches. The first model built is based on a training algorithm analysis, deriving the key driving parameters and their order term step-by-step. The mixture model is trained on the full batch and one epoch is a single closed-form expression of N , D , C , K , r , T and S . When using an EBM as the recognition model, two more variables are added to this list, which are the number of moment chains n_c and the number of Gibbs sweeps per input S_q . The training algorithm is divided into 16 blocks of different computations, which are grouped into five groups: sampling, moments/KL, estimator, input-dimensions and other. The group which the TSU can absorb is the sampling group. A few underlying definitions for counting the FLOPs are made, for example, that matrix products are counted as two operations and that backward passes do not simply count as a factor two of the forward pass, but are instead derived per block. Transcendental functions, such as a sigmoid, exponential or logarithm, are charged with a calibrated constant τ per Gibbs site and τ_s per estimator sample. On the used GPU, measurements later assign these constants the values $\tau = 104$ and $\tau_s = 25$. The full list of equations of the FLOPs per training epoch is provided for each block and group in Appendix A.25.

The second model is based on a measurement by capturing the compiled training step and reading the operation count of its graph from the XLA compiler on the GPU without executing it. This is done over a small sweep of $K = 4$ to 64 for a model with the tree recognition model and $K = 4$ to 48 for a model with the EBM recognition model. When comparing the measured results with the modelled numbers, they agree within $\kappa = 1.00$ to 1.15 for the tree and 0.97 to 0.98 for the EBM, where κ is the ratio of measured to modelled FLOPs. All measurements are based on a base configuration of $N = 512$, $D = 32$, $C = 3$, $r = 4$, $T = 8$ and $S = 12$, with K swept.

It is important to note that both models only capture the FLOPs per training epoch of the mixture model. The training of the frozen trunk and the prediction at test time are not evaluated here.

6.5.2 Results

In Figure 6.17, the two charts show the absolute number of FLOPs for a mixture model with a tree recognition model (left chart) and a mixture model with an EBM as the recognition model (right). The bars reflect the results of the analytical model, while the upper black line represents the measured total FLOPs. In the vertical bars, each colour defines one of the groups with the relative share of the total FLOPs for this configuration stated. The dotted black line is the offloadable share as measured by the GPU, which mostly sits very accurately on top of each modelled sampling group (red).

Growth in K

Every group is linear in N and at most quadratic in K , and the estimator and input-dimension groups are fixed by the task through C , r and D , so the cost of an epoch grows polynomially along every axis. One exception is the exact moment table computation of the tree, whose share rises from 3 % at $K = 8$ to 50 % at $K = 64$. Its complexity term is written arithmetically as $2NK^2$ operations because the entries for a gate are derived from the entries of its parent gate, so each of the K gates adds one pass over an $N \times K$ array. However, the XLA compiler's standard optimisation does not store the intermediary arrays because memory traffic is expensive and the extra arithmetic operation cheap. Therefore, it recomputes the parent's

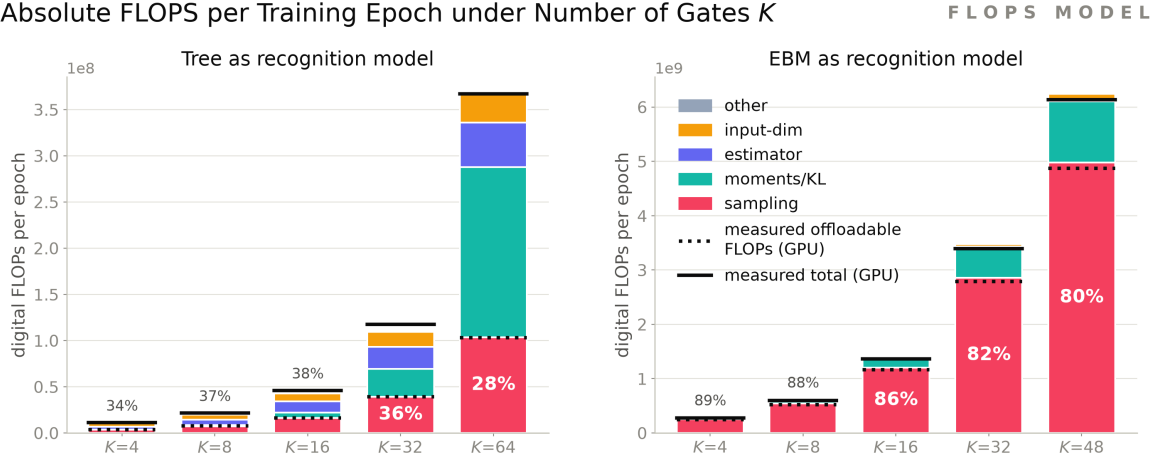
Absolute FLOPs per Training Epoch under Number of Gates K 

Figure 6.17: Digital FLOPs per training epoch by group over K for the tree (left) and the EBM recognition model (right)

entries wherever they are used and the grandparent's entries inside those, so that every one of the NK^2 table entries re-derives its full chain of up to K ancestors. Consequently, the compiled cost is $0.95NK^3$, which is shown in this chart. This is a property of the compiled graph and not of the algorithm.

Offloadable share

A model using a tree recognition model can only offload the Gibbs sampling steps during training in the prior (see Chapter 4.6). When using an EBM as the recognition model, on the other hand, the Gibbs sampling in the recognition model can be offloaded to the TSU as well. Consequently, the share of the sampling (red) is significantly higher in the runs for the model with the EBM recognition model than in the runs with the tree recognition model. For the tree, the offloadable share per epoch is between 28 and 36 % over $K = 4$ to 64 (measured), while for the EBM it is between 79 and 89 %. It shows that a large share of the computation can be offloaded from the GPU to the TSU.

Implication

Since this project is based on simulations and not on a physical chip, it cannot be shown that this offload to the TSU results in an overall gain in efficiency or wall-clock time. But the results of Jelinčič et al. [21] and Jelinčič and Walker [66], as presented in Chapter 3, indicate that a TSU performs Gibbs sampling at orders of magnitude lower energy consumption than a GPU running the digital algorithm.

In summary, these results answer the third research objective on the evidence available: The architecture scales polynomially and with an EBM recognition model, most of its cost is sampling, which can be offloaded to the TSU. This makes the developed architecture a promising candidate for an efficient and scalable hybrid approach. Whether the thermodynamic hardware can perform that sampling more efficiently on a physical device in practice is therefore a very interesting research question for future work.

Chapter 7

Limitations

While the results are promising in general, they rest on certain design choices and simulation assumptions that are critically reflected in this chapter.

First, Extropic’s TSU is a sparse, locally connected and bipartite grid with at most twelve connections per node and its efficiency rests on block updates [21]. The gate prior and the EBM recognition model are fully-coupled Boltzmann machines, whose graph is not bipartite and therefore cannot be updated in blocks. Consequently, the energy consumption for the Gibbs update on the TSU is likely to increase substantially when using a fully-connected graph and it is a separate interesting research question whether an energy advantage can be observed in that case.

Moreover, Extropic’s hardware is designed to hold global couplings and input-dependent biases. While this holds for the gate prior of the architecture, the linear recognition head of the EBM recognition model provides input-dependent biases and couplings. Consequently, not only the biases but also the programmable conductances between nodes would have to be rewritten for every input. The asymmetry was implemented on purpose to give the recognition model the full per-input freedom, but is a design choice which is not entirely transferable to the hardware.

As mentioned in E5, the scalability is assessed only through a GPU-side FLOPs model that counts the number of operations per training epoch. On a TSU, this arithmetic is not performed and the corresponding quantity is the number of site updates. The E5 experiment therefore shows the share of the GPU’s arithmetic that a TSU would absorb and not a measured gain. Only a measurement on a physical TSU can show the energy consumption and wall-clock time difference to give a reliable indication of the architecture’s energy and time advantages. Therefore, the results of E5 must be read carefully.

Another factor not represented in the simulations is physical fluctuation and noise. A physical p-bit has a non-ideal transfer function and a finite programming resolution for J and h_η , whereas the simulations used an exact sigmoid and double-precision couplings. The effects of these deviations on training were not quantified in this work.

Finally, the evaluation rests on two real datasets with three to ten seeds per configuration and experiment. A deeper analysis may sharpen certain findings presented in this project.

Chapter 8

Conclusion and Outlook

The research question of this project was to assess how discrete thermodynamic hardware can be used for scalable and efficient Bayesian uncertainty quantification. For this purpose, the EBM-gated adapter mixture model was developed, representing a novel hybrid architecture based on conventional and discrete thermodynamic hardware. It confines all randomness to a binary gate vector over low-rank adapters, which is drawn from an input-dependent Boltzmann machine prior, while a recognition model supplies the label-conditioned posterior from which the prior is learned during training. The architecture was evaluated along three axes: feasibility, competitiveness and scalability.

The mixture model reaches the Bayes ceiling on the blob task and solves the routing task by using its input-dependent routing capabilities, demonstrating that the learning mechanism works as designed. Additionally, the predictive uncertainty shows the structure the task construction expects. A further in-depth analysis of the recognition model families confirmed the theoretical hierarchy of the families in practice by showing that an EBM family follows its true enumerated posterior the closest, followed by the tree and then by mean-field. However, on every task studied throughout this project, the true gate posteriors are nearly factorised, so the choice of the recognition model has no significant impact on accuracy and the calibration metrics. In summary, the results show the successful training and prediction of the architecture and consequently demonstrate feasibility.

In-distribution, the mixture model proves competitiveness on real data by matching the accuracy and achieving calibration metrics at the same level as established UQ baselines on Fashion-MNIST and CIFAR-10. Under distribution shift, the picture is a bit more ambiguous as both datasets rank the mixture model and baselines differently. However, they move on similar levels under increased blend fraction, indicating competitiveness also under distribution shift. More in-depth sensitivity analyses showed that the accuracy the mixture model can reach highly depends on the used convolutional trunk, and that the mixture model only activates when the trunk and base map do not already solve the task. The mixture model adds calibration and quantified epistemic uncertainty, but not accuracy over the accuracy level of the convolutional trunk. Additionally, the scaling study showed that increasing the number of gates K and adapter rank r only positively impacts the accuracy and Brier score at very small configurations and that the results saturate very quickly on a plateau. Weakening the convolutional trunk changed the height of the plateaus, but only marginally moved the saturation boundary.

It is important to note that the architecture does not entirely comply with all characteristics of the TSU, such as the sparse and input-independent coupling structure, and that all results presented are based on simulations. Due to these limitations the project can only deliver a rough indication regarding the architecture’s scalability and the results must be read carefully. Under presented configurations, around 80 % of the GPU workload can be offloaded to the TSU when using an EBM as the variational family of the recognition model,

which is therefore justified rather by hardware fit than by model performance. Given the existing research findings regarding the sampling efficiency of Extropic’s TSUs, it is therefore reasonable to expect an overall efficiency gain by distributing the computational workload across the two hardware platforms. Consequently, these results give an indication that the architecture is scalable.

Taken together, the results demonstrate that the architecture learns to solve the given tasks successfully, provides calibration and quantified uncertainty at the same level as established UQ methods, and indicates scalable viability by offloading the stochastic sampling to the TSU. This project shows the potential of discrete thermodynamic hardware for Bayesian uncertainty quantification and contributes a scalable architecture proposal for future work in this research area.

For future work, a very interesting direction would be to introduce sparsity and global couplings in both the EBM prior and EBM recognition model, so they become sparse, bipartite Boltzmann machines, which Extropic’s TSUs can efficiently sample from. Since gate correlations were weak on every task, matching the TSU’s connectivity should cost little in expressiveness while restoring the block-parallel updates. Once the match between model and TSU is established, running the experiments on a physical unit would be the next logical step to directly assess the energy efficiency and wall-clock time of the architecture to overcome the limitations of a simulation. Moreover, the developed parameter optimisation pipeline offers the potential to scale up experiments to datasets of higher complexity to evaluate the influence of further hyperparameters and study tasks in which the gate posterior is genuinely correlated.

In conclusion, this work provides a solid foundation for further relevant research on a hybrid architecture using thermodynamic co-processors for Bayesian uncertainty quantification.

Bibliography

1. Begoli E, Bhattacharya T, and Kusnezov D. The Need for Uncertainty Quantification in Machine-Assisted Medical Decision Making. *Nature Machine Intelligence* 2019; 1:20–3. DOI: 10.1038/s42256-018-0004-1 (page 2)
2. Kolicic B, Caron A, Mavroudis V, and Hicks C. Inherently Interpretable and Uncertainty-Aware Models for Online Learning in Cyber-Security Problems. *2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)*. IEEE, 2024 :1–10. DOI: 10.1109/ACSACW65225.2024.00009 (page 2)
3. Guo C, Pleiss G, Sun Y, and Weinberger KQ. On Calibration of Modern Neural Networks. *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017 :1321–30 (pages 2, 4, 11)
4. Nguyen A, Yosinski J, and Clune J. Deep Neural Networks are Easily Fooled: High Confidence Predictions for Unrecognizable Images. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2015 :427–36. DOI: 10.1109/CVPR.2015.7298640 (pages 2, 4)
5. Ovadia Y, Fertig E, Ren J, Nado Z, Sculley D, Nowozin S, Dillon JV, Lakshminarayanan B, and Snoek J. Can You Trust Your Model’s Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift. *Advances in Neural Information Processing Systems*. Vol. 32. 2019 (pages 2, 4)
6. Kendall A and Gal Y. What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision? *Advances in Neural Information Processing Systems*. Vol. 30. 2017 (pages 2, 4)
7. Hüllermeier E and Waegeman W. Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine Learning* 2021; 110:457–506. DOI: 10.1007/s10994-021-05946-3 (pages 2, 4)
8. MacKay DJC. A Practical Bayesian Framework for Backpropagation Networks. *Neural Computation* 1992; 4:448–72. DOI: 10.1162/neco.1992.4.3.448 (pages 2, 5)
9. Wilson AG and Izmailov P. Bayesian Deep Learning and a Probabilistic Perspective of Generalization. *Advances in Neural Information Processing Systems*. Vol. 33. 2020 :4697–708 (pages 2, 5)
10. Izmailov P, Vikram S, Hoffman MD, and Wilson AG. What Are Bayesian Neural Network Posteriors Really Like? *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021 :4629–40 (pages 2, 6)
11. Papamarkou T, Hinkle J, Young MT, and Womble D. Challenges in Markov Chain Monte Carlo for Bayesian Neural Networks. *Statistical Science* 2022; 37:425–42. DOI: 10.1214/21-STS840 (pages 2, 6)
12. Blei DM, Kucukelbir A, and McAuliffe JD. Variational Inference: A Review for Statisticians. *Journal of the American Statistical Association* 2017; 112:859–77. DOI: 10.1080/01621459.2017.1285773 (pages 2, 6, 8)
13. Blundell C, Cornebise J, Kavukcuoglu K, and Wierstra D. Weight Uncertainty in Neural Networks. *Proceedings of the 32nd International Conference on Machine Learning*.

- Vol. 37. Proceedings of Machine Learning Research. PMLR, 2015 :1613–22 (pages 2, 5, 8)
14. Gal Y and Ghahramani Z. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. PMLR, 2016 :1050–9 (pages 2, 9)
 15. Conte T, DeBenedictis E, Ganesh N, Hylton T, Strachan JP, Williams RS, et al. Thermodynamic Computing. arXiv preprint arXiv:1911.01968 2019. arXiv: 1911 . 01968 [cs.ET] (pages 2, 14, 15)
 16. Coles PJ, Szczepanski C, Melanson D, Donatella K, Martinez AJ, and Sbahi F. Thermodynamic AI and the Fluctuation Frontier. arXiv preprint arXiv:2302.06584 2023 (pages 2, 14, 15)
 17. Aifer M, Duffield S, Donatella K, Melanson D, Klett P, Belateche Z, and Coles PJ. Thermodynamic Bayesian Inference. *2024 IEEE International Conference on Rebooting Computing (ICRC)*. IEEE, 2024 :1–20 (pages 2, 15)
 18. Melanson D, Khater MA, Aifer M, Donatella K, Gordon MH, Ahle T, and Coles PJ. Thermodynamic Computing System for AI Applications. *Nature Communications* 2025; 16:3757. DOI: 10.1038/s41467-025-58186-3 (pages 2, 14, 16)
 19. Faria R, Camsari KY, and Datta S. Implementing Bayesian Networks with Embedded Stochastic MRAM. *AIP Advances* 2018; 8:045101. DOI: 10.1063/1.5021332 (pages 2, 17)
 20. Liu S, Xiao TP, Kwon J, Debusschere BJ, Agarwal S, Incorvia JAC, and Bennett CH. Bayesian Neural Networks Using Magnetic Tunnel Junction-Based Probabilistic In-Memory Computing. *Frontiers in Nanotechnology* 2022; 4:1021943. DOI: 10.3389/fnano.2022.1021943 (pages 2, 17, 36)
 21. Jelinčič A, Lockwood O, Garlapati A, Schillinger P, Chuang I, Verdon G, and McCourt T. An Efficient Probabilistic Hardware Architecture for Diffusion-Like Models. arXiv preprint arXiv:2510.23972 2025 (pages 2, 17, 68, 69)
 22. Gawlikowski J, Tassi CRN, Ali M, Lee J, Humt M, Feng J, Zhu XX, et al. A survey of uncertainty in deep neural networks. *Artificial Intelligence Review* 2023; 56:1513–89 (pages 4, 5, 77)
 23. Hein M, Andriushchenko M, and Bitterwolf J. Why ReLU Networks Yield High-Confidence Predictions Far Away from the Training Data and How to Mitigate the Problem. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2019 :41–50. DOI: 10.1109/CVPR.2019.00013 (pages 4, 10)
 24. Gelman A, Carlin JB, Stern HS, and Rubin DB. Bayesian Data Analysis. Chapman and Hall/CRC, 1995 :3–30 (page 5)
 25. Houlisby N, Huszár F, Ghahramani Z, and Lengyel M. Bayesian Active Learning for Classification and Preference Learning. arXiv preprint arXiv:1112.5745 2011. arXiv: 1112.5745 [stat.ML] (page 6)
 26. Jordan MI, Ghahramani Z, Jaakkola TS, and Saul LK. An Introduction to Variational Methods for Graphical Models. *Machine Learning* 1999; 37:183–233. DOI: 10.1023/A:1007665907178 (page 6)
 27. Kingma DP and Welling M. Auto-Encoding Variational Bayes. arXiv preprint arXiv:1312.6114 2013. arXiv: 1312.6114 [stat.ML] (pages 7, 9)
 28. Saul LK and Jordan MI. Exploiting Tractable Substructures in Intractable Networks. *Advances in Neural Information Processing Systems*. Vol. 8. MIT Press, 1995 :486–92 (page 8)

29. Chow CK and Liu CN. Approximating Discrete Probability Distributions with Dependence Trees. *IEEE Transactions on Information Theory* 1968; 14:462–7. DOI: 10.1109/TIT.1968.1054142 (page 8)
30. Srivastava N, Hinton G, Krizhevsky A, Sutskever I, and Salakhutdinov R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* 2014; 15:1929–58 (page 9)
31. Hron J, G. Matthews AG de, and Ghahramani Z. Variational Bayesian Dropout: Pitfalls and Fixes. *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. *Proceedings of Machine Learning Research*. PMLR, 2018 :2019–28 (page 9)
32. Folgoc LL, Baltatzis V, Desai S, Devaraj A, Ellis S, Manzanera OEM, Nair A, Qiu H, Schnabel J, and Glocker B. Is MC Dropout Bayesian? *arXiv preprint arXiv:2110.04286* 2021. *arXiv: 2110.04286* (page 9)
33. Daxberger E, Kristiadi A, Immer A, Eschenhagen R, Bauer M, and Hennig P. Laplace Redux – Effortless Bayesian Deep Learning. *Advances in Neural Information Processing Systems*. Vol. 34. 2021 :20089–103 (page 9)
34. Daxberger E, Nalisnick E, Allingham JU, Antorán J, and Hernández-Lobato JM. Bayesian Deep Learning via Subnetwork Inference. *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. *Proceedings of Machine Learning Research*. PMLR, 2021 :2510–21 (page 9)
35. Sharma M, Farquhar S, Nalisnick E, and Rainforth T. Do Bayesian Neural Networks Need To Be Fully Stochastic? *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*. Vol. 206. *Proceedings of Machine Learning Research*. PMLR, 2023 :7694–722 (page 10)
36. Kristiadi A, Hein M, and Hennig P. Being Bayesian, Even Just a Bit, Fixes Overconfidence in ReLU Networks. *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. *Proceedings of Machine Learning Research*. PMLR, 2020 :5436–46 (page 10)
37. Williams RJ. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning* 1992; 8:229–56. DOI: 10.1007/BF00992696 (page 10)
38. Mohamed S, Rosca M, Figurnov M, and Mnih A. Monte Carlo Gradient Estimation in Machine Learning. *Journal of Machine Learning Research* 2020; 21:1–62 (page 10)
39. Mnih A and Rezende DJ. Variational Inference for Monte Carlo Objectives. *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. *Proceedings of Machine Learning Research*. PMLR, 2016 :2188–96 (page 10)
40. Kool W, Hoof H van, and Welling M. Buy 4 REINFORCE Samples, Get a Baseline for Free! *Deep Reinforcement Learning Meets Structured Prediction Workshop at ICLR*. 2019 (page 10)
41. Naeini MP, Cooper GF, and Hauskrecht M. Obtaining Well Calibrated Probabilities Using Bayesian Binning. *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*. Vol. 29. 1. AAAI Press, 2015 :2901–7. DOI: 10.1609/aaai.v29i1.9602 (page 11)
42. Gneiting T and Raftery AE. Strictly Proper Scoring Rules, Prediction, and Estimation. *Journal of the American Statistical Association* 2007; 102:359–78. DOI: 10.1198/016214506000001437 (page 11)
43. Brier GW. Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review* 1950; 78:1–3. DOI: 10.1175/1520-0493(1950)078<0001:V0FEIT>2.0.CO;2 (page 11)

44. LeCun Y, Chopra S, Hadsell R, Ranzato M, and Huang FJ. A Tutorial on Energy-Based Learning. *Predicting Structured Data*. Ed. by Taylor G, Hinton G, and Roweis S. MIT Press, 2006 (page 12)
45. Song Y and Kingma DP. How to Train Your Energy-Based Models. arXiv preprint arXiv:2101.03288 2021. arXiv: 2101.03288 [cs.LG] (pages 12, 13)
46. Ackley DH, Hinton GE, and Sejnowski TJ. A Learning Algorithm for Boltzmann Machines. *Cognitive Science* 1985; 9:147–69 (page 13)
47. Camsari KY, Faria R, Sutton BM, and Datta S. Stochastic p-Bits for Invertible Logic. *Physical Review X* 2017; 7:031014. DOI: 10.1103/PhysRevX.7.031014 (pages 14, 16)
48. Niazi S, Chowdhury S, Aadit NA, Mohseni M, Qin Y, and Camsari KY. Training Deep Boltzmann Networks with Sparse Ising Machines. *Nature Electronics* 2024; 7:610–9. DOI: 10.1038/s41928-024-01182-4 (pages 14, 17)
49. Geman S and Geman D. Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 1984; PAMI-6:721–41. DOI: 10.1109/TPAMI.1984.4767596 (page 14)
50. Smolensky P. Information Processing in Dynamical Systems: Foundations of Harmony Theory. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Volume 1: Foundations*. Ed. by Rumelhart DE, McClelland JL, and PDP Research Group the. Cambridge, MA: MIT Press, 1986 :194–281 (page 14)
51. Levin DA and Peres Y. Markov Chains and Mixing Times. 2nd ed. Providence, RI: American Mathematical Society, 2026. DOI: 10.1090/mbk/107 (page 14)
52. Hinton GE. Training Products of Experts by Minimizing Contrastive Divergence. *Neural Computation* 2002; 14:1771–800. DOI: 10.1162/089976602760128018 (page 14)
53. Hinton GE. A Practical Guide to Training Restricted Boltzmann Machines. *Neural Networks: Tricks of the Trade: Second Edition*. Ed. by Montavon G, Orr GB, and Müller KR. Vol. 7700. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2012 :599–619. DOI: 10.1007/978-3-642-35289-8_32 (page 14)
54. Carreira-Perpiñán MÁ and Hinton GE. On Contrastive Divergence Learning. *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics*. Vol. R5. Proceedings of Machine Learning Research. PMLR, 2005 :33–40 (page 14)
55. Poppelbaum WJ, Afuso C, and Esch JW. Stochastic Computing Elements and Systems. *Proceedings of the November 14–16, 1967, Fall Joint Computer Conference*. Vol. 31. ACM, 1967 :635–44. DOI: 10.1145/1465611.1465696 (page 14)
56. Gaines BR. Origins of Stochastic Computing. *Stochastic Computing: Techniques and Applications*. Ed. by Gross WJ and Gaudet VC. Cham: Springer, 2019 :13–37. DOI: 10.1007/978-3-030-03730-7_2 (page 14)
57. Gaines BR. Stochastic Computing. *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*. Vol. 30. ACM, 1967 :149–56. DOI: 10.1145/1465482.1465505 (page 14)
58. Aifer M, Donatella K, Gordon MH, Duffield S, Ahle T, Simpson D, Coles PJ, et al. Thermodynamic Linear Algebra. *npj Unconventional Computing* 2024; 1:13. DOI: 10.1038/s44335-024-00013-8 (page 15)
59. Normal Computing. Normal Computing Announces Tape-Out of World’s First Thermodynamic Computing Chip. Company announcement, 8 December 2025. 2025 Dec. Available from: normalcomputing.com/blog/ [Accessed on: 2026 Jun 1] (page 16)
60. Donatella K, Duffield S, Aifer M, Melanson D, Crooks GE, and Coles PJ. Thermodynamic Natural Gradient Descent. *npj Unconventional Computing* 2026; 3:5. DOI: 10.1038/s44335-026-00005-x (page 16)

61. Whitelam S. Generative Thermodynamic Computing. *Physical Review Letters* 2026; 136:037101. DOI: 10.1103/PhysRevLett.136.037101 (page 16)
62. Borders WA, Pervaiz AZ, Fukami S, Camsari KY, Ohno H, and Datta S. Integer Factorization Using Stochastic Magnetic Tunnel Junctions. *Nature* 2019; 573:390–3. DOI: 10.1038/s41586-019-1557-9 (page 16)
63. Aadit NA, Grimaldi A, Carpentieri M, Theogarajan L, Martinis JM, Finocchio G, and Camsari KY. Massively Parallel Probabilistic Computing with Sparse Ising Machines. *Nature Electronics* 2022; 5:460–8. DOI: 10.1038/s41928-022-00774-2 (page 16)
64. Singh NS, Kobayashi K, Cao Q, Selcuk K, Hu T, Niazi S, Aadit NA, Kanai S, Ohno H, Fukami S, and Camsari KY. CMOS Plus Stochastic Nanomagnets Enabling Heterogeneous Computers for Probabilistic Inference and Learning. *Nature Communications* 2024; 15:2685. DOI: 10.1038/s41467-024-46645-6 (page 17)
65. Extropic Corp. Extropic. <https://extropic.ai/>. Accessed: 2026-09-09 (page 17)
66. Jelincic A and Walker RC. Energy-Efficient Codon Optimization on Thermodynamic Hardware. arXiv preprint arXiv:2606.17327 2026. Extropic Corp. arXiv: 2606.17327 (pages 18, 68)
67. Extropic. THRML: Thermodynamic Hypergraphical Model Library. Version 0.1.3. Software library for thermodynamic hypergraphical models, probabilistic graphical models, and block Gibbs sampling. 2025. Available from: <https://github.com/extropic-ai/thrml> (page 18)
68. Xiao H, Rasul K, and Vollgraf R. Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv preprint arXiv:1708.07747 2017. arXiv: 1708.07747 [cs.LG] (page 36)
69. Krizhevsky A and Hinton G. Learning Multiple Layers of Features from Tiny Images. Tech. rep. Toronto, Ontario: University of Toronto, 2009 (page 36)
70. Cohen G, Afshar S, Tapson J, and Schaik A van. EMNIST: Extending MNIST to Handwritten Letters. *2017 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2017 :2921–6. DOI: 10.1109/IJCNN.2017.7966217 (page 36)
71. Netzer Y, Wang T, Coates A, Bissacco A, Wu B, and Ng AY. Reading Digits in Natural Images with Unsupervised Feature Learning. *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*. 2011 (page 36)

Chapter A

Appendix

A.1 Categorisation of Uncertainty Quantification Methods

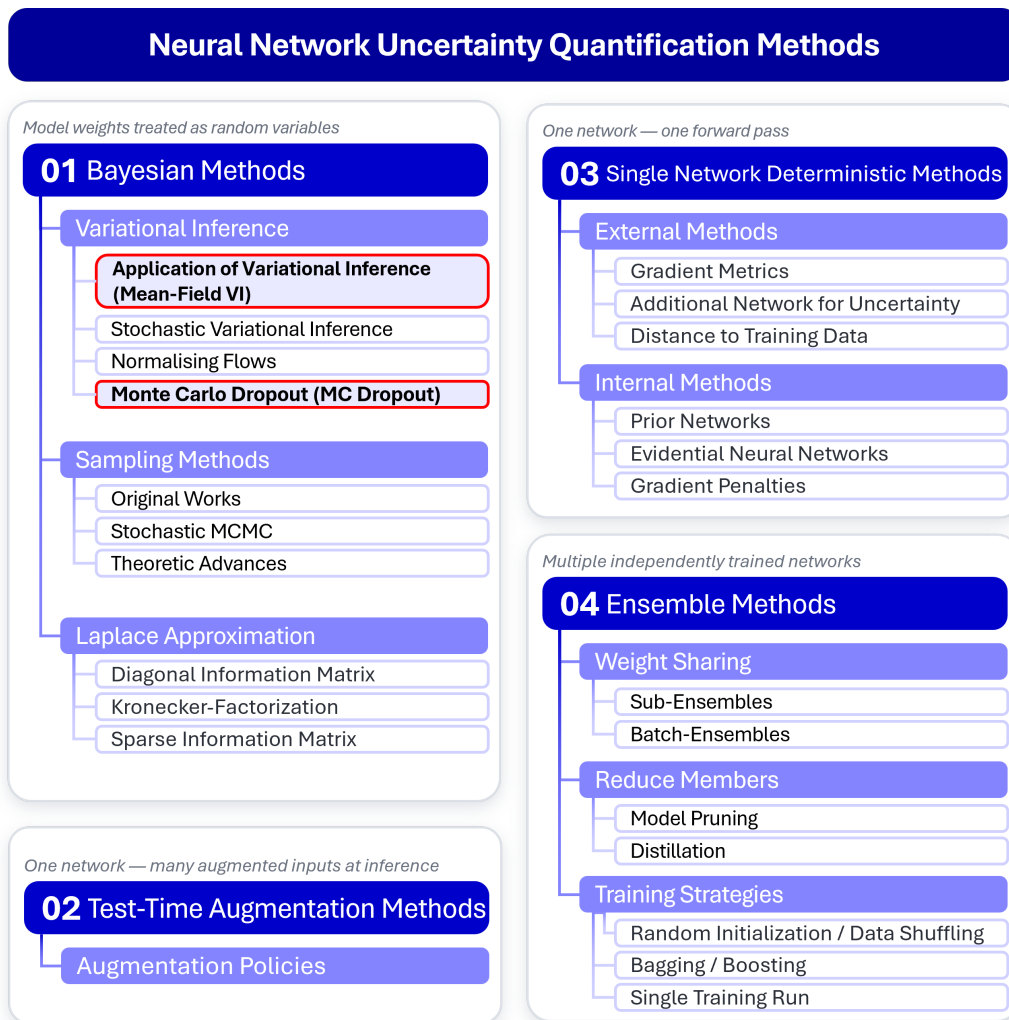


Figure A.1: A typical categorisation of uncertainty quantification methods for neural networks, organised after the taxonomy of Gawlikowski et al. [22]. The four groups differ in where the stochasticity that produces the prediction spread of Equation (3.4) is located: in a distribution over the weights, in the inputs at test time, inside a single deterministic forward pass, or across independently trained networks. Outlined in red are the methods reviewed in this section (made by author, reproduced from [22])

A.2 Derivation of the Evidence Lower Bound

Derivation of Equation (3.6) and Equation (3.7), assuming $q_\theta \ll p(\mathbf{w} \mid \mathcal{D})$ so that both divergences are finite.

$$\begin{aligned}
 \log p(\mathcal{D}) &= \mathbb{E}_{q_\theta}[\log p(\mathcal{D})] = \mathbb{E}_{q_\theta} \left[\log \frac{p(\mathcal{D}, \mathbf{w})}{p(\mathbf{w} \mid \mathcal{D})} \right] \\
 &= \mathbb{E}_{q_\theta} \left[\log \left(\frac{p(\mathcal{D}, \mathbf{w})}{q_\theta(\mathbf{w})} \cdot \frac{q_\theta(\mathbf{w})}{p(\mathbf{w} \mid \mathcal{D})} \right) \right] \\
 &= \underbrace{\mathbb{E}_{q_\theta} \left[\log \frac{p(\mathcal{D}, \mathbf{w})}{q_\theta(\mathbf{w})} \right]}_{\mathcal{L}(\theta)} + \underbrace{\mathbb{E}_{q_\theta} \left[\log \frac{q_\theta(\mathbf{w})}{p(\mathbf{w} \mid \mathcal{D})} \right]}_{\text{KL}(q_\theta(\mathbf{w}) \parallel p(\mathbf{w} \mid \mathcal{D}))} \quad (9.1)
 \end{aligned}$$

$$\begin{aligned}
 \mathcal{L}(\theta) &= \mathbb{E}_{q_\theta}[\log p(\mathcal{D} \mid \mathbf{w}) + \log p(\mathbf{w}) - \log q_\theta(\mathbf{w})] \\
 &= \mathbb{E}_{q_\theta}[\log p(\mathcal{D} \mid \mathbf{w})] - \text{KL}(q_\theta(\mathbf{w}) \parallel p(\mathbf{w})) \quad (9.2)
 \end{aligned}$$

A.3 Derivations of the ELBO Gradient

Derivation of Equation (3.10).

$$\begin{aligned}
 \nabla_\theta \mathbb{E}_{q_\theta(z)}[f_\theta(z)] &= \nabla_\theta \sum_z q_\theta(z) f_\theta(z) \\
 &= \sum_z f_\theta(z) \nabla_\theta q_\theta(z) + \sum_z q_\theta(z) \nabla_\theta f_\theta(z) \\
 &= \underbrace{\sum_z f_\theta(z) \nabla_\theta q_\theta(z)}_{\text{distributional term}} + \mathbb{E}_{q_\theta(z)}[\nabla_\theta f_\theta(z)] \quad (9.3)
 \end{aligned}$$

A.4 Derivations of the Score Function Estimator

Derivation of Equation (3.13) and Equation (3.14). Throughout, $q_\theta(z) > 0$ for every z , and the sums are written for a discrete z and become integrals otherwise.

Log-derivative identity

$$\nabla_\theta q_\theta(z) = q_\theta(z) \frac{\nabla_\theta q_\theta(z)}{q_\theta(z)} = q_\theta(z) \nabla_\theta \log q_\theta(z) \quad (9.4)$$

Substitution into the distributional term

$$\sum_z f_\theta(z) \nabla_\theta q_\theta(z) = \sum_z q_\theta(z) f_\theta(z) \nabla_\theta \log q_\theta(z)$$

$$= \mathbb{E}_{q_{\theta}(z)}[f_{\theta}(z) \nabla_{\theta} \log q_{\theta}(z)] \quad (9.5)$$

Substituting Equation (9.5) into Equation (9.3) gives Equation (3.13).

Zero mean of the score

$$\begin{aligned} \mathbb{E}_{q_{\theta}}[\nabla_{\theta} \log q_{\theta}(z)] &= \sum_z q_{\theta}(z) \nabla_{\theta} \log q_{\theta}(z) = \sum_z \nabla_{\theta} q_{\theta}(z) \\ &= \nabla_{\theta} \sum_z q_{\theta}(z) = \nabla_{\theta} 1 = 0 \end{aligned} \quad (9.6)$$

Admissibility of the baseline

$$\mathbb{E}_{q_{\theta}}[(f_{\theta}(z) - b) \nabla_{\theta} \log q_{\theta}(z)] = \mathbb{E}_{q_{\theta}}[f_{\theta}(z) \nabla_{\theta} \log q_{\theta}(z)] - b \underbrace{\mathbb{E}_{q_{\theta}}[\nabla_{\theta} \log q_{\theta}(z)]}_{=0}$$

A.5 Derivation of the Contrastive Gradient

Derivation of Equation (3.23).

$$\mathbb{E}_{\text{data}}[-\log p_{\theta}(\mathbf{x})] = \mathbb{E}_{\text{data}}[E_{\theta}(\mathbf{x})] + \log Z_{\theta} \quad (9.7)$$

$$\begin{aligned} \nabla_{\theta} \log Z_{\theta} &= \frac{1}{Z_{\theta}} \nabla_{\theta} \sum_{\mathbf{x}} \exp(-E_{\theta}(\mathbf{x})) \\ &= - \sum_{\mathbf{x}} \underbrace{\frac{\exp(-E_{\theta}(\mathbf{x}))}{Z_{\theta}}}_{p_{\theta}(\mathbf{x})} \nabla_{\theta} E_{\theta}(\mathbf{x}) \\ &= - \mathbb{E}_{p_{\theta}}[\nabla_{\theta} E_{\theta}(\mathbf{x})] \end{aligned} \quad (9.8)$$

A.6 Derivation of the Gibbs Conditional

Derivation of Equation (3.25).

$$E(\mathbf{s}) = -\frac{1}{2} \mathbf{s}^{\top} J \mathbf{s} - \mathbf{h}^{\top} \mathbf{s}, \quad J = J^{\top}, \quad \text{diag}(J) = 0.$$

With $-\frac{1}{2}(\sum_j J_{ij} s_i s_j + \sum_j J_{ji} s_j s_i) - h_i s_i$ or $-s_i(\sum_{j \neq i} J_{ij} s_j + h_i)$:

$$E(\mathbf{s}) = -s_i a_i + R(\mathbf{s}_{-i}), \quad a_i = \sum_{j \neq i} J_{ij} s_j + h_i. \quad (9.9)$$

$$\frac{p(\mathbf{x})}{p(\mathbf{x}')} = \exp(E(\mathbf{x}') - E(\mathbf{x})),$$

$$p(s_i = 1 | \mathbf{s}_{-i}) = \frac{\exp(a_i)}{1 + \exp(a_i)} = \frac{1}{1 + \exp(-a_i)} = \sigma(a_i),$$

A.7 Architectural Difference during Training and Prediction

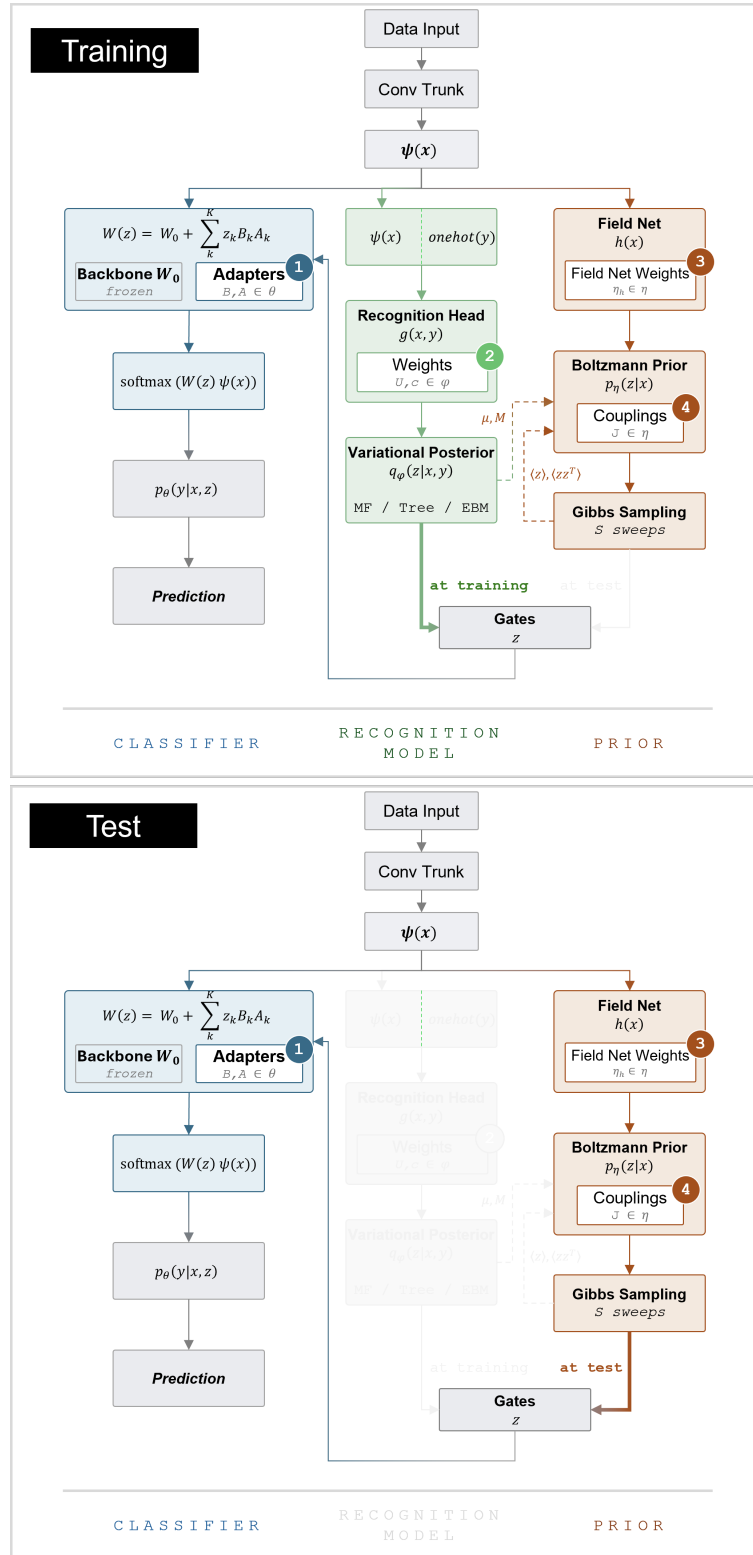


Figure A.2: The structural difference in the architecture between training and test.

A.8 Derivation of the Gated Evidence Lower Bound

Derivation of Equations (4.14) and (4.15), writing $q \equiv q_\phi(z | x, y)$ and $p(y, z | x) = p_\theta(y | x, z) p_\eta(z | x)$ as in Equation (4.9).

$$\begin{aligned}
 \log p(y | x) &= \log \sum_z p(y, z | x) \\
 &= \log \sum_z q \frac{p(y, z | x)}{q} \\
 &= \log \mathbb{E}_q \left[\frac{p(y, z | x)}{q} \right] \\
 &\geq \mathbb{E}_q \left[\log \frac{p(y, z | x)}{q} \right] =: \mathcal{L}.
 \end{aligned} \tag{9.10}$$

$$\begin{aligned}
 \mathcal{L} &= \mathbb{E}_q [\log p_\theta(y | x, z)] + \mathbb{E}_q [\log p_\eta(z | x) - \log q] \\
 &= \mathbb{E}_q [\log p_\theta(y | x, z)] - \text{KL}(q \| p_\eta(\cdot | x)).
 \end{aligned} \tag{9.11}$$

$$\begin{aligned}
 \mathcal{L} &= \mathbb{E}_q [\log p(y, z | x) - \log q] \\
 &= \mathbb{E}_q [\log p(z | x, y) + \log p(y | x) - \log q] \\
 &= \log p(y | x) - \text{KL}(q \| p(z | x, y)).
 \end{aligned} \tag{9.12}$$

A.9 Derivation of the Closed-Form Gate KL

Derivation of Equation (4.18), writing $q \equiv q_\phi(z | x, y)$ and using the moments μ , M of Equation (4.17).

$$\begin{aligned}
 \text{KL}(q \| p_\eta(\cdot | x)) &= \mathbb{E}_q [\log q] - \mathbb{E}_q [\log p_\eta(z | x)] \\
 &= -H(q) + \mathbb{E}_q [E_\eta(z, x)] + \log Z_\eta(x).
 \end{aligned} \tag{9.13}$$

$$\mathbb{E}_q [\mathbf{h}_\eta(x)^\top \mathbf{z}] = \mathbf{h}_\eta(x)^\top \mu, \tag{9.14}$$

$$\mathbb{E}_q [\mathbf{z}^\top \mathbf{J} \mathbf{z}] = \sum_{k,j} J_{kj} \mathbb{E}_q [z_k z_j] = \sum_{k \neq j} J_{kj} M_{kj} = \text{tr}(\mathbf{J} \mathbf{M}). \tag{9.15}$$

$$\mathbb{E}_q [E_\eta(z, x)] = -\mathbf{h}_\eta(x)^\top \mu - \frac{1}{2} \text{tr}(\mathbf{J} \mathbf{M}). \tag{9.16}$$

$$\text{KL}(q \| p_\eta(\cdot | x)) = -H(q) - \mathbf{h}_\eta(x)^\top \mu - \frac{1}{2} \text{tr}(\mathbf{J} \mathbf{M}) + \log Z_\eta(x). \tag{9.17}$$

A.10 Gradients of the Energy-Based Recognition Model

Derivation of the quantities the energy-based recognition family of Chapter 4.4.2 estimates from its chains. Throughout, ∇_ϕ denotes the gradient with respect to the parameters $(\mathbf{f}_\phi, \mathbf{J}_q)$ that the recognition head outputs. The remaining step to U and c is an ordinary backward pass through Equation (4.8).

Energy and sufficient statistics

$$q_\phi(\mathbf{z} \mid \mathbf{x}, y) = \frac{\exp(-E_\phi(\mathbf{z}, \mathbf{x}, y))}{Z_\phi(\mathbf{x}, y)}, \quad (9.18)$$

$$E_\phi(\mathbf{z}, \mathbf{x}, y) = -\mathbf{f}_\phi(\mathbf{x}, y)^\top \mathbf{z} - \frac{1}{2} \mathbf{z}^\top \mathbf{J}_q(\mathbf{x}, y) \mathbf{z},$$

$$\mathbf{s}(\mathbf{z}) := -\nabla_\phi E_\phi(\mathbf{z}, \mathbf{x}, y) = (\mathbf{z}, (z_k z_j)_{k < j}) \in \mathbb{R}^d, \quad d = K + \frac{1}{2}K(K-1). \quad (9.19)$$

Entropy

$$H(q_\phi) = \log Z_\phi(\mathbf{x}, y) + \mathbb{E}_{q_\phi}[E_\phi(\mathbf{z}, \mathbf{x}, y)], \quad (9.20)$$

where the expected energy is an average over the chains and $\log Z_\phi(\mathbf{x}, y)$ is the unknown offset.

Gradient of the log-normaliser

$$\begin{aligned} \nabla_\phi \log Z_\phi(\mathbf{x}, y) &= \frac{1}{Z_\phi(\mathbf{x}, y)} \nabla_\phi \sum_{\mathbf{z}} \exp(-E_\phi(\mathbf{z}, \mathbf{x}, y)) \\ &= -\sum_{\mathbf{z}} q_\phi(\mathbf{z} \mid \mathbf{x}, y) \nabla_\phi E_\phi(\mathbf{z}, \mathbf{x}, y) = \mathbb{E}_{q_\phi}[\mathbf{s}(\mathbf{z})]. \end{aligned} \quad (9.21)$$

Score

$$\nabla_\phi \log q_\phi(\mathbf{z} \mid \mathbf{x}, y) = \mathbf{s}(\mathbf{z}) - \mathbb{E}_{q_\phi}[\mathbf{s}(\mathbf{z})]. \quad (9.22)$$

Covariance identity for the gradient of an expectation

$$\begin{aligned} \nabla_\phi \mathbb{E}_{q_\phi}[g(\mathbf{z})] &= \sum_{\mathbf{z}} g(\mathbf{z}) q_\phi(\mathbf{z}) \nabla_\phi \log q_\phi(\mathbf{z}) \\ &= \mathbb{E}_{q_\phi}[g(\mathbf{z}) \mathbf{s}(\mathbf{z})] - \mathbb{E}_{q_\phi}[g(\mathbf{z})] \mathbb{E}_{q_\phi}[\mathbf{s}(\mathbf{z})] \\ &= \text{Cov}_{q_\phi}(g(\mathbf{z}), \mathbf{s}(\mathbf{z})). \end{aligned} \quad (9.23)$$

Gradient of the entropy

$$\nabla_{\phi} \mathbb{E}_{q_{\phi}}[E_{\phi}] = \underbrace{\mathbb{E}_{q_{\phi}}[\nabla_{\phi} E_{\phi}]}_{= -\mathbb{E}_{q_{\phi}}[s]} + \underbrace{\text{Cov}_{q_{\phi}}(E_{\phi}, s(z))}_{\text{Equation (9.23)}}, \quad (9.24)$$

$$\nabla_{\phi} H(q_{\phi}) = \underbrace{\mathbb{E}_{q_{\phi}}[s]}_{\nabla_{\phi} \log Z_{\phi}} - \underbrace{\mathbb{E}_{q_{\phi}}[s]}_{\text{expected energy}} + \text{Cov}_{q_{\phi}}(E_{\phi}, s(z)) = \text{Cov}_{q_{\phi}}(E_{\phi}, s(z)), \quad (9.25)$$

so the unknown offset of Equation (9.20) cancels and what remains is a difference of averages over the chains.

Leave-one-out estimator and the score

With $\ell_t = \log p_{\theta}(y | \mathbf{x}, \mathbf{z}^{(t)})$ and $G = \sum_t \ell_t$, the leave-one-out advantages sum to zero,

$$\sum_{t=1}^T A_t = \sum_{t=1}^T \left(\ell_t - \frac{G - \ell_t}{T - 1} \right) = G - \frac{T G - G}{T - 1} = 0, \quad (9.26)$$

$$\hat{\nabla}_{\phi} \mathbb{E}_{q_{\phi}}[g(\mathbf{z})] = \frac{1}{T} \sum_{t=1}^T A_t s(\mathbf{z}^{(t)}), \quad A_t = g(\mathbf{z}^{(t)}) - \frac{1}{T - 1} \sum_{u \neq t} g(\mathbf{z}^{(u)}). \quad (9.27)$$

The term $\mathbb{E}_{q_{\phi}}[s(\mathbf{z})]$ of Equation (9.22) is identical for every draw, so by Equation (9.26) it contributes nothing to Equation (9.27). The score is therefore taken from the energy alone,

$$s(\mathbf{z}) = -\nabla_{\phi} E_{\phi}(\mathbf{z}, \mathbf{x}, y) = (\mathbf{z}, (z_k z_j)_{k < j}), \quad (9.28)$$

and $Z_{\phi}(\mathbf{x}, y)$ is never computed.

A.11 Training Algorithm in Pseudocode

Algorithm 1 Conceptual training algorithm of the EBM-gated adapter mixture model

Input: features $\psi(\mathbf{x}_i)$ and labels $y_i, i = 1, \dots, N$

Input: epochs E , draws $T \geq 2$, sweeps S , step size λ , clip κ , decay ν

Initialisation

- 1: $W_0 \leftarrow$ base classifier $\triangleright$ frozen base matrix
- 2: $\theta \leftarrow \{A_k, B_k\}$ $\triangleright$ adapters
- 3: $U \leftarrow$ random, $\mathbf{c} \leftarrow \mathbf{0}$ $\triangleright$ recognition head weights
- 4: $\eta_h \leftarrow$ field network $\triangleright$ field network weights
- 5: $\mathbf{J} \leftarrow J_0 (\mathbf{1} - I)$ $\triangleright$ couplings
- 6: $\mathbf{z}_i^{\text{neg}} \sim \text{Bern}(\frac{1}{2})^K$ $\triangleright$ one persistent chain per point
- 7: $\beta_e \leftarrow$ annealing schedule $\triangleright$ zero until the warmup ends
- 8: $a_e \leftarrow \mathbf{1}[e \geq \text{warmup}]$ $\triangleright$ freezes head and prior

Training

- 9: **for** $e = 0, 1, \dots, E - 1$ **do** $\triangleright$ one epoch is one update

Positive phase

- 10: $\mathbf{u}_i \leftarrow U[\psi(\mathbf{x}_i), (y_i)] + \mathbf{c}$ $\triangleright$ parameters of q (Equation (4.8))
- 11: $(\mu_i, \mathbf{M}_i, \mathbf{H}_i) \leftarrow \text{FORWARD}(\mathbf{u}_i)$ $\triangleright$ closed-form marginals, moments, entropy
- 12: $\mathbf{h}_i \leftarrow \mathbf{h}_\eta(\psi(\mathbf{x}_i))$ $\triangleright$ prior's per-input gate fields (Equation (4.7))

Negative phase

- 13: $\mathbf{z}_i^{\text{neg}} \leftarrow \text{GIBBS}(\mathbf{h}_i, \mathbf{J}, \mathbf{z}_i^{\text{neg}}, S)$ $\triangleright S$ sweeps (Equation (3.25))

Reconstruction

- 14: $\mathbf{z}_i^{(t)} \sim q_\phi(\cdot | \mathbf{x}_i, y_i)$ $\triangleright T$ hard patterns from q_ϕ per input
- 15: $\ell_i^{(t)} \leftarrow \log p_\theta(y_i | \mathbf{x}_i, \mathbf{z}_i^{(t)})$ $\triangleright$ log-likelihood of true label
- 16: $s_i^{(t)} \leftarrow \log q_\phi(\mathbf{z}_i^{(t)} | \mathbf{x}_i, y_i)$ $\triangleright$ score of that draw
- 17: $b_i^{(t)} \leftarrow \frac{1}{T-1} \sum_{u \neq t} \ell_i^{(u)}$ $\triangleright$ leave-one-out baseline

Adapters and head

- 18: $\tilde{\mathcal{L}} \leftarrow \frac{1}{NT} \sum_{i,t} (\ell_i^{(t)} - b_i^{(t)}) s_i^{(t)}$ $\triangleright$ SFE route $\rightarrow \phi$
 $\quad + \frac{1}{NT} \sum_{i,t} \log p_\theta(y_i | \mathbf{x}_i, (\mathbf{z}_i^{(t)}))$ $\triangleright$ direct route $\rightarrow \theta$
 $\quad - \frac{\beta_e}{N} \sum_i \text{KL}_i^{\text{fb}}$ $\triangleright$ regulariser
- 19: $(g_\theta, g_\phi) \leftarrow \nabla_{\theta, \phi} \tilde{\mathcal{L}}$ $\triangleright$ gradients θ and ϕ (Equations (4.21) and (4.26))
- 20: $\theta \leftarrow \theta + \lambda(\text{clip}_\kappa(g_\theta) - \nu \theta)$ $\triangleright$ UPDATE adapters
- 21: $\phi \leftarrow \phi + a_e \lambda(\text{clip}_\kappa(g_\phi) - \nu \phi)$ $\triangleright$ UPDATE recognition head

Prior field

- 22: $\Delta \mathbf{h}_i \leftarrow \beta_e (\mu_i - \mathbf{z}_i^{\text{neg}})$ $\triangleright$ contrastive gradient
- 23: $g_{\eta_h} \leftarrow \nabla_{\eta_h} \frac{1}{N} \sum_i \mathbf{h}_\eta(\psi(\mathbf{x}_i))^\top (\Delta \mathbf{h}_i)$ $\triangleright$ gradients field network (Equation (4.25))
- 24: $\eta_h \leftarrow \eta_h + a_e \lambda(\text{clip}_\kappa(g_{\eta_h}) - \nu_h \eta_h)$ $\triangleright$ UPDATE field network

Prior couplings

- 25: $g_J \leftarrow \frac{\beta_e}{N} \sum_i (\mathbf{M}_i - \mathbf{z}_i^{\text{neg}} \mathbf{z}_i^{\text{neg}^\top})$ $\triangleright$ gradients couplings (Equation (4.23))
- 26: $g_J \leftarrow \frac{1}{2}(g_J + g_J^\top), \text{diag}(g_J) \leftarrow 0$ $\triangleright$ gradient constraints (Equation (4.6))
- 27: $\mathbf{J} \leftarrow \mathbf{J} + a_e \lambda \text{clip}_\kappa(g_J)$ $\triangleright$ UPDATE couplings

28: **end for**

- 29: **return** $\{W_0, \theta, \phi, \eta\}$ $\triangleright$ updated parameters
-

A.12 Full List of Training Stabilisation Configurations

Method	Symbol	Effect	Default
<i>Keeping the gates open</i>			
KL annealing	$\beta_e, \beta_{\max}$	raises the KL weight from zero to its final value over a fixed fraction of the run	✓
Free bits	ε	floors the per-gate marginal term of the KL, leaving the correlation part unfloored	✓
Gates-on field bias	$\mathbf{b}_2$	starts the field network's output bias positive	✓
Gates-on head bias	$\mathbf{c}$	starts the recognition head's output bias positive	
Gates-on warmup	a_e	freezes the recognition head and the prior for a fixed number of epochs	
<i>Keeping the adapters worth switching on</i>			
Adapter anchoring	ν_a	replaces the decay on θ by a spring towards the initial adapter geometry	
Warm-start noise	ζ	perturbs the initial adapters to break the symmetry between gates	
<i>Keeping the gates distinct</i>			
Coupling initialisation	J_0	starts the couplings uniformly inhibitory, favouring few-on gate patterns	✓
Gate-target warmup	γ_e, γ_0	adds a supervised target on the gate marginals that decays to zero	
Sparsity and load balance	$\rho_{\text{sp}}, \rho_{\text{lb}}$	penalise gate patterns that are not sparse or not evenly used across gates	
Reader diversity	ρ_{div}	penalises correlated readers A_k within a gate group	
<i>Keeping the optimisation stable</i>			
Gradient clipping	κ	rescales each parameter block whose gradient norm exceeds the threshold	✓
Weight decay	ν, ν_h	decays the adapters and the field network weights at separate rates	✓

Table A.1: Schedules, initialisation and stabilisation mechanisms of the training algorithm. A check mark denotes a mechanism active in every run if not turned off.

A.13 Full list of pipeline configuration options

Table A.2: Run configuration parameters (`run.config.xlsx`). Values are dropdown presets and only numeric knobs accept any typed value. A blank cell is the execution path's default.

Parameter	Values	Meaning
Meta		
<code>run_id</code>	(free text)	Run name; names runs/ <code><run_id></code> / and is the <code>results.xlsx</code> upsert key.
<code>enabled</code>	yes no	On/off switch: the runner executes enabled rows.
<code>seed</code>	0 1 2 3 42 0,1,2 0,1,2,3,4	Training seed; a comma list runs the row once per seed.
<code>notes</code>	(free text)	Free text, copied into <code>results.xlsx</code> .
Recognition (q)		
<code>estimator</code>	auto sfe-loo gumbel	Gradient estimator for the reconstruction term; must match across a tree-vs-mf pair.
<code>family</code>	tree mf tree_max_span	Recognition family $q(z x,y)$: fixed tree, mean-field, or learned Chow-Liu tree.
<code>node_order</code>	auto 0,1,2,3 0,2,1,3	Gate ordering of the fixed chain (family=tree only).
<code>tree_topology</code>	chain binary star random hier	Shape of the fixed tree; all shapes have $K-1$ edges (capacity-matched).
<code>tree_edge_weight</code>	abs_cov mi	Chow-Liu edge score: covariance or mutual information.
<code>tree_refit</code>	once periodic	Refit the learned topology once after warm-up, or at every segment.
<code>tree_struct_warmup</code>	0.3 0.5	Fraction of epochs before the first topology refit.
<code>tree_struct_warmup_family</code>	tree mf	q family during warm-up; 'mf' avoids echoing the chain back.
<code>T_samples</code>	4 8 16	Monte-Carlo gate samples per training step.
<code>tau</code>	0.5 1.0	Binary-concrete temperature (estimator=gumbel only).
<code>head_init_scale</code>	0.1 0.3	Init std of the recognition head U.
<code>c_bias</code>	0.0 0.5	Initial head bias; positive starts the gates ON.
Task		
<code>task</code>	xor blob multimodal routing hier_routing mnist cifar fashion cifar10_resnet	Which dataset / experiment the row runs.
<code>n_per</code>	100 200	Training samples per class (a total on routing tasks).
<code>n_test</code>	100 150 600	Test-set size (per class or total, task-dependent).
Base W0		
<code>w0_scale</code>	0.0 0.1 0.2 1.0	Std of the frozen random base W0; kept weak so the gates carry the work.
Adapters		
<code>K_gates</code>	2 3 4 6	Number of gated adapters K (practical cap ~12: 2^K enumerations).
<code>rank</code>	1 2 4	Rank of each adapter $B.k A.k$ (capacity per expert).
<code>gain</code>	4.0 5.0	Strength of the built-in class handle / group-anchor target.

continued on next page

Table A.2 (continued)

Parameter	Values	Meaning
adapter_noise	0.0 0.05	Warm-start noise that breaks the symmetry between gates.
adapter_init_scale	0.1	Init std of free random adapters.
Anchoring		
anchor_type	none frozen-exact per-gate-soft group-mean group-sum	Spring holding adapters near their init geometry; the main anti-collapse knob.
wd_anchor	0.0 0.1	Stiffness of that anchor spring.
EBM prior (J)		
j_init	0.0 -0.5 -1.5	Initial off-diagonal coupling J; negative = gates inhibit each other.
gibbs_sweeps_S	6 12 20	Gibbs sweeps per step for the negative phase.
Field h(x)		
field_kind	linear equivariant equivariant_nl routable_mlp routable_sum mlp	Network producing the per-input gate field h(x); *_mlp/routable allow per-gate routing.
field_n_hidden	8 16 24	Hidden width of the field network.
field_bias_b2	0.0 0.3 0.5	Constant field output bias (gates-on bootstrap).
field_v_scale	0.1	Init scale of the linear field weights V.
Readout / Baselines		
readout	count one-hot	XOR only: count = ordinal ladder (R3), one-hot = per-gate class boost (R5).
baselines	none dropout allon dropout+allon	Input-agnostic control models trained alongside the run.
dropout_keep	matched 0.5 0.6	Keep rate of the dropout control; 'matched' = model's own gate rate.
Recipe (anti-collapse)		
epochs	1200 1500 2000	Full-batch training steps (no minibatching).
lr	0.05	Learning rate (plain SGD).
beta_max	0.5 1.0	Final KL weight after annealing.
anneal_frac	0.4	Fraction of epochs over which beta ramps 0 → beta_max.
warmup	0	Epochs with prior (h, J) and head frozen.
free_bits	0.0 0.02	Per-gate KL floor; stops the KL from crushing informative gates.
wd	0.005 0.01	Weight decay on adapter/head/field weights (biases exempt).
clip	1.0	Global-norm gradient clip.
gamma0	0.0 1.0	Initial strength of the decaying q-target training wheel.
warm_frac	0.0 0.15	Fraction of epochs over which gamma decays to zero.
label_dropout_p	0.0 0.3 0.5	Fraction of samples whose label is hidden from q (x-only inference).
MoE (multimodal only)		
div	0.0 5.0	Reader-diversity weight: group experts read different input directions.

continued on next page

Table A.2 (continued)

Parameter	Values	Meaning
sparse	0.0 1.5	Sparsity pressure: one expert active per sample.
loadbal	0.0 1.5	Load-balancing pressure: experts used equally often.
Task: XOR		
xor_P	1 2	Number of (u,v) input pairs; $K = D = 2P$ on the count readout.
xor_small	0.5	Low anchor position of the ordinal ladder.
xor_big	2.0	High anchor position of the ordinal ladder.
Task: Blob		
blob_sigma	0.8 1.1 1.4	Std of the three Gaussian clouds (overlap = Bayes ceiling).
blob_R.C	2.5	Radius of the triangle the class centres sit on.
Task: Multimodal		
mm_R_class	3.0	Radius of the class centres.
mm_delta	1.3	Separation of the two sub-modes within a class.
mm_sigma	0.6	Std of each sub-mode.
Task: Routing		
routing_R	3	Number of conflicting rules; K defaults to R .
routing_DC	6	Width of the content block.
ctx	3.0 0.4	Context-block scale: large = clean routing, small = ambiguous.
routing_sigma_c	1.2	Std of the content vector.
Task: Hier-Routing		
hr_groups	2 3	Superior rules G ; $K = G + G^*M$ gates.
hr_rules_per_group	2 3	Sub-rules M per group.
hr_classes	3 6 10	Number of classes C (a count, unlike the class LISTS of mnist/cifar).
hr_ctx_super	3.0 0.4	Strength of the super-context block when present.
hr_ctx_sub	3.0 0.4	Strength of the sub-context block when present.
hr_ctx_sub_mask_p	0.0 0.3 0.5	$P(\text{drop sub-context})$: sibling ambiguity within a known group.
hr_ctx_super_mask_p	0.3 0.0 0.5	$P(\text{drop super-context})$: what makes the hierarchy visible to Chow-Liu.
hr_DC	6	Width of the content block.
hr_H	3 2 4	Width of a group's shared subspace; must be $< \text{hr_DC}$.
hr_sigma_c	1.2	Std of the content vector.
hr_gate_layout	contiguous interleaved shuffled	Leaf-to-gate assignment; decides if siblings are chain-adjacent.
hr_sibling_overlap	0.0 0.5 1.0	How much sibling sub-rules share (1 = indistinguishable from y).
Task: MNIST		
mnist_mode	plain routing	plain = calibration control; routing = conflicting label maps (gates buy accuracy).
mnist_classes	all 0,1,2 0,1,2,3,4	Which digits take part; sets C ('all' = 0-9).

continued on next page

Table A.2 (continued)

Parameter	Values	Meaning
mnist_downsample	1 2 4	s x s block-mean downsample (1 → 28x28, 4 → 7x7).
mnist_pca	0 30 50	PCA components kept (0 = raw pixels).
mnist_R	2 3	Routing mode: number of conflicting label maps.
mnist_ctx	3.0 0.4	Routing mode: context-block scale (3.0 clean, 0.4 ambiguous).
Task: Fashion		
fashion_classes	all 0,1,2 0,1,2,3,4	Which Fashion-MNIST classes take part; sets C.
fashion_trunk	lenet55k lenet10k none	Input regime: frozen LeNet-5 features (55k/10k images) or raw pixels.
fashion_trunk_epochs	20	Training epochs of the frozen trunk (part of its cache key).
fashion_w0_alpha	1.0 0.95 0.9 0.8 0.0	Frozen base = (1-a)*trained + a*random; a=0 collapses the mixture.
fashion_probe	letters noise rotate	Headline OOD probe: EMNIST letters, noise, or rotation.
fashion_ood_fraction	0.5 0.3 0.7 0.9	Corruption strength f of the headline OOD columns.
fashion_ood_n	1000 500 2000	Corrupted images per fraction.
fashion_n_samples	100 50 200	Gate samples per test input at prediction (Liu et al.: 100).
fashion_ood_ladder	no yes	yes = also write the full 3-probe x 10-fraction OOD ladder.
Task: C10-ResNet		
c10r_classes	all 0,1,2	Which CIFAR-10 classes take part.
c10r_trunk	resnet45k resnet10k none	Frozen ResNet trunk (45k/10k training images) or none.
c10r_trunk_epochs	100 50	Training epochs of the frozen trunk (cache key).
c10r_trunk_schedule	step cosine constant	Trunk LR schedule; 'step' is the trained reference trunk.
c10r_w0_alpha	1.0 0.9 0.5 0.0	Trained-vs-random frozen head mix; 0.0 = deterministic control row.
c10r_probe	svhn noise rotate	Headline OOD probe: SVHN, noise, or rotation.
c10r_ood_fraction	0.5 0.3 0.7 0.9	Corruption strength of the headline OOD columns.
c10r_ood_n	1000 500 2000	OOD images per fraction.
c10r_n_samples	100 50 200	Gate samples per test input at prediction.
c10r_ood_ladder	no yes	yes = also write the full OOD fraction ladder.
Task: CIFAR		
cifar_mode	plain routing	plain = calibration control; routing = conflicting label maps.
cifar_classes	all 0,1,2 0,1,8,9	Which classes take part ('all' or a comma list).
cifar_features	rconv raw pca	Input basis: rconv raw pca – sets the linear-model ceiling.
cifar_gray	no yes	Collapse RGB to luminance first (colour ablation).
cifar_downsample	1 2 4	s x s block-mean pooling within each channel.
cifar_pca	0 200 500	PCA components kept when features=pca (0 = off).

continued on next page

Table A.2 (continued)

Parameter	Values	Meaning
cifar_filters	200 400 64	rconv: number of random filters; $D = \text{filters} * 2 * \text{pool}^2$.
cifar_patch	6 4 8	rconv: side length of the random square patches.
cifar_pool	2 3 4	rconv: average-pooling grid over each response map.
cifar_feat_seed	0 1 2	rconv: seed of the random filter bank.
cifar_R	2 3	Routing mode: number of conflicting label maps (derangements).
cifar_ctx	3.0 0.4	Routing mode: context-block scale.

A.14 Pipeline Implementation Flow Chart

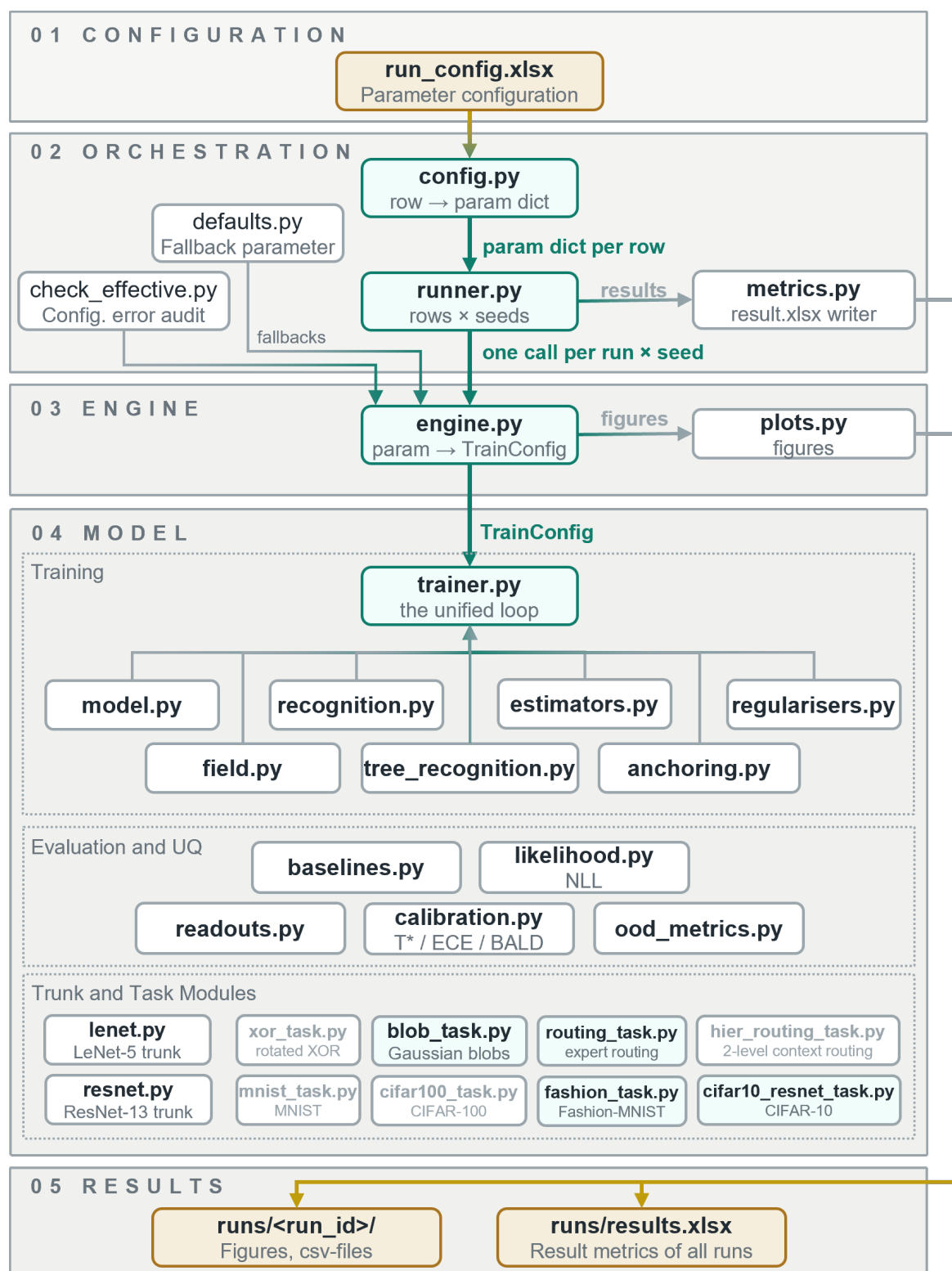


Figure A.3: Pipeline implementation architecture

A.15 Example Images under Distribution Shift

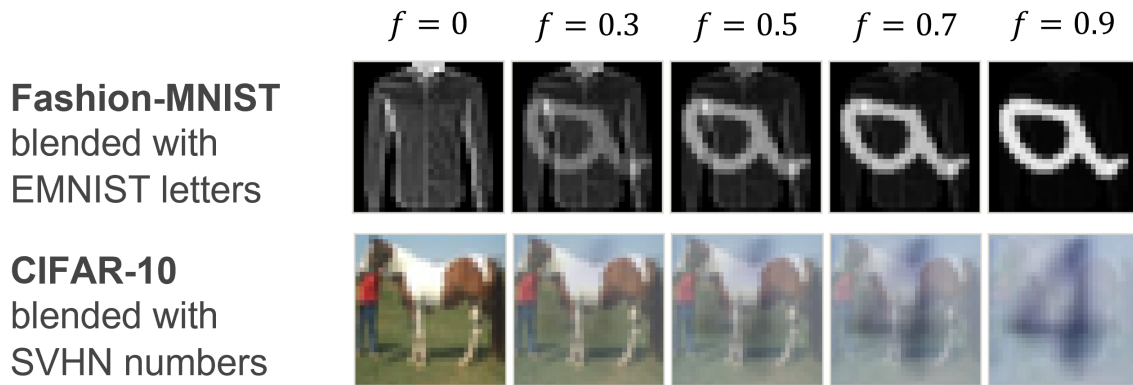


Figure A.4: Examples of the out-of-distribution blends of Equation (5.1) for both datasets. Fashion-MNIST is blended with EMNIST and CIFAR-10 with SVHN.

A.16 Dataset Summary

	Fashion-MNIST	CIFAR-10
Total images	70 000	60 000
Classes C	10, balanced	10, balanced
Channels	1 (greyscale)	3 (RGB)
Resolution	28×28	32×32
Backbone	LeNet-5	ResNet-13
Parameters of frozen trunk	60 856	1 225 952
Parameters of replaced layer	850	2 570
Feature dimension D	85	257
Out-of-distribution probe	EMNIST letters	SVHN

Table A.3: The two dataset instantiations

A.17 Experiments Overview

ID	Question the experiment answers	Primary objective	Data
E1	Whether the mechanism works at all and what the approximate posterior over the gates has to represent	Feasibility	Artificial: blob task, routing task, designed target
E2	Whether accuracy and calibration hold up against the established baselines in distribution and under distribution shift	Competitiveness	Fashion-MNIST + EMNIST blend; CIFAR-10 + SVHN blend
E3	When the gating mechanism engages, how the model behaves under moving blend fraction and how increased expressive capacity influences the metrics	Competitiveness	Fashion-MNIST + EMNIST blend; CIFAR-10 + SVHN blend
E4	Whether the energy-based recognition family changes the results and what sampling budget it requires	Competitiveness	Fashion-MNIST, CIFAR-10
E5	How the cost of a training epoch grows and which share of it the thermodynamic hardware would absorb	Scalability	— (analytical)

Table A.4: The five experiment groups against the research objectives of Chapter 2

A.18 E1.3 Plotted values of the KL against the target distribution

J_0	$\overline{\text{corr}}_{p^*}$	mean-field	tree (chain)	tree (star / Chow-Liu)	EBM
0.25	0.047	0.0225	0.0145	0.0155	0.0000
0.5	0.073	0.0669	0.0423	0.0467	0.0000
1	0.100	0.1634	0.1022	0.1163	0.0000
1.5	0.115	0.2529	0.1603	0.1816	0.0000
2	0.126	0.3354	0.2180	0.2418	0.0000

Table A.5: Best achievable $\text{KL}(q \| p^*)$ in nats on the designed target against the inhibition strength J_0 . The second column gives the target’s mean absolute pairwise correlation.

A.19 E2: Full Parameter Configuration List

Table A.6: Hyperparameters behind the two base tables, named as in the runner configuration workbook (Table A.2).

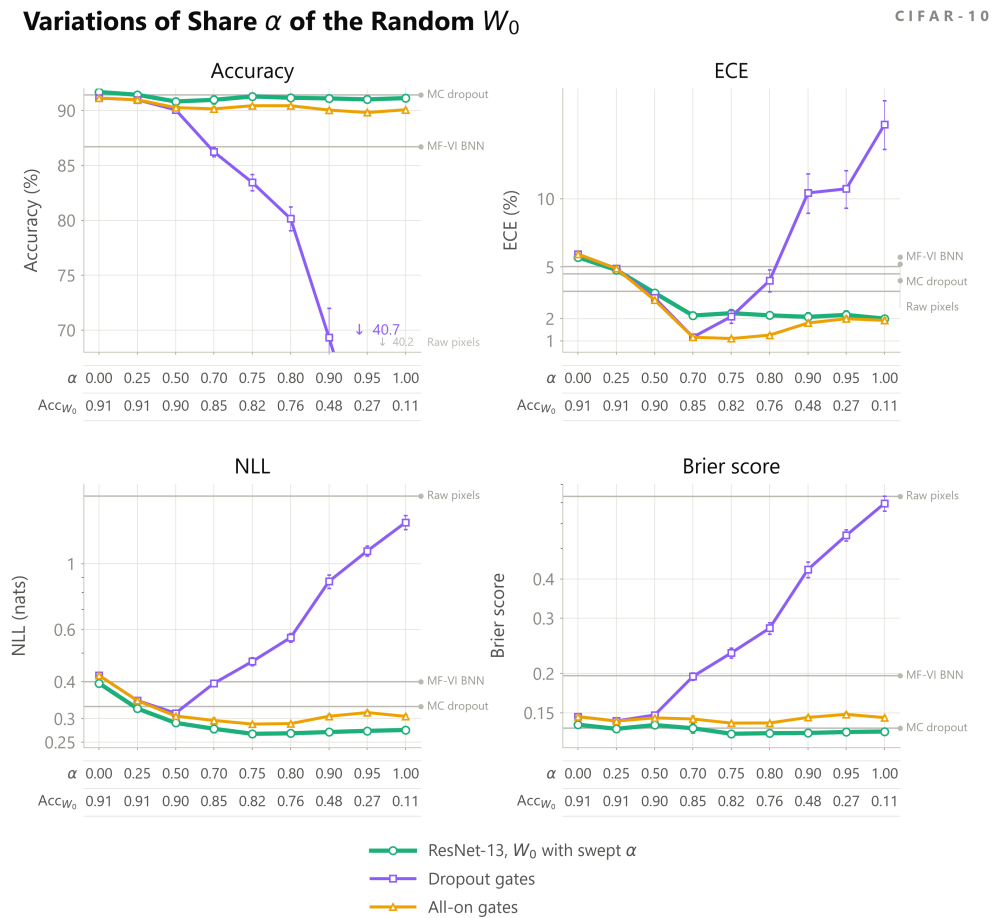
Parameter	Fashion-MNIST	CIFAR-10
Meta		
seed	0,1,...,9 (10 seeds)	0,1,...,9 (10 seeds)
Recognition (q)		
estimator	sfe-loo	sfe-loo
family	tree	tree
tree_topology	chain	chain
tree_refit	none	none
T_samples	8	8
tau	1.0 (inert under sfe-loo)	1.0 (inert under sfe-loo)
head_init_scale	0.1	0.1
c.bias	0.5	0.5
Task		
task	fashion	cifar10_resnet
n_per (per class, head only)	1000	4500
n_test	10000	10000
Base W_0 and adapters		
w0_scale	0.1	0.1
K_gates	10	10
rank	8	8
adapter_init_scale	0.1	0.1
adapter_noise	0.0	0.0
anchor_type	none (free adapters)	none (free adapters)
EBM prior (J) and field $h(x)$		
j_init	-1.5	-1.5
gibbs_sweeps_S	12	12
field_kind	mlp	mlp
field_n_hidden	24	24
field_bias_b2	0.5	0.5
Recipe (anti-collapse)		
epochs (head, full batch)	2400	2400
lr	0.1	0.1
beta_max	1.0	1.0
anneal_frac	0.4	0.4
warmup	0	0
free_bits	0.02	0.02
wd	0.01	0.01
clip	1.0	1.0
gamma0	1.0	1.0
warm_frac	0.15	0.15
label_dropout_p	0.0	0.0
Baselines		
VI BNN, KL weight	0.2	0.2

continued on next page

Table A.6 (continued)

Parameter	Fashion-MNIST	CIFAR-10
VI BNN, posterior draws	100	100
MC dropout, drop probability p	0.5	0.5
MC dropout, forward passes	100	100

A.20 E3.1 Metrics under α -Variations for CIFAR-10

Figure A.5: Accuracy, ECE, NLL and Brier score under the α -sweep on CIFAR-10

A.21 E3.1: Gate-on Rate under α -Variations for CIFAR-10

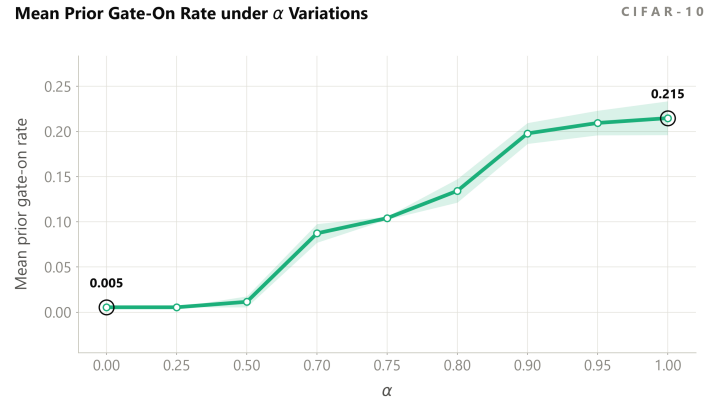


Figure A.6: Gate-on rate under the α -sweep on CIFAR-10

A.22 E3.2 Uncertainty Decomposition under f -Variations for CIFAR-10

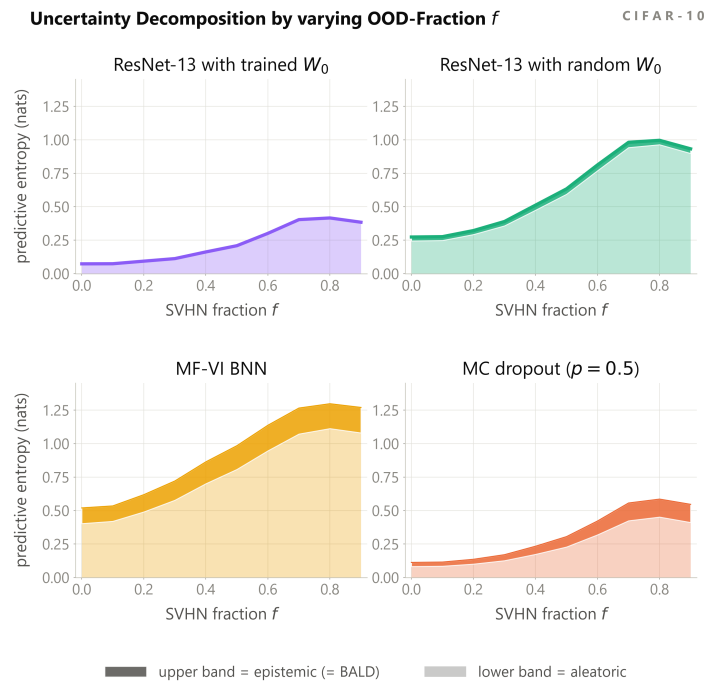


Figure A.7: Uncertainty Decomposition under the f -sweep on CIFAR-10

A.23 E3.3 Capacity Scaling Table for the $w = 0.25$ Model

Capacity Scaling Study by Varying Number of Gates K and Adapter Rank r
LeNet-5 Trunk with $w = 0.25$ (# of Parameters: 6.7k)

FASHION-MNIST

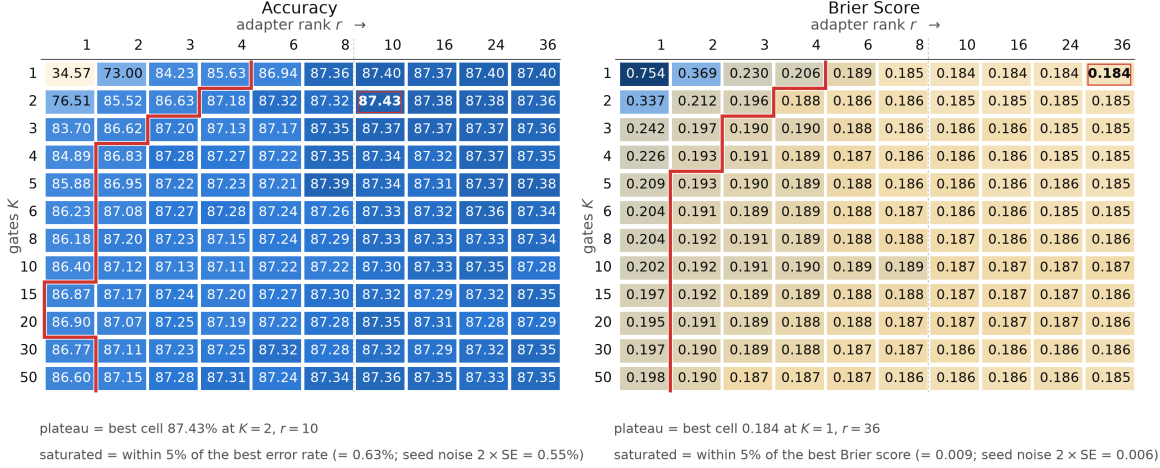


Figure A.8: Accuracy and Brier score of the mixture model with the weakened LeNet-5 trunk ($w = 0.25$) over the $K \times r$ grid

A.24 E4.1 Capacity Scaling Table using an EBM Recognition Model

Capacity Scaling Study by Varying Number of Gates K and Adapter Rank r
EBM Recognition (Gibbs, $S = 12$) — LeNet-5 Trunk with $w = 1$ (# of Parameters: 61.7k)

FASHION-MNIST

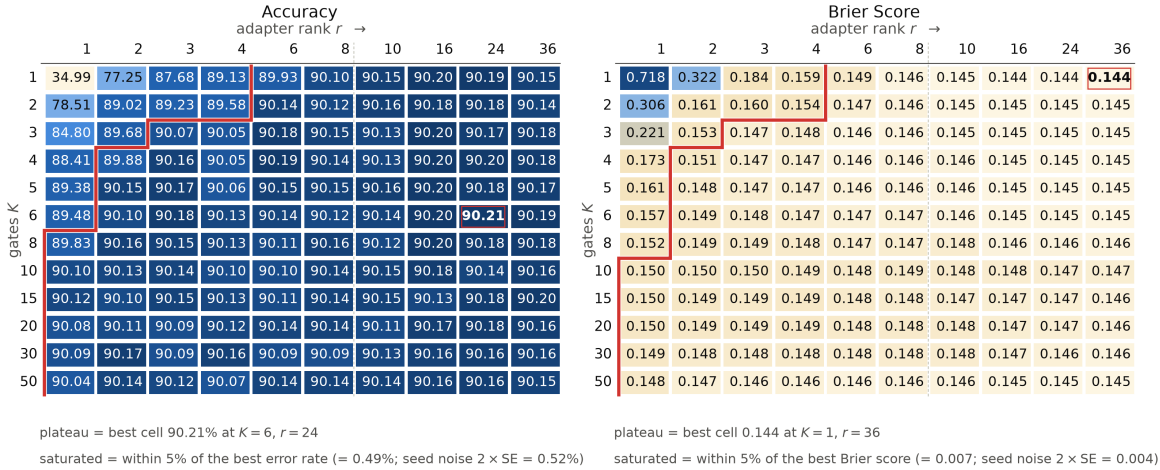


Figure A.9: Accuracy and Brier score of the mixture model with an EBM recognition model and the full LeNet-5 trunk over the $K \times r$ grid on Fashion-MNIST

A.25 FLOPs per training epoch by block

Table A.7: FLOPs per training epoch by block and group for the two recognition models

Block	Drivers	Tree recognition model	EBM recognition model
Sampling (TSU)		$(2NK^2 + 1.3\tau NK) S$	$(2NK^2 + 1.3\tau NK) (S + (n_c + T) S_q)$
Prior Gibbs sweeps	S, N, K	$2SNK^2 + 1.3\tau SNK$	$2SNK^2 + 1.3\tau SNK$
Moment chains	n_c, S_q, N, K		$2n_c S_q NK^2 + 1.3\tau n_c S_q NK$
Estimator sample chains	T, S_q, N, K		$2TS_q NK^2 + 1.3\tau TS_q NK$
Moments and KL		$0.95NK^3 + 27NK^2 + 10NK$	$23.5 n_c N n_{\text{pre}} + 1.3\tau n_c NK + 9NK^2 + 10NK$
Exact moment table	N, K	$18NK^2 + 0.95NK^3$	
Sufficient statistics over chains	$n_c, N, n_{\text{pre}}, K$		$23.5 n_c N n_{\text{pre}} + 1.3\tau n_c NK$
KL quadratic term and free bits	N, K	$4NK^2 + 10NK$	$4NK^2 + 10NK$
Prior correlations $\langle zz^\top \rangle$	N, K	$2NK^2$	$2NK^2$
Coupling update	N, K	$3NK^2$	$3NK^2$
Estimator		$TN(6CKr + (10 + 4\tau_s)K + 5C + 6)$	$TN(6CKr + 5C + 6) + 4TNn_{\text{pre}}$
Gate sampling	T, N, K	$(4 + 2\tau_s) TNK$	
Recognition score log q	T, N, K or n_{pre}	$(6 + 2\tau_s) TNK$	$4TNn_{\text{pre}}$
Writers $B a(x)$	T, N, C, K, r	$6TNCKr$	$6TNCKr$
Softmax and label pick	T, N, C	$5TNC$	$5TNC$
Leave-one-out baseline	T, N	$6TN$	$6TN$
Input dimension		$4NH_f(D + K) + 4NKrD + 2NDC + 4N(D + C) n_{\text{pre}}$	
Field network $h(x)$	N, H_f, D, K	$4NH_f(D + K)$	$4NH_f(D + K)$
Readers $a = Ax$	N, K, r, D	$4NKrD$	$4NKrD$
Base logits W_0x	N, D, C	$2NDC$	$2NDC$
Recognition head pre = U feat + c	N, D, C, n_{pre}	$4N(D + C)(2K - 1)$	$4N(D + C) K(K + 1)/2$
Other		$8P_{\text{params}} + 6NKC$	
Parameter update, clipping, decay	P_{params}	$8P_{\text{params}}$	$8P_{\text{params}}$
Class-target warm-up ($\gamma_0 > 0$)	N, K, C	$6NKC$	$6NKC$